Tax System Data Warehouse
Based on Data Vault 2.0 Methodology
with Medallion Architecture

SSIS Package Design & Implementation Guide

`Master_Complete_Pipeline.dtsx` | `Master_Full_Load.dtsx` | `Master_Incremental_Load.dtsx`

SSIS 2022 | SQL Server 2025 (v17) | Medallion Architecture | Full & Incremental Pipelines

9	23	6	11	8
Staging Tables	Bronze Objects	Silver Objects	Gold Objects	Control Tables

Version 1.0 | February 2026

Prepared by
Mr. Sot So
Master of Science in Data Science and Engineering
Faculty of Engineering, Royal University of Phnom Penh

Supervisor: Mr. Chap Chanpiseth

© 2026 Mr. Sot So. All rights reserved.
A final report submitted in partial fulfillment of the requirements for the degree of Master of Science in Data Science and Engineering, Royal University of Phnom Penh.

CONFIDENTIAL

SSIS PACKAGE ORCHESTRATION Master Pipeline Hierarchy

SQL Server Agent Job , Scheduled Trigger (Daily / Weekly)

|

triggers

▼

ENTRY POINT

Master_Complete_Pipeline.dtsx

Owns batch lifecycle , SQL_StartMasterBatch → SCR_ValidateAndInitialize → EXT_EvaluateLoadType → EPT_* → SCR_SetBatchSuccess → SQL_EndMasterBatch

`BatchID ← OUTPUT  LoadType (FULL | INCREMENTAL)  v_SnapshotDate`

Step 3: EXT_EvaluateLoadType → Precedence Constraints branch by pp_LoadType

▼ **pp_LoadType == "FULL"**

Master_Full_Load.dtsx

Receives: BatchID, LoadType='FULL', SnapshotDate

1. **STG_Full_Load_All.dtsx** ← *BatchID + LoadType*
2. **BRZ_Load_All_Hubs.dtsx** ← *p_BatchID*
3. **BRZ_Load_All_Satellites.dtsx** ← *p_BatchID*
4. **BRZ_Load_All_Links.dtsx** ← *p_BatchID*
5. **SLV_Load_All.dtsx** ← *BatchID + SnapshotDate*
6. **GLD_Load_All_Dimensions.dtsx** ← *p_BatchID*
7. **GLD_Load_All_Facts.dtsx** ← *BatchID + SnapshotDate*

STG_Full_Load_All → 9 child packages

`STG_Category  STG_Structure  STG_Activity  STG_Taxpayer  STG_Owner  STG_Officer  STG_MonthlyDecl  STG_AnnualDecl  STG_Payment`

Each receives: BatchID + LoadType="FULL"

▼ **pp_LoadType == "INCREMENTAL"**

Master_Incremental_Load.dtsx

Receives: BatchID, LoadType='INCREMENTAL', SnapshotDate

1. **STG_Incremental_Load_All.dtsx** ← *BatchID + LoadType*
2. **BRZ_Load_All_Hubs.dtsx** ← *p_BatchID*
3. **BRZ_Load_All_Satellites.dtsx** ← *p_BatchID*
4. **BRZ_Load_All_Links.dtsx** ← *p_BatchID*
5. **SLV_Load_All.dtsx** ← *BatchID + SnapshotDate*
6. **GLD_Load_All_Dimensions.dtsx** ← *p_BatchID*
7. **GLD_Load_All_Facts.dtsx** ← *BatchID + SnapshotDate*

STG_Incremental_Load_All → 9 child packages

`STG_Category  STG_Structure  STG_Activity  STG_Taxpayer  STG_Owner  STG_Officer  STG_MonthlyDecl  STG_AnnualDecl  STG_Payment`

Each receives: BatchID + LoadType="INCREMENTAL" — Uses ETL_Watermark for delta

▼ *Both paths feed the same data layer sequence below* ▼

© 2026 Mr. Sot So — Royal University of Phnom Penh

DATA LAYER ARCHITECTURE

Source → Staging → Bronze → Silver → Gold

SOURCE

TaxSystemDB — 9 Tables

Category Structure Activity Taxpayer Owner Officer MonthlyDeclaration AnnualDeclaration Payment

FULL: Truncate + Reload | INCREMENTAL: Watermark delta

STAGING

DV_Staging — 9 Tables

STG_Category STG_Structure STG_Activity STG_Taxpayer STG_Owner STG_Officer STG_MonthlyDeclaration STG_AnnualDeclaration STG_Payment

Each loaded by usp_Load_STG_*(@BatchID, @LoadType) with usp_StartStep / usp_EndStep

Hash business keys + insert-only load

BRONZE, Data Vault 2.0

DV_Bronze — 23 Objects (9 Hubs + 9 Satellites + 5 Links)

HUBS	SATELLITES	LINKS
HUB_Category	SAT_Category	LNK_TaxpayerDeclaration
HUB_Structure	SAT_Structure	LNK_DeclarationPayment
HUB_Activity	SAT_Activity	LNK_TaxpayerOfficer
HUB_Taxpayer	SAT_Taxpayer	LNK_TaxpayerOwner
HUB_Owner	SAT_Owner	LNK_TaxpayerAnnualDecl
HUB_Officer	SAT_Officer	
HUB_Declaration	SAT_MonthlyDecl	
HUB_Payment	SAT_Payment	
HUB_AnnualDecl	SAT_AnnualDecl	

PIT snapshots + Bridge + Business rules

SILVER, Business Vault

DV_Silver — 6 Objects (3 PITs + 1 Bridge + 2 Business)

PIT TABLES	BRIDGE	BUSINESS VAULT
PIT_Taxpayer	BRG_Taxpayer_Owner	BUS_ComplianceScore
PIT_Declaration		BUS_MonthlyMetrics
PIT_Payment		

Star schema dimensional transformation

© 2026 Mr. Sot So — Royal University of Phnom Penh

▼

★ GOLD — Star Schema

DV_Gold – 11 Objects (7 Dimensions + 4 Facts)

DIMENSIONS (SCD 1 & 2)

- DIM_Taxpayer (SCD2)
- DIM_Officer (SCD2)
- DIM_Category
- DIM_Structure
- DIM_Activity
- DIM_PaymentMethod
- DIM_Status

FACT TABLES

- FACT_MonthlyDeclaration
- FACT_Payment
- FACT_MonthlySnapshot
- FACT_DeclarationLifecycle

CONFIDENTIAL

ETL CONTROL FRAMEWORK Batch Tracking • Watermarks • Error Logging

ETL_BatchLog *One row per pipeline run*	**ETL_StepLog** *One row per procedure execution*
ETL_ErrorLog *Error capture from SSIS OnError handlers (BatchID, ErrorSeverity, ErrorSource, ErrorMessage)*	**ETL_Watermark** *Delta tracking per table (incremental)*
ETL_Process *Process registry with source/target mappings*	

BATCHID LIFECYCLE Created once • Passed everywhere • Closed once

SQL_Start MasterBatch *Creates v_BatchID*	SCR_Validate AndInitialize *Validates + sets DateTime*	EXT_Evaluate LoadType *Routes by pp_LoadType*	EPT_Master Full/Incremental *Receives p_ params*	SCR_SetBatch Success *v_BatchStatus=Success*	SQL_End MasterBatch *Closes batch*
→	→	→	→	→	

ERROR PROPAGATION Option 2: v_ErrorMessage + OnError Event Handler + Script Task

Stored Procedure (usp_Load_STG_*) TRY/CATCH usp_EndStep(Failed)	Layer Package (STG_*.dtsx) OnError → SCR_CaptureError → SQL_EndStepOnError → SQL_LogErrorOnError	Master_Full_Load / Master_Incremental _Load.dtsx OnError → SCR_CaptureError → SQL_LogErrorOnError Does NOT call SQL_EndMasterBatch	Master_Complete _Pipeline.dtsx OnError → SCR_CaptureError → SQL_EndBatchOnError → SQL_LogErrorOnError **OWNS BATCH LIFECYCLE**
→	→	→	

CONFIDENTIAL

1. Architecture Overview

The master orchestration layer consists of three SSIS packages that form a strict parent-child hierarchy. Master_Complete_Pipeline.dtsx is the single entry point that owns the entire batch lifecycle. It creates the BatchID and passes it down to one of two child master packages depending on the chosen load strategy. Those child master packages in turn call into the layer-specific orchestrator packages (STG_Full_Load_All.dtsx, STG_Incremental_Load_All.dtsx, BRZ_Load_All_Hubs.dtsx, etc.).

Technology Stack

Component	Version / Detail
Database Engine	SQL Server 2025 (v17.0), RTM 17.0.1000.7
ETL Platform	SQL Server Integration Services (SSIS) 2022
Development IDE	Visual Studio 2026 Community + SSIS Projects extension
Management Tool	SQL Server Management Studio (SSMS) 2022
Data Vault Methodology	Data Vault 2.0 (Hub / Satellite / Link / PIT / Bridge / Business Vault)
Architecture Pattern	Medallion Architecture (Staging → Bronze → Silver → Gold)
Hash Algorithm	HASHBYTES('SHA2_256', ...), VARBINARY(32) hash keys
Deployment Target	SSISDB Catalog (Project Deployment Model)
Scheduling	SQL Server Agent Job

1.1 Package Hierarchy

Package	Role	Owned By
Master_Complete_Pipeline.dtsx	Entry point, batch lifecycle owner	SQL Agent Job
Master_Full_Load.dtsx	Full-load orchestration (all 4 layers)	Master_Complete_Pipeline
Master_Incremental_Load.dtsx	Incremental-load orchestration (all 4 layers)	Master_Complete_Pipeline
STG_Full_Load_All.dtsx	Full load for all 9 staging tables	Master_Full_Load
STG_Incremental_Load_All.dtsx	Incremental load for all 9 staging tables	Master_Incremental_Load
BRZ_Load_All_Hubs.dtsx	Load all 9 Bronze Hubs	Master_Full/Incremental
BRZ_Load_All_Satellites.dtsx	Load all 9 Bronze Satellites	Master_Full/Incremental
BRZ_Load_All_Links.dtsx	Load all 5 Bronze Links	Master_Full/Incremental
SLV_Load_All.dtsx	Load Silver PIT, Bridge, Business tables	Master_Full/Incremental
GLD_Load_All_Dimensions.dtsx	Load all 7 Gold Dimensions	Master_Full/Incremental
GLD_Load_All_Facts.dtsx	Load all 4 Gold Facts	Master_Full/Incremental

© 2026 Mr. Sot So — Royal University of Phnom Penh

1.2 BatchID Flow

The BatchID is the single most important value in the entire pipeline. It is created once by Master_Complete_Pipeline.dtsx and propagated through every child package so that all ETL_StepLog and ETL_ErrorLog entries can be traced back to the same execution run.

Key Design Principle

Master_Complete_Pipeline.dtsx OWNS the batch lifecycle (usp_StartBatch / usp_EndBatch).
Child packages (Master_Full_Load, Master_Incremental_Load) NEVER create their own batch.
All packages receive BatchID as a parent package variable via parameter binding.
Staging packages also receive LoadType ('FULL' or 'INCREMENTAL').

1.3 Naming Conventions

All SSIS objects follow a strict naming convention with standardized prefixes. This ensures consistency across the project and makes the control flow self-documenting.

Task Prefixes

Prefix	Task Type	Example
SQL_	Execute SQL Task	SQL_StartMasterBatch
SCR_	Script Task	SCR_ValidateAndInitialize
EXT_	Expression Task	EXT_EvaluateLoadType
EPT_	Execute Package Task	EPT_Master_Full_Load
DFT_	Data Flow Task	DFT_Load_STG_Category
SEQC_	Sequence Container	SEQC_BronzeLayer
FELC_	Foreach Loop Container	FELC_IterateFiles
FLC_	For Loop Container	FLC_RetryLoop
FSYS_	File System Task	FSYS_ArchiveFiles
SMTP_	Send Mail Task	SMTP_NotifyFailure

Variable & Parameter Prefixes

Prefix	Scope	Example
v_	User Variable (package-scoped)	User::v_BatchID, User::v_SnapshotDate
p_	Package Parameter (passed between packages)	$Package::p_BatchID, $Package::p_LoadType

© 2026 Mr. Sot So — Royal University of Phnom Penh

pp_	Project Parameter (shared across all packages)	$Project::pp_LoadType, $Project::pp_ServerName
(none)	System Variable (read-only)	System::ErrorDescription, System::PackageName

Connection Manager Prefixes

Prefix	Type	Example
CM_	OLE DB Connection	CM_ETL_Control, CM_ETL_Control_OnError, CM_TaxSystemDB, CM_DV_Staging, CM_DV_Bronze, CM_DV_Silver, CM_DV_Gold

Package File Naming

Pattern	Layer	Example
Master_*.dtsx	Master orchestration	Master_Complete_Pipeline.dtsx
STG_*.dtsx	Staging layer	STG_Full_Load_All.dtsx, STG_Category.dtsx
BRZ_*.dtsx	Bronze layer	BRZ_Load_All_Hubs.dtsx
SLV_*.dtsx	Silver layer	SLV_Load_All.dtsx
GLD_*.dtsx	Gold layer	GLD_Load_All_Dimensions.dtsx

Naming Convention Benefits

Instantly identify object type from the prefix, no need to open the editor.
Alphabetical sorting groups related objects together in Solution Explorer.
OnError event handlers can parse System::SourceName to categorize failures.
Consistent naming makes code reviews and documentation significantly easier.

© 2026 Mr. Sot So — Royal University of Phnom Penh

2. Project Parameters

Project parameters are defined at the SSIS project level (Project.params) and shared across all packages in the project. They are the primary mechanism for environment-specific configuration and SQL Agent Job overrides.

2.1 Core Parameters

Parameter Name	Data Type	Default Value	Purpose
$Project::pp_LoadType	String	FULL	Controls FULL vs INCREMENTAL, overridden by SQL Agent Job per schedule
$Project::pp_Environment	String	DEV	DEV / UAT / PROD, used in logging, alerts, and connection routing

2.2 Connection Parameters

Connection parameters allow a single SSISDB deployment to work across DEV, UAT, and PROD without repackaging. Connection managers use expressions that reference these project parameters.

Parameter Name	Data Type	Default Value	Purpose
$Project::pp_ServerName	String	localhost	SQL Server instance name, changes per environment
$Project::pp_ETLControlDB	String	ETL_Control	ETL control/metadata database name
$Project::pp_TaxSystemDB	String	TaxSystemDB	Source system database name

Connection Manager Expressions

Set these expressions on each project connection manager's ConnectionString property:

CM_ETL_Control (OLE DB)

```
"Data Source=" + @[$Project::pp_ServerName]
+ ";Initial Catalog=" + @[$Project::pp_ETLControlDB]
+ ";Provider=SQLOLEDB.1;Integrated Security=SSPI;"
```

CM_ETL_Control_OnError (OLE DB)

Duplicate of CM_ETL_Control with identical ConnectionString. Used exclusively by OnError event handler tasks to ensure error logging commits independently of the parent task's transaction scope.

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
"Data Source=" + @[$Project::pp_ServerName]
+ ";Initial Catalog=" + @[$Project::pp_ETLControlDB]
+ ";Provider=SQLOLEDB.1;Integrated Security=SSPI;"
```

Why a Separate Connection Manager for OnError?

When a task fails inside a transaction, SSIS rolls back ALL work on that connection, including OnError handler writes.

Even with TransactionOption = NotSupported on OnError tasks, the shared CM_ETL_Control connection is already enlisted in the parent's transaction.

A separate connection (CM_ETL_Control_OnError) creates an independent database session that is never enlisted in any parent transaction.

Result: SQL_EndStepOnError and SQL_LogErrorOnError always commit, even when the parent task rolls back.

CM_TaxSystemDB (OLE DB)

```
"Data Source=" + @[$Project::pp_ServerName]
+ ";Initial Catalog=" + @[$Project::pp_TaxSystemDB]
+ ";Provider=SQLOLEDB.1;Integrated Security=SSPI;"
```

CM_DV_Staging / CM_DV_Bronze / CM_DV_Silver / CM_DV_Gold (OLE DB)

Same pattern, replace Initial Catalog with the corresponding database name:

```
"Data Source=" + @[$Project::pp_ServerName]
+ ";Initial Catalog=DV_Staging;"  -- or DV_Bronze, DV_Silver, DV_Gold
+ ";Provider=SQLOLEDB.1;Integrated Security=SSPI;"
```

2.3 Logging & Alert Parameters

Parameter Name	Data Type	Default Value	Purpose
$Project::pp_LogLevel	String	INFO	INFO / DEBUG / ERROR, controls logging verbosity in stored procedures
$Project::pp_EnableAlerts	Boolean	True	Toggle email/alert notifications on failure
$Project::pp_AlertEmail	String	etl-team@company.com	Recipient email address for failure notifications

Project vs Package vs User Variables

Project Parameters (pp_), Shared across ALL packages, overridable at deployment/execution. Use for environment config, load strategy, logging.

Package Parameters (p_), Passed from parent to child packages via Execute Package Task bindings. Use for BatchID, LoadType, SnapshotDate propagation.

© 2026 Mr. Sot So — Royal University of Phnom Penh

User Variables (v_), Runtime state within a single package. Use for BatchID (generated), BatchStatus, ErrorMessage, SnapshotDate.

2.4 How pp_LoadType Flows Through the Pipeline

The project parameter pp_LoadType is the single source of truth for the load strategy. It flows into the pipeline as follows:

1. **SQL Agent Job** overrides $Project::pp_LoadType at execution (FULL or INCREMENTAL)
2. **Master_Complete_Pipeline.dtsx** reads @[$Project::pp_LoadType] via v_LoadType expression
3. **EXT_EvaluateLoadType** → Precedence constraints evaluate @[$Project::pp_LoadType]
4. **EPT_Master_Full_Load / EPT_Master_Incremental_Load** receives pp_LoadType as p_LoadType via parameter binding
5. **Child packages** propagate p_LoadType down to staging procedures

v_LoadType Expression (Master_Complete_Pipeline.dtsx)

In the Variables pane, set an expression on User::v_LoadType:

```
@[$Project::pp_LoadType]
```

This ensures v_LoadType always reflects the project parameter value at runtime, whether set by default or overridden by SQL Agent.

Environment-Specific Overrides

After deploying to SSISDB, configure project parameters per environment using:

SSISDB → Right-click project → Configure → Set pp_ServerName, pp_ETLControlDB, pp_TaxSystemDB

Or use T-SQL: EXEC [SSISDB].[catalog].[set_object_parameter_value] ...

This eliminates the need to maintain separate .dtsx packages per environment.

CONFIDENTIAL

3. Staging Packages, Detailed Implementation

The staging layer is the entry point for source data into the Data Vault pipeline. It extracts data from TaxSystemDB (source) and lands it in DV_Staging using a consistent pattern across all 9 entities. Two orchestrator packages (STG_Full_Load_All.dtsx and STG_Incremental_Load_All.dtsx) coordinate the execution, while 9 individual child packages perform the actual data movement.

3.1 Package Inventory

Orchestrator Packages (01_Staging folder)

Package	Called By	Purpose
STG_Full_Load_All.dtsx	Master_Full_Load.dtsx	Orchestrates TRUNCATE + full extract for all 9 tables
STG_Incremental_Load_All.dtsx	Master_Incremental_Load.dtsx	Orchestrates watermark-based delta extract for all 9 tables

Child Packages (01_Staging folder)

Package	Source Table	Staging Table	Busin
STG_Category.dtsx	TaxSystemDB.dbo.Category	DV_Staging.dbo.STG_Category	Categ
STG_Structure.dtsx	TaxSystemDB.dbo.Structure	DV_Staging.dbo.STG_Structure	Struct
STG_Activity.dtsx	TaxSystemDB.dbo.Activity	DV_Staging.dbo.STG_Activity	Activit
STG_Taxpayer.dtsx	TaxSystemDB.dbo.Taxpayer	DV_Staging.dbo.STG_Taxpayer	TaxID
STG_Owner.dtsx	TaxSystemDB.dbo.Owner	DV_Staging.dbo.STG_Owner	Owne
STG_Officer.dtsx	TaxSystemDB.dbo.Officer	DV_Staging.dbo.STG_Officer	Office
STG_MonthlyDeclaration.dtsx	TaxSystemDB.dbo.MonthlyDeclaration	DV_Staging.dbo.STG_MonthlyDeclaration	Decla
STG_AnnualDeclaration.dtsx	TaxSystemDB.dbo.AnnualDeclaration	DV_Staging.dbo.STG_AnnualDeclaration	Annua
STG_Payment.dtsx	TaxSystemDB.dbo.Payment	DV_Staging.dbo.STG_Payment	Paym

© 2026 Mr. Sot So — Royal University of Phnom Penh

3.2 Staging Table Design

Every staging table follows a consistent pattern: source columns + ETL metadata columns. Staging tables have NO primary keys, foreign keys, or hash columns — they are pure landing zones. Hash computation is deferred to the Bronze layer.

ETL Metadata Columns (present in every staging table)

Column	Data Type	Nullable	Purpose
SourceSystem	NVARCHAR(50)	YES	Source system identifier. Embedded in the OLE DB Source SQL query as N'TaxSystemDB'.
RecordSource	NVARCHAR(50)	YES	ETL record source tag. Embedded in the OLE DB Source SQL query as N'TaxSystemDB.dbo.<Table>'.
LoadDateTime	DATETIME	YES	Timestamp when the record was staged. Embedded in SQL query via GETDATE().

Metadata Approach: SQL Command from Variable (Expression-Based)

All ETL metadata columns (SourceSystem, RecordSource, LoadDateTime) are embedded in a dynamically built SQL string via SSIS expression variables (v_SQL_FullLoad, v_SQL_IncrementalLoad).

This eliminates the need for a Derived Column transformation in the Data Flow pipeline.

The OLE DB Source uses 'SQL command from variable' access mode to dynamically build the full SQL string.

The N'...' prefix ensures NVARCHAR output matching the NVARCHAR(50) staging table columns.

This approach produces a cleaner 3-component data flow: OLE_SRC → RCT_CountRows → OLE_DST.

All 9 packages share the same structure; only variable default values and the SQL column list differ. The FROM clause uses @[User::v_TableName] for dynamic table name resolution.

Example: STG_Category (actual DDL)

```
CREATE TABLE DV_Staging.dbo.STG_Category (
    -- Source columns (from TaxSystemDB.dbo.Category)
    CategoryID          INT             NULL,
    CategoryName        VARCHAR(100)    NULL,
    CategoryDescription VARCHAR(500)    NULL,
    IsActive            BIT             NULL,
    CreatedDate         DATETIME        NULL,
    UpdatedDate         DATETIME        NULL,

    -- ETL metadata columns (embedded in OLE DB Source SQL query)
    SourceSystem        NVARCHAR(50)    NULL,
    RecordSource        NVARCHAR(50)    NULL,
    LoadDateTime     DATETIME        NULL,
);
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

No Hash Columns in Staging

Staging tables do NOT contain HashKey or HashDiff columns.
Hash computation (HASHBYTES) is performed at the Bronze layer during Hub/Satellite loading.
This keeps the staging layer simple: extract from source, add metadata, land in staging.
Bronze stored procedures compute hashes on-the-fly when inserting into Hub and Satellite tables.

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

3.3 STG_Full_Load_All.dtsx, Orchestrator

This package receives BatchID and LoadType from Master_Full_Load.dtsx and executes all 9 staging child packages in sequence.

Package Parameters (received from parent)

Parameter	Data Type	Passed From	Purpose
$Package::p_BatchID	Int32	User::v_BatchID	Current batch identifier
$Package::p_LoadType	String	User::v_LoadType	Always 'FULL' for this package
$Package::p_SnapshotDate	DateTime	User::v_SnapshotDate	Pipeline execution timestamp

Control Flow, 9 Execute Package Tasks (Sequential)

Step	Task	Parameters Passed to Child
1	EPT_STG_Category (Execute Package Task)	p_BatchID, p_LoadType
2	EPT_STG_Structure (Execute Package Task)	p_BatchID, p_LoadType
3	EPT_STG_Activity (Execute Package Task)	p_BatchID, p_LoadType
4	EPT_STG_Taxpayer (Execute Package Task)	p_BatchID, p_LoadType
5	EPT_STG_Owner (Execute Package Task)	p_BatchID, p_LoadType
6	EPT_STG_Officer (Execute Package Task)	p_BatchID, p_LoadType
7	EPT_STG_MonthlyDeclaration (Execute Package Task)	p_BatchID, p_LoadType
8	EPT_STG_AnnualDeclaration (Execute Package Task)	p_BatchID, p_LoadType
9	EPT_STG_Payment (Execute Package Task)	p_BatchID, p_LoadType

Parameter Bindings (each EPT task)

Child Parameter	Bind to Variable/Parameter	Direction	Data Type
$Package::p_BatchID	$Package::p_BatchID	Input	Int32
$Package::p_LoadType	$Package::p_LoadType	Input	String

© 2026 Mr. Sot So — Royal University of Phnom Penh

Sequential Execution Order

Staging packages run sequentially (not parallel) to avoid lock contention on the source database.

Reference tables first (Category, Structure, Activity), then transactional entities (Taxpayer, Owner, Officer), then declarations and payments.

If source DB can handle concurrency, consider grouping independent tables in parallel Sequence Containers.

3.4 STG_Incremental_Load_All.dtsx, Orchestrator

Identical structure to STG_Full_Load_All.dtsx. The only difference is that p_LoadType = 'INCREMENTAL', which causes child packages to use watermark-based delta extraction instead of TRUNCATE + full extract.

Differences from STG_Full_Load_All.dtsx

Aspect	STG_Full_Load_All	STG_Incremental_Load_All
p_LoadType value	FULL	INCREMENTAL
Staging table prep	TRUNCATE before load	No truncate, append delta
Source query	SELECT * (all rows)	WHERE UpdatedDate > @LastWatermark
Watermark update	Not needed (full reload)	usp_UpdateWatermark after success
Called by	Master_Full_Load.dtsx	Master_Incremental_Load.dtsx

Why Two Separate Orchestrators?

Both orchestrators call the SAME 9 child packages (STG_Category.dtsx, etc.).

The child packages receive p_LoadType and branch internally to handle FULL vs INCREMENTAL.

Separate orchestrators allow different scheduling (Full = weekly, Incremental = nightly).

Alternative: Use a single STG_Load_All.dtsx that receives p_LoadType, this is also valid.

© 2026 Mr. Sot So — Royal University of Phnom Penh

3.5 Child Package Template (STG_*.dtsx)

All 9 staging child packages follow the same design pattern. This section documents the template using STG_Category.dtsx as the reference implementation.

Package Parameters

Parameter	Data Type	Passed From	Purpose
$Package::p_BatchID	Int32	Parent EPT binding	Current batch identifier
$Package::p_LoadType	String	Parent EPT binding	FULL or INCREMENTAL

Package Variables

Variable	Data Type	Default	Purpose
User::v_StepName	String	Load_STG_Category	Step name for usp_StartStep logging (change per package)
User::v_TableName	String	Category	Source table name for watermark lookup and truncate (change per package)
User::v_SourceSystem	String	TaxSystemDB	Source system identifier, concatenated into SQL expression
User::v_RecordSource	String	TaxSystemDB.dbo.Category	Record source tag, concatenated into SQL expression (change per package)
User::v_SQL_FullLoad	String	(Expression)	Full SQL built by expression. EvaluateAsExpression = True. Used by DFT_Full_Load.
User::v_SQL_IncrementalLoad	String	(Expression)	Incremental SQL built by expression. EvaluateAsExpression = True. Used by DFT_Incremental_Load.
User::v_RowCount	Int32	0	Rows loaded (set by Row Count transformation in DFT)
User::v_ErrorMessage	String	(empty)	Error details captured by OnError handler
User::v_ErrorSource	String	(Expression)	Auto-set via expression: @[System::PackageName]. No manual change needed.
User::v_StepID	Int32	0	Step tracking ID from usp_StartStep
User::v_LastWatermark	DateTime	1900-01-01	Last successful watermark for INCREMENTAL delta extraction

© 2026 Mr. Sot So — Royal University of Phnom Penh

Expression Variables, v_SQL_FullLoad and v_SQL_IncrementalLoad

These two variables have EvaluateAsExpression = True. They dynamically build the complete SQL query string by concatenating metadata values from other variables. The OLE DB Source uses 'SQL command from variable' access mode to consume them.

v_SQL_FullLoad Expression (Category example):

```
"SELECT CategoryID, CategoryName, CategoryDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad Expression (Category example):

```
"SELECT CategoryID, CategoryName, CategoryDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

Why 'SQL command from variable' Is Required

The OLE DB Source in SSIS Data Flow cannot parse ? parameter placeholders in the SELECT column list.

? parameters only work in WHERE clauses for OLE DB Source (unlike Execute SQL Task which supports them everywhere).

Attempting to use ? in the SELECT list produces: 'Parameters cannot be extracted from the SQL command'.

Solution: Build the entire SQL string in a variable expression, then use 'SQL command from variable' access mode.

The N'...' prefix is needed because variable values are concatenated as string literals into SQL, not passed as typed parameters.

The (DT_WSTR,23) cast converts DateTime to string for concatenation in the incremental query.

Variable-Driven Package Design

All 9 staging packages share the SAME control flow and data flow structure.

Only the variable default values differ between packages: v_StepName, v_TableName, v_RecordSource.

v_SourceSystem is always 'TaxSystemDB' for all 9 packages.

The SQL expression variables (v_SQL_FullLoad, v_SQL_IncrementalLoad) use @[User::v_TableName] for the FROM clause, so only the column list needs updating per package.

© 2026 Mr. Sot So — Royal University of Phnom Penh

To set EvaluateAsExpression: In Variables pane, click the variable → in Properties (F4) set EvaluateAsExpression = True, then enter the expression in Expression property.

Connection Managers Used

Name	Type	Database	Used By
CM_TaxSystemDB	OLE DB	TaxSystemDB	Source: OLE DB Source in Data Flow
CM_DV_Staging	OLE DB	DV_Staging	Destination: OLE DB Destination in Data Flow
CM_ETL_Control	OLE DB	ETL_Control	Step logging: usp_StartStep / usp_EndStep
CM_ETL_Control_OnError	OLE DB	ETL_Control	OnError handler: usp_EndStep / usp_LogError

3.6 Control Flow Design (STG_Category.dtsx)

The control flow uses two separate Data Flow Tasks — one for FULL load and one for INCREMENTAL load — each using 'SQL command from variable' access mode with expression-built SQL strings (v_SQL_FullLoad, v_SQL_IncrementalLoad). Expression-based precedence constraints route execution to the correct path based on p_LoadType. All task-level SQL uses variable parameters (v_StepName, v_TableName) instead of static values, making all 9 packages structurally identical.

Step	Task	Description
1	SQL_StartStep (Execute SQL Task)	Calls usp_StartStep with v_StepName. Maps OUTPUT to v_StepID.
2a	SQL_TruncateStaging (Execute SQL Task)	TRUNCATE TABLE STG_<v_TableName>. FULL path only. Expression-based SQL.
2b	SQL_GetWatermark (Execute SQL Task)	Gets last watermark for v_TableName into v_LastWatermark. INCREMENTAL path only.
3a	DFT_Full_Load (Data Flow Task)	FULL path: SQL from v_SQL_FullLoad variable → count rows → load to staging.
3b	DFT_Incremental_Load (Data Flow Task)	INCREMENTAL path: SQL from v_SQL_IncrementalLoad variable (includes WHERE clause) → count rows → load to staging.
4	SQL_UpdateWatermark (Execute SQL Task)	Updates ETL_Watermark for v_TableName. INCREMENTAL path only.
5	SQL_EndStep (Execute SQL Task)	Calls usp_EndStep('Success', @RowCount).

© 2026 Mr. Sot So — Royal University of Phnom Penh

Control Flow Diagram

```
SQL_StartStep
    ├── [FULL] ──→ SQL_TruncateStaging ──→ DFT_Full_Load ──────────────┐
    │                                                                  ├─→ SQL_EndStep
    └── [INCR] ──→ SQL_GetWatermark ──→ DFT_Incremental_Load ──┘
                                                  │
                                                  └─→ SQL_UpdateWatermark ─→ SQL_EndStep
```

Precedence Constraint Summary

From → To	Evaluation	Expression / Constraint
SQL_StartStep → SQL_TruncateStaging	Expression	@[$Package::p_LoadType] == "FULL"
SQL_StartStep → SQL_GetWatermark	Expression	@[$Package::p_LoadType] == "INCREMENTAL"
SQL_TruncateStaging → DFT_Full_Load	Constraint (Success)	Green arrow
SQL_GetWatermark → DFT_Incremental_Load	Constraint (Success)	Green arrow
DFT_Full_Load → SQL_EndStep	Constraint (Success)	Green arrow
DFT_Incremental_Load → SQL_UpdateWatermark	Constraint (Success)	Green arrow
SQL_UpdateWatermark → SQL_EndStep	Constraint (Success)	Green arrow

Logical OR on SQL_EndStep

SQL_EndStep receives arrows from both DFT_Full_Load (FULL path) and SQL_UpdateWatermark (INCREMENTAL path).

Right-click SQL_EndStep → Properties → set LogicalAnd = False (Logical OR).

Only one predecessor needs to succeed for SQL_EndStep to execute.

© 2026 Mr. Sot So — Royal University of Phnom Penh

═══ Step 1: SQL_StartStep (Execute SQL Task) ═══

General Tab

Property	Value
Name	SQL_StartStep
Connection	CM_ETL_Control
SQLSourceType	Direct input
ResultSet	None

SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_StartStep
    @BatchID  = ?,
    @StepName = ?,
    @StepID   = ? OUTPUT;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_StepName	Input	NVARCHAR	1	@StepName
User::v_StepID	Output	LONG	2	@StepID OUTPUT

Variable-Driven Step Name

The @StepName parameter is now mapped to User::v_StepName instead of a static string.
Change the v_StepName default value in each package (e.g., 'Load_STG_Taxpayer', 'Load_STG_Payment').
This makes all 9 packages structurally identical, only variable defaults differ.

═══ Step 2a: SQL_TruncateStaging (Execute SQL Task), FULL Path ═══

Only runs on FULL load path. Clears the staging table before loading all source rows.

General Tab

Property	Value
Name	SQL_TruncateStaging

© 2026 Mr. Sot So — Royal University of Phnom Penh

Connection	CM_DV_Staging
SQLSourceType	Direct input
ResultSet	None

SQLStatement (Expression-Based)

Because TRUNCATE TABLE does not support parameterized table names via ?, use an Expression on the SQLStatementSource property:

```
"TRUNCATE TABLE dbo.STG_" + @[User::v_TableName] + ";"
```

How to Set Expression-Based SQLStatement

1. Click SQL_TruncateStaging in the control flow.
2. In Properties window (F4), find Expressions property → click the [...] button.
3. In the Property Expressions Editor, select SqlStatementSource from the Property dropdown.
4. Enter: "TRUNCATE TABLE dbo.STG_" + @[User::v_TableName] + ";"
5. Click Evaluate Expression to verify it produces: TRUNCATE TABLE dbo.STG_Category;
6. Click OK. The General Tab SQLStatement field will now show the expression icon.

Incoming Precedence Constraint (from SQL_StartStep)

Property	Value
Evaluation operation	Expression
Expression	@[$Package::p_LoadType] == "FULL"

How to Set Expression-Only Precedence Constraint

1. Double-click the green arrow from SQL_StartStep to SQL_TruncateStaging.
2. Change Evaluation operation from 'Constraint' to 'Expression'.
3. Enter: @[$Package::p_LoadType] == "FULL"
4. The arrow turns blue with an 'fx' label, indicating expression-based evaluation.

═══ Step 2b: SQL_GetWatermark (Execute SQL Task), INCREMENTAL Path ═══

Only runs on INCREMENTAL load path. Retrieves the last successful watermark for delta extraction.

General Tab

Property	Value
Name	SQL_GetWatermark

© 2026 Mr. Sot So — Royal University of Phnom Penh

Connection	CM_ETL_Control
SQLSourceType	Direct input
ResultSet	Single row

SQLStatement

```
SELECT ISNULL(
    (SELECT TOP 1 LastLoadDate FROM dbo.ETL_Watermark WHERE TableName = ? AND IsActive = 1),
    '1900-01-01'
) AS LastWatermark
```

```
SELECT ISNULL((SELECT TOP 1 LastLoadDate FROM dbo.ETL_Watermark WHERE TableName = ? AND IsActive = 1),
'1900-01-01') AS LastWatermark
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into SQL_GetWatermark for ALL 9 staging packages:

✗ **COMMON ERROR, "Single Row result set but no rows returned":** If ETL_Watermark table is empty (after clean/first run), the original SQL returns 0 rows and SSIS fails. The ISNULL wrapper guarantees exactly 1 row is always returned. Default '1900-01-01' means the incremental query uses WHERE UpdatedDate > '1900-01-01', capturing all rows (same as full load).

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
User::v_TableName	Input	NVARCHAR	0	WHERE TableName = ?

Result Set Tab

Result Name	Variable Name
0	User::v_LastWatermark

Incoming Precedence Constraint (from SQL_StartStep)

Property	Value
Evaluation operation	Expression
Expression	@[$Package::p_LoadType] == "INCREMENTAL"

Result Name Must Be '0' (Not Column Alias)

CONFIDENTIAL

For OLE DB connections with ResultSet = 'Single row', SSIS uses ordinal index 0, 1, 2... as Result Name.
The SQL column alias 'LastWatermark' is ignored, SSIS maps by position, not by name.
Enter 0 in the Result Name column, and select User::v_LastWatermark as the Variable Name.

CONFIDENTIAL

═══ Step 3a: DFT_Full_Load (Data Flow Task), FULL Path ═══

Extracts ALL rows from the source table with ETL metadata columns built dynamically via the v_SQL_FullLoad expression variable. The OLE DB Source uses 'SQL command from variable' access mode.

Data Flow Pipeline (3 components)

#	Component	Description
1	OLE_SRC_Category (OLE DB Source)	SQL command from variable: User::v_SQL_FullLoad
2	RCT_CountRows (Row Count)	Counts rows into User::v_RowCount
3	OLE_DST_STG_Category (OLE DB Destination)	Fast Load into DV_Staging.dbo.STG_Category

Component 1: OLE_SRC_Category (OLE DB Source)

Step-by-step Configuration:

- 1. Drag an OLE DB Source onto the DFT_Full_Load canvas.
- 2. Double-click to open the OLE DB Source Editor.
- 3. Set OLE DB connection manager: CM_TaxSystemDB
- 4. Set Data access mode: SQL command from variable
- 5. Set Variable name: User::v_SQL_FullLoad
- 6. Click Columns tab → verify all 9 columns appear (6 source + 3 metadata).
- 7. Click OK.

OLE DB Source Editor Settings

Property	Value
OLE DB connection manager	CM_TaxSystemDB
Data access mode	SQL command from variable
Variable name	User::v_SQL_FullLoad

SQL Generated at Runtime (by v_SQL_FullLoad expression)

When p_BatchID = 1, the expression evaluates to:

```
SELECT CategoryID, CategoryName, CategoryDescription,
       IsActive, CreatedDate, UpdatedDate,
       N'TaxSystemDB'              AS SourceSystem,
       N'TaxSystemDB.dbo.Category' AS RecordSource,
       GETDATE()                   AS LoadDateTime
FROM dbo.Category
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

Columns Tab May Show Warning, This Is Normal

When you first select 'SQL command from variable', SSIS evaluates the expression to determine output columns.

Click the Columns tab to verify all 9 columns appear. If columns do not appear, ensure v_SQL_FullLoad has EvaluateAsExpression = True and the expression is valid.

No Parameters button is needed, all values are embedded in the variable expression.

Output Columns (from variable SQL)

Column	Data Type (SSIS)	Length	Source
CategoryID	DT_I4 (four-byte integer)	—	Source table
CategoryName	DT_STR	100	Source table
CategoryDescription	DT_STR	500	Source table
IsActive	DT_BOOL	—	Source table
CreatedDate	DT_DBTIMESTAMP	—	Source table
UpdatedDate	DT_DBTIMESTAMP	—	Source table
SourceSystem	DT_WSTR (unicode)	50	N'...' literal via v_SourceSystem
RecordSource	DT_WSTR (unicode)	50	N'...' literal via v_RecordSource
LoadDateTime	DT_DBTIMESTAMP	—	GETDATE() in SQL

Component 2: RCT_CountRows (Row Count Transformation)

Step-by-step Configuration:

- 1. Drag a Row Count transformation onto the canvas.
- 2. Connect the green output from OLE_SRC_Category to RCT_CountRows.
- 3. Double-click RCT_CountRows → set VariableName: User::v_RowCount
- 4. Click OK.

Component 3: OLE_DST_STG_Category (OLE DB Destination)

Step-by-step Configuration:

- 1. Drag an OLE DB Destination onto the canvas.
- 2. Connect the green output from RCT_CountRows to OLE_DST_STG_Category.
- 3. Double-click to open the OLE DB Destination Editor.

© 2026 Mr. Sot So — Royal University of Phnom Penh

- 4. Set OLE DB connection manager: CM_DV_Staging
- 5. Set Data access mode: Table or view – fast load
- 6. Name of the table or the view: [dbo].[STG_Category]
- 7. Configure Fast Load options (see table below).
- 8. Click Mappings tab → verify all 9 column mappings (6 source + 3 metadata).
- 9. Click OK.

OLE DB Destination Editor, Connection Manager Tab

Property	Value
OLE DB connection manager	CM_DV_Staging
Data access mode	Table or view – fast load
Name of the table or the view	[dbo].[STG_Category]
Keep identity	Unchecked
Keep nulls	Checked
Table lock	Checked
Check constraints	Unchecked
Rows per batch	10000
Maximum insert commit size	0 (commit all at end)

Mappings Tab, Column Mapping

Input Column (pipeline)	Destination Column (STG_Category)	Source
CategoryID	CategoryID	SQL variable expression
CategoryName	CategoryName	SQL variable expression
CategoryDescription	CategoryDescription	SQL variable expression
IsActive	IsActive	SQL variable expression
CreatedDate	CreatedDate	SQL variable expression
UpdatedDate	UpdatedDate	SQL variable expression
SourceSystem	SourceSystem	N'...' + v_SourceSystem
RecordSource	RecordSource	N'...' + v_RecordSource
LoadDateTime	LoadDateTime	GETDATE() in SQL

© 2026 Mr. Sot So — Royal University of Phnom Penh

═══ Step 3b: DFT_Incremental_Load (Data Flow Task), INCREMENTAL Path ═══

Extracts only delta rows using a dynamically built SQL query from v_SQL_IncrementalLoad that includes the WHERE clause with the watermark value.

Data Flow Pipeline (3 components, same structure as DFT_Full_Load)

#	Component	Description
1	OLE_SRC_Category (OLE DB Source)	SQL command from variable: User::v_SQL_IncrementalLoad
2	RCT_CountRows (Row Count)	Same as DFT_Full_Load: counts into User::v_RowCount
3	OLE_DST_STG_Category (OLE DB Destination)	Same as DFT_Full_Load: Fast Load to STG_Category

RCT and DST Are Identical to DFT_Full_Load

Components 2 and 3 are configured exactly the same as in DFT_Full_Load.
Only the OLE DB Source differs, it uses User::v_SQL_IncrementalLoad instead of v_SQL_FullLoad.
You can copy-paste the DFT_Full_Load and then change only the variable name in the OLE DB Source.

Component 1: OLE_SRC_Category (OLE DB Source), Incremental Query

Step-by-step Configuration:

- 1. Drag an OLE DB Source onto the DFT_Incremental_Load canvas.
- 2. Double-click to open the OLE DB Source Editor.
- 3. Set OLE DB connection manager: CM_TaxSystemDB
- 4. Set Data access mode: SQL command from variable
- 5. Set Variable name: User::v_SQL_IncrementalLoad
- 6. Click Columns tab → verify all 9 columns appear.
- 7. Click OK.

OLE DB Source Editor Settings

Property	Value
OLE DB connection manager	CM_TaxSystemDB
Data access mode	SQL command from variable
Variable name	User::v_SQL_IncrementalLoad

SQL Generated at Runtime (by v_SQL_IncrementalLoad expression)

© 2026 Mr. Sot So — Royal University of Phnom Penh

When p_BatchID = 5 and v_LastWatermark = 2025-01-15 10:30:00, the expression evaluates to:

```
SELECT CategoryID, CategoryName, CategoryDescription,
       IsActive, CreatedDate, UpdatedDate,
       N'TaxSystemDB'              AS SourceSystem,
       N'TaxSystemDB.dbo.Category' AS RecordSource,
       GETDATE()                   AS LoadDateTime
FROM dbo.Category
WHERE UpdatedDate > '2025-01-15 10:30:00.000'
```

DateTime Format in Expression

The (DT_WSTR,23) cast converts DateTime to string format: 'yyyy-MM-dd HH:mm:ss.fff'.
SQL Server can parse this format natively when comparing with datetime columns.
The single quotes around the datetime value are added by the expression string concatenation.
Ensure v_LastWatermark is populated by SQL_GetWatermark BEFORE DFT_Incremental_Load executes.

CONFIDENTIAL

═══ Step 4: SQL_UpdateWatermark (Execute SQL Task), INCREMENTAL Path Only ═══

Only runs on INCREMENTAL path. Advances the watermark after successful load. This step is connected directly after DFT_Incremental_Load.

General Tab

Property	Value
Name	SQL_UpdateWatermark
Connection	CM_ETL_Control
SQLSourceType	Direct input
ResultSet	None

SQLStatement

```
UPDATE dbo.ETL_Watermark
SET LastValue    = CONVERT(VARCHAR(500), GETDATE(), 120),
    LastLoadDate = GETDATE(),
    UpdatedDate  = GETDATE()
WHERE TableName  = ?

  AND IsActive   = 1;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
User::v_TableName	Input	NVARCHAR	0	WHERE TableName = ?

Incoming Precedence Constraint (from DFT_Incremental_Load)

Property	Value
Evaluation operation	Constraint
Value	Success

═══ Step 5: SQL_EndStep (Execute SQL Task) ═══

General Tab

Property	Value
Name	SQL_EndStep
Connection	CM_ETL_Control

CONFIDENTIAL

SQLSourceType	Direct input
ResultSet	None

SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_EndStep
    @StepID  = ?,
    @Status  = 'Success',
    @RowCount = ?;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
User::v_StepID	Input	LONG	0	@StepID
User::v_RowCount	Input	LONG	1	@RowCount

Incoming Precedence Constraints (Logical OR)

From	Evaluation	When
DFT_Full_Load	Constraint (Success)	FULL path: directly after Data Flow completes
SQL_UpdateWatermark	Constraint (Success)	INCREMENTAL path: after watermark update

Right-click SQL_EndStep → Properties → set LogicalAnd = False (Logical OR).

CONFIDENTIAL

3.6.1 All 9 Staging Packages, Variable Values and SQL Expressions

All 9 staging packages share the identical control flow and data flow structure described in Section 3.6. Only the package variable default values and the v_SQL expression column lists differ per package. The FROM clause uses @[User::v_TableName] so it auto-adjusts. This section provides the complete variable configuration and SSIS expressions for each package.

Quick Reference, Variable Default Values Per Package

Package (.dtsx)	v_StepName	v_TableName	v_RecordSource
STG_Category	Load_STG_Category	Category	TaxSystemDB.dbo.Category
STG_Structure	Load_STG_Structure	Structure	TaxSystemDB.dbo.Structure
STG_Activity	Load_STG_Activity	Activity	TaxSystemDB.dbo.Activity
STG_Taxpayer	Load_STG_Taxpayer	Taxpayer	TaxSystemDB.dbo.Taxpayer
STG_Owner	Load_STG_Owner	Owner	TaxSystemDB.dbo.Owner
STG_Officer	Load_STG_Officer	Officer	TaxSystemDB.dbo.Officer
STG_MonthlyDeclaration	Load_STG_MonthlyDeclaration	MonthlyDeclaration	TaxSystemDB.dbo.MonthlyDeclaration
STG_AnnualDeclaration	Load_STG_AnnualDeclaration	AnnualDeclaration	TaxSystemDB.dbo.AnnualDeclaration
STG_Payment	Load_STG_Payment	Payment	TaxSystemDB.dbo.Payment

v_SourceSystem = 'TaxSystemDB' for all 9 packages (no change needed).

OLE DB Source Configuration (same for all 9 packages)

DFT	Property	Value
DFT_Full_Load	Data access mode	SQL command from variable
DFT_Full_Load	Variable name	User::v_SQL_FullLoad
DFT_Incremental_Load	Data access mode	SQL command from variable
DFT_Incremental_Load	Variable name	User::v_SQL_IncrementalLoad

No Parameters button is needed, all values are embedded in the variable expression.

1. STG_Category.dtsx, v_SQL Expressions

v_SQL_FullLoad (EvaluateAsExpression = True):

```
"SELECT CategoryID, CategoryName, CategoryDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad (EvaluateAsExpression = True):

```
"SELECT CategoryID, CategoryName, CategoryDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

2. STG_Structure.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT StructureID, StructureName, StructureDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT StructureID, StructureName, StructureDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

3. STG_Activity.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT ActivityID, ActivityName, ActivityDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT ActivityID, ActivityName, ActivityDescription, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

4. STG_Taxpayer.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT TaxpayerID, TaxID, LegalBusinessName, TradingName, CategoryID, StructureID, "
+ "RegistrationDate, EstimatedAnnualRevenue, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
```

CONFIDENTIAL

```
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT TaxpayerID, TaxID, LegalBusinessName, TradingName, CategoryID, StructureID, "
+ "RegistrationDate, EstimatedAnnualRevenue, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

5. STG_Owner.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT OwnerID, TaxpayerID, OwnerName, OwnerType, OwnershipPercentage, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT OwnerID, TaxpayerID, OwnerName, OwnerType, OwnershipPercentage, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

6. STG_Officer.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT OfficerID, OfficerCode, FirstName, LastName, Department, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT OfficerID, OfficerCode, FirstName, LastName, Department, "
+ "IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

CONFIDENTIAL

7. STG_MonthlyDeclaration.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT DeclarationID, TaxpayerID, DeclarationMonth, DeclarationYear, "
+ "GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount, "
+ "DeclarationDate, DueDate, Status, OfficerID, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT DeclarationID, TaxpayerID, DeclarationMonth, DeclarationYear, "
+ "GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount, "
+ "DeclarationDate, DueDate, Status, OfficerID, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

8. STG_AnnualDeclaration.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT AnnualDeclarationID, TaxpayerID, DeclarationYear, "
+ "GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount, "
+ "DeclarationDate, DueDate, Status, OfficerID, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT AnnualDeclarationID, TaxpayerID, DeclarationYear, "
+ "GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount, "
+ "DeclarationDate, DueDate, Status, OfficerID, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

9. STG_Payment.dtsx, v_SQL Expressions

v_SQL_FullLoad:

```
"SELECT PaymentID, TaxpayerID, DeclarationID, AnnualDeclarationID, "
+ "PaymentAmount, PaymentDate, PaymentMethod, ReferenceNumber, "
+ "Status, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
```

CONFIDENTIAL

```
+ "FROM dbo." + @[User::v_TableName]
```

v_SQL_IncrementalLoad:

```
"SELECT PaymentID, TaxpayerID, DeclarationID, AnnualDeclarationID, "
+ "PaymentAmount, PaymentDate, PaymentMethod, ReferenceNumber, "
+ "Status, IsActive, CreatedDate, UpdatedDate, "
+ "N'" + @[User::v_SourceSystem] + "' AS SourceSystem, "
+ "N'" + @[User::v_RecordSource] + "' AS RecordSource, "
+ "GETDATE() AS LoadDateTime "
+ "FROM dbo." + @[User::v_TableName] + " "
+ "WHERE UpdatedDate > '" + (DT_WSTR,23)@[User::v_LastWatermark] + "'"
```

Cloning Packages Efficiently

Build STG_Category.dtsx first as the template package with all control flow, data flow, OnError handlers, and variable definitions.

Copy STG_Category.dtsx and rename to STG_Structure.dtsx, STG_Activity.dtsx, etc.

In each copy, change: (1) the 4 simple variable defaults (v_StepName, v_TableName, v_SourceSystem, v_RecordSource), (2) the column list in v_SQL_FullLoad and v_SQL_IncrementalLoad expressions (FROM clause auto-adjusts via v_TableName), and (3) the OLE DB Destination table name.

The control flow tasks (SQL_StartStep, SQL_TruncateStaging, SQL_GetWatermark, SQL_UpdateWatermark, SQL_EndStep) need no changes because they use variables.

The metadata portion of the SQL expressions (N'...' + v_SourceSystem, N'...' + v_RecordSource, GETDATE()) is identical across all 9 packages.

After Cloning, Refresh OLE DB Source Metadata

SSIS caches column metadata (names, data types, lengths) from the original package.

After cloning, the cached metadata still reflects the original table (e.g., STG_Category).

This causes truncation warnings when the new table has longer column values (e.g., RecordSource).

For EACH cloned package, refresh metadata in BOTH DFT_Full_Load and DFT_Incremental_Load:

1. Open OLE DB Source → Columns tab → click OK (forces metadata refresh from the expression variable).
2. Open OLE DB Destination → change Table name to the new staging table (e.g., STG_Structure).
3. Click Mappings tab → verify columns auto-map correctly → click OK.

If you skip this step, you may see: 'Truncation may occur due to retrieving data from database column with length of X' or column mapping errors at runtime.

© 2026 Mr. Sot So — Royal University of Phnom Penh

3.7 Staging Stored Procedures

Each staging table has a corresponding stored procedure that handles both FULL and INCREMENTAL logic via the @LoadType parameter. With the DFT-based approach in Section 3.6, these SPs serve as an alternative (Option A) for teams preferring SQL-based ETL over SSIS Data Flow Tasks.

Procedure Template: usp_Load_STG_Category

```
CREATE OR ALTER PROCEDURE dbo.usp_Load_STG_Category
    @BatchID   INT,
    @LoadType  NVARCHAR(20),
    @RowCount  INT = 0 OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @LastWatermark DATETIME;

    BEGIN TRY

        -- ===========================
        -- FULL LOAD: Truncate + Insert All
        -- ===========================
        IF @LoadType = 'FULL'
        BEGIN
            TRUNCATE TABLE DV_Staging.dbo.STG_Category;

            INSERT INTO DV_Staging.dbo.STG_Category
                (CategoryID, CategoryName, CategoryDescription,
                 IsActive, CreatedDate, UpdatedDate,
                 SourceSystem, RecordSource, LoadDateTime)
            SELECT
                CategoryID, CategoryName, CategoryDescription,
                IsActive, CreatedDate, UpdatedDate,
                'TaxSystemDB',   -- SourceSystem (source has none)
                'TaxSystemDB',   -- RecordSource
                GETDATE()        -- LoadDateTime
            FROM TaxSystemDB.dbo.Category;

            SET @RowCount = @@ROWCOUNT;
        END

        -- ===========================
        -- INCREMENTAL LOAD: Watermark-Based Delta
        -- ===========================
        ELSE IF @LoadType = 'INCREMENTAL'
        BEGIN
            -- 1. Get last watermark
            SELECT @LastWatermark = LastLoadDate
            FROM ETL_Control.dbo.ETL_Watermark
            WHERE TableName  = 'Category'
```

CONFIDENTIAL

```
            AND ColumnName = 'UpdatedDate'
            AND IsActive   = 1;

        -- 2. Extract delta
        INSERT INTO DV_Staging.dbo.STG_Category
            (CategoryID, CategoryName, CategoryDescription,
             IsActive, CreatedDate, UpdatedDate,
             SourceSystem, RecordSource, LoadDateTime)
        SELECT
            CategoryID, CategoryName, CategoryDescription,
            IsActive, CreatedDate, UpdatedDate,
            'TaxSystemDB',
            'TaxSystemDB',
            GETDATE()
        FROM TaxSystemDB.dbo.Category
        WHERE UpdatedDate > @LastWatermark;

        SET @RowCount = @@ROWCOUNT;

        -- 3. Update watermark on success
        UPDATE ETL_Control.dbo.ETL_Watermark
        SET LastValue    = CONVERT(VARCHAR(500), GETDATE(), 120),
            LastLoadDate = GETDATE(),
            UpdatedDate  = GETDATE()
        WHERE TableName  = 'Category'
            AND ColumnName = 'UpdatedDate'
            AND IsActive   = 1;
    END

    END TRY
    BEGIN CATCH
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
END
```

Group B Tables: SourceSystem Comes from Source

For tables where the source already has a SourceSystem column (Taxpayer, Officer, Owner, MonthlyDeclaration, AnnualDeclaration, Payment),
the INSERT should map src.SourceSystem directly instead of the hardcoded 'TaxSystemDB' literal.
Only Group A tables (Category, Structure, Activity) need the hardcoded SourceSystem value.

3.8 Complete Stored Procedure List

All 9 staging stored procedures follow the usp_Load_STG_Category template above. Replace the table name, column list, and business key hash accordingly.

CONFIDENTIAL

Stored Procedure	Source Table	Business Key Column
usp_Load_STG_Category	Category	CategoryID
usp_Load_STG_Structure	Structure	StructureID
usp_Load_STG_Activity	Activity	ActivityID
usp_Load_STG_Taxpayer	Taxpayer	TaxID
usp_Load_STG_Owner	Owner	OwnerID
usp_Load_STG_Officer	Officer	OfficerCode
usp_Load_STG_MonthlyDeclaration	MonthlyDeclaration	DeclarationID
usp_Load_STG_AnnualDeclaration	AnnualDeclaration	AnnualDeclarationID
usp_Load_STG_Payment	Payment	PaymentID

3.9 Full Load vs Incremental Load Comparison

Aspect	Full Load	Incremental Load
Trigger	Weekly (Sunday)	Nightly (Mon–Sat)
Pre-load action	TRUNCATE TABLE STG_* (SQL_TruncateStaging)	No truncate (SQL_GetWatermark reads last load date)
Data Flow Task	DFT_Full_Load (SELECT all rows)	DFT_Incremental_Load (SELECT WHERE UpdatedDate > ?)
Data volume	High (full table scan)	Low (delta only)
Watermark	Not used (no SQL_GetWatermark/SQL_UpdateWatermark)	Read before load → update after load
Post-load	DFT → SQL_EndStep directly	DFT → SQL_UpdateWatermark → SQL_EndStep
Idempotent	Yes (TRUNCATE + reload)	No (appends duplicates if re-run)
Recovery	Re-run safely	Reset watermark first, then re-run

3.10 OnError Handler (Staging Package Level)

Each staging child package has its own OnError handler that logs the error at the step level without closing the parent batch. Error propagation goes: Child SP → Child Package OnError → Orchestrator OnError → Master OnError.

OnError Tasks (3 steps)

#	Task	Description
1	SCR_CaptureError (Script Task)	Reads System::ErrorDescription into User::v_ErrorMessage.
2	SQL_EndStepOnError (Execute SQL Task)	Calls usp_EndStep(@StepID, 'Failed', 0) to close the step as failed.

© 2026 Mr. Sot So — Royal University of Phnom Penh

3	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError(@BatchID, 'Error', @ErrorSource, @ErrorMessage) to record error details.

Critical: OnError Tasks Require Separate Connection + NotSupported

Two settings are required for OnError handler tasks to commit independently:

1. Connection: Use CM_ETL_Control_OnError (not CM_ETL_Control) for SQL_EndStepOnError and SQL_LogErrorOnError.
   → A shared connection is already enlisted in the parent's transaction. OnError writes get rolled back.
   → A separate connection creates an independent database session that always commits.

2. TransactionOption: Set to NotSupported on all OnError tasks (SCR_CaptureError, SQL_EndStepOnError, SQL_LogErrorOnError).
   → F4 Properties → TransactionOption = NotSupported.

Without BOTH settings, SQL_EndStepOnError and SQL_LogErrorOnError execute successfully (green ✓) but their writes are rolled back → StepEndTime and StepLogID stay NULL.

Stored Procedures Handle Unset Variables (NULLIF Guard)

SSIS OnError fires at EVERY level as errors propagate up (child → orchestrator → master).
At orchestrator/master level, v_StepID = 0 (default INT) because those packages don't call usp_StartStep.
usp_EndStep: Skips UPDATE if @StepID is 0 or NULL (IF @StepID IS NULL OR @StepID = 0 RETURN).
usp_LogError: Converts 0 to NULL before INSERT (SET @StepLogID = NULLIF(@StepLogID, 0)).
This prevents FK violations when OnError cascades to levels where v_StepID is unset.

SQL_EndStepOnError, SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_EndStep
    @StepID       = ?,
    @Status       = 'Failed',
    @RowCount     = 0,
    @ErrorMessage = ?;
```

SQL_EndStepOnError, Parameter Mapping

Variable Name	Direction	Data Type	Param #	Maps To
User::v_StepID	Input	LONG	0	@StepID
User::v_ErrorMessage	Input	NVARCHAR	1	@ErrorMessage

© 2026 Mr. Sot So — Royal University of Phnom Penh

SQL_LogErrorOnError, SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_LogError
    @BatchID       = ?,
    @StepLogID     = ?,
    @ErrorSeverity = 'Error',
    @ErrorSource   = ?,
    @ErrorMessage  = ?;
```

SQL_LogErrorOnError, Parameter Mapping

Variable Name	Direction	Data Type	Param #	Maps To
$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_StepID	Input	LONG	1	@StepLogID
User::v_ErrorSource	Input	NVARCHAR	2	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	3	@ErrorMessage

Error Propagation Chain

1. usp_Load_STG_Category raises error (CATCH block) → DFT/SQL task fails.
2. STG_Category.dtsx OnError fires → logs step as Failed + records error in ETL_ErrorLog.
4. Error propagates to STG_Full_Load_All.dtsx → EPT task fails.
7. Error propagates to Master_Full_Load.dtsx → EPT task fails.
5. Error propagates to Master_Complete_Pipeline.dtsx → OnError closes batch as Failed.

3.11 OnError Handler (Orchestrator Level, STG_Full_Load_All.dtsx)

The orchestrator-level OnError handler serves a different purpose from the child-level OnError. When a child package (e.g., STG_Category.dtsx) fails, its OnError fires first and logs the specific step failure. The error then bubbles up to the orchestrator (STG_Full_Load_All.dtsx), where the orchestrator's own OnError fires. This handler does NOT duplicate the step-level logging, instead, it captures the higher-level context: which child package failed within the orchestrator.

Package Variables for OnError

Variable	Data Type	Default	Purpose
User::v_ErrorMessage	String	(empty)	Captured by SCR_CaptureError in OnError handler

© 2026 Mr. Sot So — Royal University of Phnom Penh

User::v_ErrorSource	String	(Expression)	Auto-set via expression: @[System::PackageName]. No manual change needed.

OnError Event Handler Tasks (2 steps)

#	Task	Description
1	SCR_CaptureError (Script Task)	Reads System::ErrorDescription into User::v_ErrorMessage.
2	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError to record orchestrator-level error. No usp_EndStep call, orchestrator does not own any steps.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on both OnError tasks.

═══ Step 1: SCR_CaptureError (Script Task) ═══

Same Script Task pattern as child package OnError. Captures System::ErrorDescription into User::v_ErrorMessage.

ReadOnlyVariables

System::ErrorDescription

ReadWriteVariables

User::v_ErrorMessage

Script (C#)

```
public void Main()
{
    string errorDesc = Dts.Variables["System::ErrorDescription"].Value.ToString();
    Dts.Variables["User::v_ErrorMessage"].Value = errorDesc;
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

═══ Step 2: SQL_LogErrorOnError (Execute SQL Task) ═══

Calls usp_LogError only. Unlike the child-level OnError, there is NO usp_EndStep call because the orchestrator does not register its own steps, step tracking belongs to the child packages.

General Tab

Property	Value

CONFIDENTIAL

Name	SQL_LogErrorOnError
Connection	CM_ETL_Control_OnError
SQLSourceType	Direct input
ResultSet	None

SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_LogError
    @BatchID       = ?,
    @ErrorSeverity = 'Error',
    @ErrorSource   = ?,
    @ErrorMessage  = ?;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_ErrorSource	Input	NVARCHAR	1	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	2	@ErrorMessage

Child vs Orchestrator OnError, Key Differences

Aspect	Child Package (STG_Category.dtsx)	Orchestrator (STG_Full_Load_All.dtsx)
Owns steps?	Yes, calls usp_StartStep / usp_EndStep	No, does not register its own steps
OnError calls	SQL_EndStepOnError (usp_EndStep) + SQL_LogErrorOnError (usp_LogError)	SQL_LogErrorOnError (usp_LogError only)
@ErrorSource	v_ErrorSource = @[System::PackageName] (auto)	v_ErrorSource = @[System::PackageName] (auto)
Error detail	Knows exact SP/task that failed	Knows which child package failed (from ErrorDescription)
Variables needed	v_StepID, v_ErrorMessage, v_ErrorSource	v_ErrorMessage, v_ErrorSource (no v_StepID)
Propagation	Error bubbles up to orchestrator	Error bubbles up to Master_Full_Load.dtsx

Orchestrator Error Behavior, Sibling Package Execution

© 2026 Mr. Sot So — Royal University of Phnom Penh

When EPT_STG_Category fails, STG_Full_Load_All.dtsx's OnError fires. By default, the remaining child packages (EPT_STG_Structure through EPT_STG_Payment) will NOT execute because the orchestrator's MaximumErrorCount defaults to 1.

To allow independent packages to continue after a sibling fails:

1. Set MaximumErrorCount = 0 on the orchestrator (or a high number).
2. Change precedence constraints from Success to Completion (green → blue arrows).

Use case: If STG_Category failure should NOT prevent STG_Taxpayer from loading (they are independent entities), use Completion constraints. If downstream packages depend on the failed package's data, keep Success constraints.

Orchestrator OnError Pattern, All Layers

The same OnError pattern (SCR_CaptureError → SQL_LogErrorOnError with usp_LogError only) applies to all layer orchestrators:

Staging: STG_Full_Load_All.dtsx / STG_Incremental_Load_All.dtsx
Bronze: BRZ_Load_All_Hubs.dtsx / BRZ_Load_All_Satellites.dtsx / BRZ_Load_All_Links.dtsx
Silver: SLV_Load_All.dtsx
Gold: GLD_Load_All_Dimensions.dtsx / GLD_Load_All_Facts.dtsx

v_ErrorSource uses expression @[System::PackageName] which auto-resolves at runtime. No manual change needed when cloning. The SQL, Script Task, and parameter mapping are identical across all packages.

Master_Full_Load.dtsx and Master_Incremental_Load.dtsx also follow this pattern (see Section 8.6). Only Master_Complete_Pipeline.dtsx differs, it calls usp_EndBatch('Failed') because it owns the batch.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4. Bronze Packages, Detailed Implementation

The Bronze layer implements the core Data Vault 2.0 model: Hubs (business keys), Satellites (descriptive attributes), and Links (relationships). It reads from DV_Staging and writes to DV_Bronze using insert-only loading. Three orchestrator packages coordinate 23 target objects.

4.1 Package Inventory

Orchestrator Packages (02_Bronze folder)

Package	Called By	Purpose
BRZ_Load_All_Hubs.dtsx	Master_Full_Load / Master_Incremental_Load	Loads all 9 Hub tables from staging
BRZ_Load_All_Satellites.dtsx	Master_Full_Load / Master_Incremental_Load	Loads all 9 Satellite tables from staging
BRZ_Load_All_Links.dtsx	Master_Full_Load / Master_Incremental_Load	Loads all 5 Link tables from staging

Hub Child Packages (02_Bronze folder)

Package	Source	Target	Business Key
BRZ_HUB_Category.dtsx	STG_Category	HUB_Category	CategoryID
BRZ_HUB_Structure.dtsx	STG_Structure	HUB_Structure	StructureID
BRZ_HUB_Activity.dtsx	STG_Activity	HUB_Activity	ActivityID
BRZ_HUB_Taxpayer.dtsx	STG_Taxpayer	HUB_Taxpayer	TaxID
BRZ_HUB_Owner.dtsx	STG_Owner	HUB_Owner	OwnerID
BRZ_HUB_Officer.dtsx	STG_Officer	HUB_Officer	OfficerCode
BRZ_HUB_Declaration.dtsx	STG_MonthlyDeclaration	HUB_Declaration	DeclarationID
BRZ_HUB_Payment.dtsx	STG_Payment	HUB_Payment	PaymentID
BRZ_HUB_AnnualDecl.dtsx	STG_AnnualDeclaration	HUB_AnnualDecl	AnnualDeclarationID

Satellite Child Packages (02_Bronze folder)

Package	Source	Target	Change Detection
BRZ_SAT_Category.dtsx	STG_Category	SAT_Category	HASHBYTES on-the-fly
BRZ_SAT_Structure.dtsx	STG_Structure	SAT_Structure	HASHBYTES on-the-fly
BRZ_SAT_Activity.dtsx	STG_Activity	SAT_Activity	HASHBYTES on-the-fly

© 2026 Mr. Sot So — Royal University of Phnom Penh

BRZ_SAT_Taxpayer.dtsx	STG_Taxpayer	SAT_Taxpayer	HASHBYTES on-the-fly
BRZ_SAT_Owner.dtsx	STG_Owner	SAT_Owner	HASHBYTES on-the-fly
BRZ_SAT_Officer.dtsx	STG_Officer	SAT_Officer	HASHBYTES on-the-fly
BRZ_SAT_MonthlyDecl.dtsx	STG_MonthlyDeclaration	SAT_MonthlyDecl	HASHBYTES on-the-fly
BRZ_SAT_Payment.dtsx	STG_Payment	SAT_Payment	HASHBYTES on-the-fly
BRZ_SAT_AnnualDecl.dtsx	STG_AnnualDeclaration	SAT_AnnualDecl	HASHBYTES on-the-fly

Link Child Packages (02_Bronze folder)

Package	Source(s)	Target	Foreign Keys
BRZ_LNK_TaxpayerDeclaration.dtsx	STG_MonthlyDeclaration	LNK_TaxpayerDeclaration	TaxpayerID + DeclarationID
BRZ_LNK_DeclarationPayment.dtsx	STG_Payment	LNK_DeclarationPayment	DeclarationID + PaymentID
BRZ_LNK_TaxpayerOfficer.dtsx	STG_Officer	LNK_TaxpayerOfficer	TaxpayerID + OfficerID
BRZ_LNK_TaxpayerOwner.dtsx	STG_Owner	LNK_TaxpayerOwner	TaxpayerID + OwnerID
BRZ_LNK_TaxpayerAnnualDecl.dtsx	STG_AnnualDeclaration	LNK_TaxpayerAnnualDecl	TaxpayerID + AnnualDeclarationID

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.2 Bronze Table Design

Hub, Satellite, and Link tables follow strict Data Vault 2.0 patterns. All use insert-only loading — no updates or deletes.

Hub Table Template: HUB_Category

```
CREATE TABLE DV_Bronze.dbo.HUB_Category (
    HUB_Category_HK    VARBINARY(32)   NOT NULL,  -- HASHBYTES('SHA2_256', BusinessKey)
    CategoryID          INT             NOT NULL,  -- Business Key (from source PK)
    HUB_LoadDate        DATETIME        NOT NULL DEFAULT GETDATE(),
    HUB_RecordSource    VARCHAR(50)     NOT NULL DEFAULT 'TaxSystemDB',

    CONSTRAINT PK_HUB_Category PRIMARY KEY NONCLUSTERED (HUB_Category_HK),
    CONSTRAINT UK_HUB_Category_BK UNIQUE (CategoryID)
);
```

Satellite Table Template: SAT_Category

```
CREATE TABLE DV_Bronze.dbo.SAT_Category (
    HUB_Category_HK    VARBINARY(32)   NOT NULL,  -- FK to HUB_Category
    SAT_LoadDate        DATETIME        NOT NULL DEFAULT GETDATE(),
    SAT_EndDate         DATETIME        NULL,      -- NULL = current record
    SAT_HashDiff        VARBINARY(32)   NOT NULL,  -- Change detection hash
    SAT_RecordSource    VARCHAR(50)     NOT NULL DEFAULT 'TaxSystemDB',

    -- Descriptive attributes (from STG_Category)
    CategoryName        VARCHAR(100)    NULL,
    CategoryDescription VARCHAR(500)    NULL,
    IsActive            BIT             NULL,

    CONSTRAINT PK_SAT_Category PRIMARY KEY (HUB_Category_HK, SAT_LoadDate),
    CONSTRAINT FK_SAT_Category_HUB FOREIGN KEY (HUB_Category_HK)
        REFERENCES dbo.HUB_Category (HUB_Category_HK)
);
```

Link Table Template: LNK_TaxpayerDeclaration

```
CREATE TABLE DV_Bronze.dbo.LNK_TaxpayerDeclaration (
    LNK_TaxpayerDeclaration_HK  VARBINARY(32)  NOT NULL,  -- Hash of composite key
    HUB_Taxpayer_HK             VARBINARY(32)  NOT NULL,  -- FK to HUB_Taxpayer
    HUB_Declaration_HK          VARBINARY(32)  NOT NULL,  -- FK to HUB_Declaration
    LNK_LoadDate                DATETIME       NOT NULL DEFAULT GETDATE(),
    LNK_RecordSource            VARCHAR(50)    NOT NULL DEFAULT 'TaxSystemDB',

    CONSTRAINT PK_LNK_TaxpayerDeclaration PRIMARY KEY NONCLUSTERED
        (LNK_TaxpayerDeclaration_HK),
    CONSTRAINT FK_LNK_TxpDecl_Taxpayer FOREIGN KEY (HUB_Taxpayer_HK)
        REFERENCES dbo.HUB_Taxpayer (HUB_Taxpayer_HK),
    CONSTRAINT FK_LNK_TxpDecl_Declaration FOREIGN KEY (HUB_Declaration_HK)
```

CONFIDENTIAL

```
        REFERENCES dbo.HUB_Declaration (HUB_Declaration_HK)
);
```

Data Vault Insert-Only Pattern

Hubs: Lookup Transform (No Match = new business key), only new records are inserted.
Satellites: Lookup + Conditional Split (HashDiff change detection) — a new row for every change.
Links: Lookup Transform (No Match = new relationship), only new records are inserted.
This means Bronze layer does NOT distinguish between FULL and INCREMENTAL loads.
Whatever data exists in staging is processed identically, the Lookup-based deduplication handles it.

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.3 BRZ_Load_All_Hubs.dtsx, Orchestrator

This package receives only p_BatchID from Master_Full_Load or Master_Incremental_Load (no LoadType needed, Bronze is always insert-only).

Package Parameters

Parameter	Data Type	Passed From	Purpose
$Package::p_BatchID	Int32	User::v_BatchID (parent)	Current batch identifier

Control Flow, 9 Execute Package Tasks (Sequential)

Step	Task	Parameters Passed to Child
1	EPT_BRZ_HUB_Category	p_BatchID
2	EPT_BRZ_HUB_Structure	p_BatchID
3	EPT_BRZ_HUB_Activity	p_BatchID
4	EPT_BRZ_HUB_Taxpayer	p_BatchID
5	EPT_BRZ_HUB_Owner	p_BatchID
6	EPT_BRZ_HUB_Officer	p_BatchID
7	EPT_BRZ_HUB_Declaration	p_BatchID
8	EPT_BRZ_HUB_Payment	p_BatchID
9	EPT_BRZ_HUB_AnnualDecl	p_BatchID

Hub Load Order

Hubs must be loaded BEFORE Satellites and Links because Satellites/Links reference Hub hash keys via foreign keys.
This is enforced at the Master level: EPT_BRZ_Load_All_Hubs runs before EPT_BRZ_Load_All_Satellites.

4.4 BRZ_Load_All_Satellites.dtsx, Orchestrator

Same structure as BRZ_Load_All_Hubs.dtsx. Receives p_BatchID, executes 9 Satellite child packages sequentially.

Control Flow, 9 Execute Package Tasks (Sequential)

Step	Task	Parameters Passed to Child
1	EPT_BRZ_SAT_Category	p_BatchID

© 2026 Mr. Sot So — Royal University of Phnom Penh

2	EPT_BRZ_SAT_Structure	p_BatchID
3	EPT_BRZ_SAT_Activity	p_BatchID
4	EPT_BRZ_SAT_Taxpayer	p_BatchID
5	EPT_BRZ_SAT_Owner	p_BatchID
6	EPT_BRZ_SAT_Officer	p_BatchID
7	EPT_BRZ_SAT_MonthlyDecl	p_BatchID
8	EPT_BRZ_SAT_Payment	p_BatchID
9	EPT_BRZ_SAT_AnnualDecl	p_BatchID

4.5 BRZ_Load_All_Links.dtsx, Orchestrator

Receives p_BatchID, executes 5 Link child packages sequentially.

Control Flow, 5 Execute Package Tasks (Sequential)

Step	Task	Parameters Passed to Child
1	EPT_BRZ_LNK_TaxpayerDeclaration	p_BatchID
2	EPT_BRZ_LNK_DeclarationPayment	p_BatchID
3	EPT_BRZ_LNK_TaxpayerOfficer	p_BatchID
4	EPT_BRZ_LNK_TaxpayerOwner	p_BatchID
5	EPT_BRZ_LNK_TaxpayerAnnualDecl	p_BatchID

Link Load Order

Links depend on Hub hash keys for both sides of the relationship.
All Hubs and Satellites must be loaded before Links. Links can run in parallel with each other if desired.

CONFIDENTIAL

4.6 Hub Child Packages

All 9 Hub child packages follow the same pattern: a 3-step Control Flow (SQL_StartStep → DFT_Load_Hub → SQL_EndStep) where the DFT uses a 4-component pipeline (OLE_SRC → LKP_HUB → ROW_CNT → OLE_DST). The OLE DB Source computes HASHBYTES in the SQL query, and the Lookup Transform checks for existing Hub records. Only new business keys (No Match output) are inserted.

> ❌ **COMMON ERROR, HASHBYTES VARBINARY MISMATCH:** HASHBYTES() returns VARBINARY(8000) by default. ALL HASHBYTES calls in OLE_SRC SQL MUST be wrapped with CAST(HASHBYTES(...) AS VARBINARY(32)). Without this CAST, SSIS metadata shows VARBINARY(8000) but the Bronze tables expect VARBINARY(32), causing PrimeOutput errors (0xC02020C4) or silent data loss. This applies to ALL Hub, Satellite, and Link packages.

BRZ_HUB_Category.dtsx is the fully documented reference template. Remaining Hub packages document only the differences (v_TableName, v_BKColumn, and source SQL).

CONFIDENTIAL

4.6.1 BRZ_HUB_Category.dtsx, Reference Template

Package Parameters

Parameter	Data Type	Description
p_BatchID	Int32	Current ETL batch ID from orchestrator

Package Variables

Variable Name	Data Type	Default Value	EvaluateAsExpression?
v_TableName	String	Category	No
v_BKColumn	String	CategoryID	No
v_StepName	String	(expression)	Yes
v_SourceSQL	String	(expression)	Yes
v_LookupSQL	String	(expression)	Yes
v_DestTable	String	(expression)	Yes
v_RowCount	Int32	0	No
v_StepID	Int32	0	No
v_ErrorMessage	String	(empty)	No
v_ErrorSource	String	(expression)	Yes

Expression Definitions (F4 → Properties → EvaluateAsExpression = True)

```
v_StepName:     "Load_HUB_" + @[User::v_TableName]
v_DestTable:    "HUB_" + @[User::v_TableName]
v_LookupSQL:    "SELECT HUB_" + @[User::v_TableName] + "_HK FROM DV_Bronze.dbo.HUB_" +
@[User::v_TableName]
v_ErrorSource:  @[System::PackageName]
```

v_SourceSQL Expression (full expression):

```
"SELECT DISTINCT CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg.&quot;"
+ @[User::v_BKColumn]
+ " AS NVARCHAR(MAX)))))) AS VARBINARY(32)) AS HUB_&quot;"
+ @[User::v_TableName] + "_HK, stg." + @[User::v_BKColumn]
+ ", GETDATE() AS HUB_LoadDate, stg.RecordSource AS HUB_RecordSource FROM DV_Staging.dbo.STG_"
+ @[User::v_TableName] + " stg"
```

At runtime (v_TableName = 'Category', v_BKColumn = 'CategoryID'):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.CategoryID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Category_HK,
    stg.CategoryID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_Category stg
```

Connection Managers Used

Connection Manager	Database	Used By
CM_DV_Staging	DV_Staging	OLE_SRC (reads staging data)
CM_DV_Bronze	DV_Bronze	LKP_HUB + OLE_DST
CM_ETL_Control	ETL_Control	SQL_StartStep, SQL_EndStep
CM_ETL_Control_OnError	ETL_Control	OnError handler tasks

© 2026 Mr. Sot So — Royal University of Phnom Penh

Control Flow (3 Steps)

```
SQL_StartStep
    │  (Success)
    ▼
DFT_Load_Hub          ← OLE_SRC → LKP → ROW_CNT → OLE_DST
    │  (Success)
    ▼
SQL_EndStep
```

Step 1: SQL_StartStep (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control`
SQLSourceType	Direct input

```
SET NOCOUNT ON;
EXEC dbo.usp_StartStep @BatchID = ?, @StepName = ?, @StepID = ? OUTPUT;
```

Parameter Mapping:

Variable Name	Direction	Data Type	Param Name	Size
`$Package::p_BatchID`	Input	LONG	`0`	-1
`User::v_StepName`	Input	VARCHAR	`1`	200
`User::v_StepID`	Output	LONG	`2`	-1

CONFIDENTIAL

Step 2: DFT_Load_Hub (Data Flow Task)

This Data Flow reads from staging, computes HASHBYTES in the source SQL, checks for existing Hub records via Lookup, and inserts only new business keys. This section provides full click-by-click instructions.

Data Flow Diagram

```
OLE_SRC_STG_Category
    │  (all rows with HASHBYTES computed in SQL)
    ▼
LKP_HUB_Category
    │
    ├── Match Output ──────────→ (discarded, Hub record already exists)
    │
    └── No Match Output —→ ROW_CNT —→ OLE_DST_HUB_Category
```

4 Components to Add (in order)

#	Component Name	Toolbox Category	Type
1	`OLE_SRC_STG_Category`	Other Sources	OLE DB Source
2	`LKP_HUB_Category`	Common	Lookup
3	`ROW_CNT`	Common	Row Count
4	`OLE_DST_HUB_Category`	Other Destinations	OLE DB Destination

Data Flow Path Connections

From	To	Output Type
`OLE_SRC`	`LKP_HUB`	Default Output (all rows)
`LKP_HUB`	`ROW_CNT`	No Match Output (new rows only)
`ROW_CNT`	`OLE_DST`	Default Output

✓ **TIP:** Leave the LKP Match Output unconnected. Matched rows (existing Hubs) are silently discarded.

CONFIDENTIAL

Component 1: OLE_SRC_STG_Category (OLE DB Source)

Property	Value
Connection Manager	`CM_DV_Staging`
Data Access Mode	SQL command from variable
Variable Name	`User::v_SourceSQL`

Columns Tab, Metadata Setup (one-time per entity)

When using 'SQL command from variable', the Columns tab shows a warning because the variable is not available at design time. To set up metadata:
1. Temporarily set Data Access Mode to 'SQL command'. 2. Paste the runtime SQL into the SQL command text box. 3. Click Columns tab, SSIS reads column metadata. 4. Switch back to 'SQL command from variable'. 5. Set Variable Name to User::v_SourceSQL.

Output Columns:

Column	Data Type	Source
`HUB_Category_HK`	DT_BYTES (32)	HASHBYTES('SHA2_256', BK)
`CategoryID`	DT_I4	stg.CategoryID
`HUB_LoadDate`	DT_DBTIMESTAMP	GETDATE()
`HUB_RecordSource`	DT_WSTR (50)	stg.RecordSource

Component 2: LKP_HUB_Category (Lookup Transform)

General Tab:

Property	Value
Cache mode	Full cache
Connection type	OLE DB Connection Manager
No match handling	Redirect rows to no match output

Connection Tab:

Property	Value
Connection Manager	`CM_DV_Bronze`
SQL source	Use results of an SQL query
SQL via Expression	Set on parent DFT (Control Flow → F4 → Expressions)

Setting Lookup SQL via Expression:

Switch to Control Flow tab → select DFT_Load_Hub → F4 (Properties) → Expressions → click '...' → Property = [LKP_HUB].[SqlCommand] → Expression = @[User::v_LookupSQL].
At runtime evaluates to: SELECT HUB_Category_HK FROM DV_Bronze.dbo.HUB_Category

Columns Tab, Step-by-Step Setup:

1. Drag HUB_Category_HK from Available Input Columns (left) to HUB_Category_HK in Available Lookup Columns (right), this creates the join condition.
2. In the bottom grid, UNCHECK the checkbox for HUB_Category_HK under 'Lookup Column', we don't need it as output, only for matching.
3. Verify the bottom grid shows: Input Column = HUB_Category_HK, Lookup Column = HUB_Category_HK, Lookup Operation = '<ignore>'.

Advanced Tab:

Property	Value
Enable memory restriction	Unchecked (use Full cache)
Treat duplicate keys as errors	Checked (Hub HK is unique)

© 2026 Mr. Sot So — Royal University of Phnom Penh

Component 3: ROW_CNT (Row Count Transform)

Property	Value
VariableName	`User::v_RowCount`

Configuration: Select ROW_CNT → F4 (Properties) → set VariableName to User::v_RowCount. Counts new Hub rows inserted (No Match output from Lookup).

Component 4: OLE_DST_HUB_Category (OLE DB Destination)

Property	Value
Connection Manager	`CM_DV_Bronze`
Data Access Mode	Table or view — fast load
Table Name	Set via expression on parent DFT
Table Lock	Checked
Check Constraints	Checked

Setting Destination Table via Expression:
Control Flow tab → select DFT_Load_Hub → F4 → Expressions → [OLE_DST].[OpenRowset] → @[User::v_DestTable]. At runtime: HUB_Category.

Column Mappings:

Input Column	Destination Column	Auto-Matched?
`HUB_Category_HK`	`HUB_Category_HK`	Yes
`CategoryID`	`CategoryID`	Yes
`HUB_LoadDate`	`HUB_LoadDate`	Yes
`HUB_RecordSource`	`HUB_RecordSource`	Yes

Debugging: Data Viewer Placements

Location	What to Verify
OLE_SRC → LKP	All rows with correct HASHBYTES values
LKP No Match → ROW_CNT	Only new business keys (not already in Hub)

CONFIDENTIAL

Step 3: SQL_EndStep (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control`
SQLSourceType	Direct input

```
SET NOCOUNT ON;
EXEC dbo.usp_EndStep @StepID = ?, @Status = 'Success', @RowCount = ?;
```

Parameter Mapping:

Variable Name	Direction	Data Type	Param Name	Size
`User::v_StepID`	Input	LONG	`0`	-1
`User::v_RowCount`	Input	LONG	`1`	-1

CONFIDENTIAL

OnError Event Handler (BRZ_HUB_Category.dtsx)

Every Bronze child package has an OnError handler at the package level. The pattern is identical across all layers. All OnError tasks MUST use CM_ETL_Control_OnError and set TransactionOption = NotSupported.

OnError Step 1: SCR_CaptureError (Script Task)

Property	Value
ReadOnlyVariables	`System::ErrorDescription`
ReadWriteVariables	`User::v_ErrorMessage`

```
public void Main()
{
    string errorDesc = Dts.Variables["System::ErrorDescription"].Value.ToString();
    Dts.Variables["User::v_ErrorMessage"].Value = errorDesc;
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

OnError Step 2: SQL_EndStepOnError (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control_OnError`
TransactionOption	NotSupported

```
EXEC dbo.usp_EndStep @StepID = ?, @Status = 'Failed', @RowCount = 0, @ErrorMessage = ?
```

Parameter Mapping: Param 0 = User::v_StepID (LONG), Param 1 = User::v_ErrorMessage (VARCHAR, 4000)

OnError Step 3: SQL_LogErrorOnError (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control_OnError`
TransactionOption	NotSupported

```
EXEC dbo.usp_LogError @BatchID = ?, @StepLogID = ?, @ErrorSeverity = 'Error', @ErrorSource = ?,
@ErrorMessage = ?
```

Parameter Mapping: Param 0 = $Package::p_BatchID (LONG), Param 1 = User::v_StepID (LONG), Param 2 = User::v_ErrorSource (VARCHAR, 200), Param 3 = User::v_ErrorMessage (VARCHAR, 4000)

CONFIDENTIAL

Complete Hub DFT Property Summary (BRZ_HUB_Category)

Component	Type	Connection	Key Setting
OLE_SRC	OLE DB Source	CM_DV_Staging	SQL command from variable (v_SourceSQL)
LKP_HUB	Lookup	CM_DV_Bronze	Full cache, redirect no match, v_LookupSQL
ROW_CNT	Row Count	-	User::v_RowCount
OLE_DST	OLE DB Destination	CM_DV_Bronze	Fast load, table lock, v_DestTable

CONFIDENTIAL

4.6.2 BRZ_HUB_Structure.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	Structure
v_BKColumn	StructureID

All expression variables auto-update: v_StepName → Load_HUB_Structure, v_DestTable → HUB_Structure, v_LookupSQL → SELECT HUB_Structure_HK FROM DV_Bronze.dbo.HUB_Structure

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.StructureID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Structure_HK,
    stg.StructureID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_Structure stg
```

Output Columns:

Column	Data Type
HUB_Structure_HK	DT_BYTES (32)
StructureID	DT_I4
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_Structure, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_Structure'.

CONFIDENTIAL

4.6.3 BRZ_HUB_Activity.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	Activity
v_BKColumn	ActivityID

All expression variables auto-update: v_StepName → Load_HUB_Activity, v_DestTable → HUB_Activity, v_LookupSQL → SELECT HUB_Activity_HK FROM DV_Bronze.dbo.HUB_Activity

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.ActivityID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Activity_HK,
    stg.ActivityID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_Activity stg
```

Output Columns:

Column	Data Type
HUB_Activity_HK	DT_BYTES (32)
ActivityID	DT_I4
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_Activity, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_Activity'.

CONFIDENTIAL

4.6.4 BRZ_HUB_Taxpayer.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	Taxpayer
v_BKColumn	TaxID

All expression variables auto-update: v_StepName → Load_HUB_Taxpayer, v_DestTable → HUB_Taxpayer, v_LookupSQL → SELECT HUB_Taxpayer_HK FROM DV_Bronze.dbo.HUB_Taxpayer

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.TaxID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Taxpayer_HK,
    stg.TaxID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_Taxpayer stg
```

Output Columns:

Column	Data Type
HUB_Taxpayer_HK	DT_BYTES (32)
TaxID	DT_STR (20)
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_Taxpayer, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_Taxpayer'.

CONFIDENTIAL

4.6.5 BRZ_HUB_Owner.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	Owner
v_BKColumn	OwnerID

All expression variables auto-update: v_StepName → Load_HUB_Owner, v_DestTable → HUB_Owner, v_LookupSQL → SELECT HUB_Owner_HK FROM DV_Bronze.dbo.HUB_Owner

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.OwnerID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Owner_HK,
    stg.OwnerID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_Owner stg
```

Output Columns:

Column	Data Type
HUB_Owner_HK	DT_BYTES (32)
OwnerID	DT_I4
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_Owner, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_Owner'.

CONFIDENTIAL

4.6.6 BRZ_HUB_Officer.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	Officer
v_BKColumn	OfficerCode

All expression variables auto-update: v_StepName → Load_HUB_Officer, v_DestTable → HUB_Officer, v_LookupSQL → SELECT HUB_Officer_HK FROM DV_Bronze.dbo.HUB_Officer

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.OfficerCode AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Officer_HK,
    stg.OfficerCode,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_Officer stg
```

Output Columns:

Column	Data Type
HUB_Officer_HK	DT_BYTES (32)
OfficerCode	DT_STR (20)
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_Officer, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_Officer'.

CONFIDENTIAL

4.6.7 BRZ_HUB_Declaration.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	Declaration
v_BKColumn	DeclarationID

All expression variables auto-update: v_StepName → Load_HUB_Declaration, v_DestTable → HUB_Declaration, v_LookupSQL → SELECT HUB_Declaration_HK FROM DV_Bronze.dbo.HUB_Declaration

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Declaration_HK,
    stg.DeclarationID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_MonthlyDeclaration stg
```

Output Columns:

Column	Data Type
HUB_Declaration_HK	DT_BYTES (32)
DeclarationID	DT_I4
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_Declaration, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_Declaration'.

CONFIDENTIAL

4.6.8 BRZ_HUB_Payment.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	Payment
v_BKColumn	PaymentID

All expression variables auto-update: v_StepName → Load_HUB_Payment, v_DestTable → HUB_Payment, v_LookupSQL → SELECT HUB_Payment_HK FROM DV_Bronze.dbo.HUB_Payment

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.PaymentID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Payment_HK,
    stg.PaymentID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_Payment stg
```

Output Columns:

Column	Data Type
HUB_Payment_HK	DT_BYTES (32)
PaymentID	DT_I4
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_Payment, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_Payment'.

CONFIDENTIAL

4.6.9 BRZ_HUB_AnnualDecl.dtsx, Clone from Category

Variable Changes:

Variable	New Value
v_TableName	AnnualDecl
v_BKColumn	AnnualDeclarationID

All expression variables auto-update: v_StepName → Load_HUB_AnnualDecl, v_DestTable → HUB_AnnualDecl, v_LookupSQL → SELECT HUB_AnnualDecl_HK FROM DV_Bronze.dbo.HUB_AnnualDecl

v_SourceSQL (core query at runtime):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.AnnualDeclarationID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_AnnualDecl_HK,
    stg.AnnualDeclarationID,
    GETDATE()          AS HUB_LoadDate,
    stg.RecordSource   AS HUB_RecordSource
FROM DV_Staging.dbo.STG_AnnualDeclaration stg
```

Output Columns:

Column	Data Type
HUB_AnnualDecl_HK	DT_BYTES (32)
AnnualDeclarationID	DT_I4
HUB_LoadDate	DT_DBTIMESTAMP
HUB_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.HUB_AnnualDecl, fast load, table lock. All 4 columns auto-match by name.
OnError: Change @PackageName references in SQL_LogErrorOnError to 'BRZ_HUB_AnnualDecl'.

CONFIDENTIAL

4.7 Hub Business Key Reference

Each Hub package uses the same DFT pattern. Only the source SQL and destination table change. Use this table to configure each Hub package.

Hub Business Keys by Table

Hub Table	Staging Source	Business Key Column	Business Key Type
HUB_Category	STG_Category	CategoryID	INT
HUB_Structure	STG_Structure	StructureID	INT
HUB_Activity	STG_Activity	ActivityID	INT
HUB_Taxpayer	STG_Taxpayer	TaxID	VARCHAR(20)
HUB_Owner	STG_Owner	OwnerID	INT
HUB_Officer	STG_Officer	OfficerCode	VARCHAR(20)
HUB_Declaration	STG_MonthlyDeclaration	DeclarationID	INT
HUB_Payment	STG_Payment	PaymentID	INT
HUB_AnnualDecl	STG_AnnualDeclaration	AnnualDeclarationID	INT

Cloning Hub Packages, 2-Variable Change:
Build BRZ_HUB_Category.dtsx first as the template. To clone: 1) Copy package in Solution Explorer → rename. 2) Change v_TableName to the new entity. 3) Change v_BKColumn to the new business key column. All expressions auto-update. 4) Open DFT → OLE_SRC → re-run metadata setup (Columns tab) to pick up the new column types. 5) Open OLE_DST → re-verify column mappings.

CONFIDENTIAL

4.8 Satellite Child Packages

Satellites are more complex than Hubs, they detect attribute changes via HashDiff comparison and maintain history by end-dating previous records. The DFT uses a 6-component pipeline (OLE_SRC → LKP_SAT → CSPL_DetectChanges → UNION_New_Changed → ROW_CNT → OLE_DST) with a post-DFT Execute SQL Task (SQL_EndDateOld) to close previous versions.

❌ **COMMON ERROR, SAT OLE_SRC HASHBYTES without CAST:** Same as Hub packages: wrap ALL HASHBYTES calls with CAST(...AS VARBINARY(32)). Both HUB_*_HK and SAT_HashDiff columns must use CAST. Without it, LKP_SAT silently fails to match due to VARBINARY size mismatch, causing incorrect change detection or data loss.

❌ **COMMON ERROR, LKP_SAT "Redirect rows to error output" vs "no match output":** The LKP_SAT General tab dropdown has THREE options: "Fail component", "Redirect rows to error output", and "Redirect rows to no match output". You MUST select "Redirect rows to no match output" (the THIRD option). If you accidentally select "Redirect rows to error output" (the SECOND option), all new rows on fresh load silently go to the unconnected Error Output path and 0 rows get inserted. This was the root cause of SAT_MonthlyDecl = 0 rows on full load.

BRZ_SAT_Category.dtsx is the fully documented reference template. Remaining Satellite packages document only the differences (v_TableName and source SQL).

CONFIDENTIAL

4.8.1 BRZ_SAT_Category.dtsx, Reference Template

Package Parameters

Parameter	Data Type	Description
p_BatchID	Int32	Current ETL batch ID from orchestrator

Package Variables

Variable Name	Data Type	Default Value	EvaluateAsExpression?
v_TableName	String	Category	No
v_StepName	String	(expression)	Yes
v_LookupSQL	String	(expression)	Yes
v_EndDateSQL	String	(expression)	Yes
v_DestTable	String	(expression)	Yes
v_RowCount	Int32	0	No
v_StepID	Int32	0	No
v_ErrorMessage	String	(empty)	No
v_ErrorSource	String	(expression)	Yes

Key Expression Definitions:

```
v_StepName:   "Load_SAT_" + @[User::v_TableName]
v_DestTable:  "SAT_" + @[User::v_TableName]
v_ErrorSource: @[System::PackageName]
```

v_LookupSQL Expression:

```
"SELECT HUB_" + @[User::v_TableName] + "_HK, SAT_HashDiff"
+ " FROM DV_Bronze.dbo.SAT_" + @[User::v_TableName]
+ " WHERE SAT_EndDate IS NULL"
```

At runtime: SELECT HUB_Category_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Category WHERE SAT_EndDate IS NULL

v_EndDateSQL Expression (close previous versions):

```
"SET NOCOUNT ON; ;WITH LatestPerHK AS ("
+ "SELECT HUB_" + @[User::v_TableName] + "_HK, MAX(SAT_LoadDate) AS MaxLoadDate"
+ " FROM DV_Bronze.dbo.SAT_" + @[User::v_TableName]
+ " WHERE SAT_EndDate IS NULL GROUP BY HUB_" + @[User::v_TableName] + "_HK"
+ " HAVING COUNT(*) > 1)"
+ " UPDATE sat SET sat.SAT_EndDate = lpk.MaxLoadDate"
+ " FROM DV_Bronze.dbo.SAT_" + @[User::v_TableName] + " sat"
+ " INNER JOIN LatestPerHK lpk ON sat.HUB_" + @[User::v_TableName] + "_HK = lpk.HUB_" +
@[User::v_TableName] + "_HK"
+ " WHERE sat.SAT_EndDate IS NULL AND sat.SAT_LoadDate < lpk.MaxLoadDate;"
```

Connection Managers Used

Connection Manager	Database	Used By
CM_DV_Staging	DV_Staging	OLE_SRC (reads staging data with HASHBYTES)
CM_DV_Bronze	DV_Bronze	LKP_SAT, OLE_DST, SQL_EndDateOld
CM_ETL_Control	ETL_Control	SQL_StartStep, SQL_EndStep
CM_ETL_Control_OnError	ETL_Control	OnError handler tasks

© 2026 Mr. Sot So — Royal University of Phnom Penh

Control Flow (4 Steps)

```
SQL_StartStep
    │  (Success)
    ▼
DFT_Load_Satellite     ← OLE_SRC → LKP → CSPL → UNION → ROW_CNT → OLE_DST
    │  (Success)
    ▼
SQL_EndDateOld         ← Close previous versions (UPDATE SAT_EndDate)
    │  (Success)
    ▼
SQL_EndStep
```

CONFIDENTIAL

Step 2: DFT_Load_Satellite (Data Flow Task)

This is the most complex DFT in the system. It computes two hashes (Hub HK + HashDiff), uses Lookup to find current Satellite records, applies Conditional Split for change detection, and merges new + changed rows via Union All before inserting.

Data Flow Diagram

```
OLE_SRC_STG_Category
    │  (rows with HUB_HK + SAT_HashDiff computed in SQL)
    ▼
LKP_SAT_Category       ← Lookup current SAT records (SAT_EndDate IS NULL)
    │
    ├── No Match Output ──────────────────────→ UNION_New_Changed
    │                                                  │
    └── Match Output ─→ CSPL_DetectChanges                  │
                            │                          │
                            ├─ Changed ──────────────┤
                            │                          │
                            └─ Unchanged → (discard)    ▼
                                                ROW_CNT ─→ OLE_DST_SAT_Category
```

6 Components to Add (in order)

#	Component Name	Toolbox Category	Type
1	`OLE_SRC_STG_Category`	Other Sources	OLE DB Source
2	`LKP_SAT_Category`	Common	Lookup
3	`CSPL_DetectChanges`	Common	Conditional Split
4	`UNION_New_Changed`	Common	Union All
5	`ROW_CNT`	Common	Row Count
6	`OLE_DST_SAT_Category`	Other Destinations	OLE DB Destination

Data Flow Path Connections

From	To	Output Type
`OLE_SRC`	`LKP_SAT`	Default Output (all rows)
`LKP_SAT`	`CSPL_DetectChanges`	Match Output (existing records)
`LKP_SAT`	`UNION_New_Changed`	No Match Output (new records)
`CSPL_DetectChanges`	`UNION_New_Changed`	Changed output (HashDiff differs)
`UNION_New_Changed`	`ROW_CNT`	Default Output
`ROW_CNT`	`OLE_DST`	Default Output

© 2026 Mr. Sot So — Royal University of Phnom Penh

Component 1: OLE_SRC (OLE DB Source), CM_DV_Staging

Data Access Mode: SQL command (direct input). The Satellite source SQL is NOT expression-driven because it includes HASHBYTES CONCAT which varies per entity and cannot easily be parameterized.

SQL Command (BRZ_SAT_Category):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.CategoryID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Category_HK,
    CAST(HASHBYTES('SHA2_256',
        CONCAT(
            ISNULL(stg.CategoryName, ''), '|',
            ISNULL(stg.CategoryDescription, ''), '|',
            ISNULL(CAST(stg.IsActive AS NVARCHAR(1)), '')
        )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.CategoryName,
    stg.CategoryDescription,
    stg.IsActive
FROM DV_Staging.dbo.STG_Category stg
```

⚠ **IMPORTANT:** Every attribute in the CONCAT must be wrapped with ISNULL(..., ''). Without this, a single NULL value causes the entire CONCAT to return NULL, making all rows appear 'changed' on every run.

Output Columns:

Column	Data Type	Source
HUB_Category_HK	DT_BYTES (32)	HASHBYTES on BK
SAT_HashDiff	DT_BYTES (32)	HASHBYTES on all attrs
SAT_LoadDate	DT_DBTIMESTAMP	GETDATE()
SAT_RecordSource	DT_WSTR (50)	stg.RecordSource
CategoryName	DT_STR (100)	stg.CategoryName
CategoryDescription	DT_STR (500)	stg.CategoryDescription
IsActive	DT_BOOL	stg.IsActive

© 2026 Mr. Sot So — Royal University of Phnom Penh

Component 2: LKP_SAT (Lookup Transform), CM_DV_Bronze

General Tab: Cache mode = Full cache, No match handling = Redirect rows to no match output.

Connection Tab: Set Lookup SQL via expression on parent DFT → [LKP_SAT].[SqlCommand] → @[User::v_LookupSQL].
At runtime: SELECT HUB_Category_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Category WHERE SAT_EndDate IS NULL

Columns Tab, Step-by-Step Setup:
1. Drag HUB_Category_HK from Input (left) to HUB_Category_HK in Lookup (right), join condition.
2. In the bottom grid, CHECK the SAT_HashDiff checkbox under Lookup Column, we need it for change detection.
3. Set Output Alias = LKP_SAT_HashDiff (rename to avoid conflict with the source column).
4. Verify bottom grid: HUB_Category_HK maps to HUB_Category_HK (join), SAT_HashDiff outputs as LKP_SAT_HashDiff.

Component 3: CSPL_DetectChanges (Conditional Split)
This component receives the Match output from LKP_SAT (existing records). It compares the source HashDiff against the Lookup HashDiff.

Conditional Split Editor:

Output Name	Condition	Order
Changed	`SAT_HashDiff != LKP_SAT_HashDiff`	1
Default Output (Unchanged)	(all remaining rows)	-

✓ **TIP:** 'Unchanged' is the default output. These rows have identical HashDiff values (no attribute changes) and are silently discarded (no downstream connection).

Component 4: UNION_New_Changed (Union All Transform)
Two inputs: LKP No Match Output (new records) and CSPL Changed output. Union All merges them into a single stream for insertion. SSIS auto-maps columns with matching names. Verify all columns appear in the grid.

Components 5-6: ROW_CNT + OLE_DST (OLE DB Destination)
ROW_CNT: VariableName = User::v_RowCount. Counts new + changed rows.

OLE_DST: CM_DV_Bronze, fast load, table lock. Destination table set via expression [OLE_DST].[OpenRowset] = @[User::v_DestTable].

Column Mappings (BRZ_SAT_Category):

Input Column	Destination Column
`HUB_Category_HK`	`HUB_Category_HK`
`SAT_LoadDate`	`SAT_LoadDate`
(not mapped, DEFAULT)	`SAT_EndDate`
`SAT_HashDiff`	`SAT_HashDiff`
`SAT_RecordSource`	`SAT_RecordSource`
`CategoryName`	`CategoryName`
`CategoryDescription`	`CategoryDescription`
`IsActive`	`IsActive`

CONFIDENTIAL

⚠ **IMPORTANT:** SAT_EndDate is NOT mapped, it defaults to NULL (current record). The SQL_EndDateOld task (Step 3) sets it when a newer version arrives.

CONFIDENTIAL

Step 3: SQL_EndDateOld (Execute SQL Task)

After the DFT inserts new satellite rows, this task end-dates the previous versions. It finds any HK with multiple current records (SAT_EndDate IS NULL) and closes all except the latest.

Property	Value
Connection	`CM_DV_Bronze`
SQLSourceType	Direct input (expression override)

Expressions: F4 → Expressions → SqlStatementSource → @[User::v_EndDateSQL]
At runtime with v_TableName = 'Category':

```
SET NOCOUNT ON;
;WITH LatestPerHK AS (
    SELECT HUB_Category_HK, MAX(SAT_LoadDate) AS MaxLoadDate
    FROM DV_Bronze.dbo.SAT_Category
    WHERE SAT_EndDate IS NULL
    GROUP BY HUB_Category_HK
    HAVING COUNT(*) > 1
)
UPDATE sat
SET sat.SAT_EndDate = lpk.MaxLoadDate
FROM DV_Bronze.dbo.SAT_Category sat
INNER JOIN LatestPerHK lpk
    ON sat.HUB_Category_HK = lpk.HUB_Category_HK
WHERE sat.SAT_EndDate IS NULL
  AND sat.SAT_LoadDate < lpk.MaxLoadDate;
```

No parameter mapping needed — SQL fully driven by v_EndDateSQL expression. When v_TableName changes, the SQL auto-updates.

Debugging: Recommended Data Viewer Placements (SAT DFT)

Location	What to Verify
OLE_SRC → LKP	HASHBYTES values for HK and HashDiff
LKP No Match → UNION	New business keys not yet in Satellite
LKP Match → CSPL	Existing records with Lookup HashDiff attached
CSPL Changed → UNION	Only rows where HashDiff differs
UNION → ROW_CNT	Merged stream of new + changed rows

Complete Satellite DFT Property Summary (BRZ_SAT_Category)

Component	Type	Connection	Key Setting
`OLE_SRC`	OLE DB Source	`CM_DV_Staging`	SQL command (direct), HASHBYTES in SQL
`LKP_SAT`	Lookup	`CM_DV_Bronze`	Full cache, redirect no match, v_LookupSQL
`CSPL`	Conditional Split	-	SAT_HashDiff != LKP_SAT_HashDiff
`UNION`	Union All	-	Merges new + changed rows
`ROW_CNT`	Row Count	-	`User::v_RowCount`
`OLE_DST`	OLE DB Destination	`CM_DV_Bronze`	Fast load, v_DestTable

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.8.2 BRZ_SAT_Structure.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	Structure

All expressions auto-update: v_StepName → Load_SAT_Structure, v_DestTable → SAT_Structure, v_LookupSQL → SELECT HUB_Structure_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Structure WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.StructureID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Structure_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(stg.StructureName, ''), '|',
        ISNULL(stg.StructureDescription, ''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)), '')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.StructureName,
    stg.StructureDescription,
    stg.IsActive
FROM DV_Staging.dbo.STG_Structure stg
```

Output Columns:

Column	Data Type
HUB_Structure_HK	DT_BYTES (32)
SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP
SAT_RecordSource	DT_WSTR (50)
StructureName	DT_STR (100)
StructureDescription	DT_STR (500)
IsActive	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_Structure, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_Structure'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.8.3 BRZ_SAT_Activity.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	Activity

All expressions auto-update: v_StepName → Load_SAT_Activity, v_DestTable → SAT_Activity, v_LookupSQL → SELECT HUB_Activity_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Activity WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.ActivityID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Activity_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(stg.ActivityName, ''), '|',
        ISNULL(stg.ActivityDescription, ''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)), '')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.ActivityName,
    stg.ActivityDescription,
    stg.IsActive
FROM DV_Staging.dbo.STG_Activity stg
```

Output Columns:

Column	Data Type
HUB_Activity_HK	DT_BYTES (32)
SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP
SAT_RecordSource	DT_WSTR (50)
ActivityName	DT_STR (100)
ActivityDescription	DT_STR (500)
IsActive	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_Activity, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_Activity'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.8.4 BRZ_SAT_Taxpayer.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	Taxpayer

All expressions auto-update: v_StepName → Load_SAT_Taxpayer, v_DestTable → SAT_Taxpayer, v_LookupSQL → SELECT HUB_Taxpayer_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Taxpayer WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.TaxID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Taxpayer_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(stg.LegalBusinessName,''), '|',
        ISNULL(stg.TradingName,''), '|',
        ISNULL(CAST(stg.CategoryID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.StructureID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.RegistrationDate AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.EstimatedAnnualRevenue AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)),'')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.LegalBusinessName,
    stg.TradingName,
    stg.CategoryID,
    stg.StructureID,
    stg.RegistrationDate,
    stg.EstimatedAnnualRevenue,
    stg.IsActive
FROM DV_Staging.dbo.STG_Taxpayer stg
```

Output Columns:

Column	Data Type
HUB_Taxpayer_HK	DT_BYTES (32)
SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP
SAT_RecordSource	DT_WSTR (50)
LegalBusinessName	DT_STR (300)
TradingName	DT_STR (300)
CategoryID	DT_I4
StructureID	DT_I4
RegistrationDate	DT_DBDATE
EstimatedAnnualRevenue	DT_NUMERIC (18,2)
IsActive	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_Taxpayer, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_Taxpayer'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.8.5 BRZ_SAT_Owner.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	Owner

All expressions auto-update: v_StepName → Load_SAT_Owner, v_DestTable → SAT_Owner, v_LookupSQL → SELECT HUB_Owner_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Owner WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.OwnerID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Owner_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(CAST(stg.TaxpayerID AS NVARCHAR(20)),''), '|',
        ISNULL(stg.OwnerName,''), '|',
        ISNULL(stg.OwnerType,''), '|',
        ISNULL(CAST(stg.OwnershipPercentage AS NVARCHAR(10)),''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)),'')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.TaxpayerID,
    stg.OwnerName,
    stg.OwnerType,
    stg.OwnershipPercentage,
    stg.IsActive
FROM DV_Staging.dbo.STG_Owner stg
```

Output Columns:

Column	Data Type
HUB_Owner_HK	DT_BYTES (32)
SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP
SAT_RecordSource	DT_WSTR (50)
TaxpayerID	DT_I4
OwnerName	DT_STR (200)
OwnerType	DT_STR (50)
OwnershipPercentage	DT_NUMERIC (5,2)
IsActive	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_Owner, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_Owner'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.8.6 BRZ_SAT_Officer.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	Officer

All expressions auto-update: v_StepName → Load_SAT_Officer, v_DestTable → SAT_Officer, v_LookupSQL → SELECT HUB_Officer_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Officer WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.OfficerCode AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Officer_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(CAST(stg.OfficerID AS NVARCHAR(20)),''), '|',
        ISNULL(stg.FirstName,''), '|',
        ISNULL(stg.LastName,''), '|',
        ISNULL(stg.Department,''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)),'')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.OfficerID,
    stg.FirstName,
    stg.LastName,
    stg.Department,
    stg.IsActive
FROM DV_Staging.dbo.STG_Officer stg
```

Output Columns:

Column	Data Type
HUB_Officer_HK	DT_BYTES (32)
SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP
SAT_RecordSource	DT_WSTR (50)
OfficerID	DT_I4
FirstName	DT_STR (100)
LastName	DT_STR (100)
Department	DT_STR (100)
IsActive	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_Officer, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_Officer'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.8.7 BRZ_SAT_MonthlyDecl.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	MonthlyDecl

All expressions auto-update: v_StepName → Load_SAT_MonthlyDecl, v_DestTable → SAT_MonthlyDecl, v_LookupSQL → SELECT HUB_Declaration_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_MonthlyDecl WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Declaration_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(CAST(stg.TaxpayerID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.DeclarationMonth AS NVARCHAR(2)),''), '|',
        ISNULL(CAST(stg.DeclarationYear AS NVARCHAR(4)),''), '|',
        ISNULL(CAST(stg.GrossRevenue AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.TaxableRevenue AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.TaxAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.PenaltyAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.InterestAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.TotalAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.DeclarationDate AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.DueDate AS NVARCHAR(30)),''), '|',
        ISNULL(stg.Status,''), '|',
        ISNULL(CAST(stg.OfficerID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)),'')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.TaxpayerID,
    stg.DeclarationMonth,
    stg.DeclarationYear,
    stg.GrossRevenue,
    stg.TaxableRevenue,
    stg.TaxAmount,
    stg.PenaltyAmount,
    stg.InterestAmount,
    stg.TotalAmount,
    stg.DeclarationDate,
    stg.DueDate,
    stg.Status,
    stg.OfficerID,
    stg.IsActive
FROM DV_Staging.dbo.STG_MonthlyDeclaration stg
```

Output Columns:

Column	Data Type
HUB_Declaration_HK	DT_BYTES (32)

SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP
SAT_RecordSource	DT_WSTR (50)
TaxpayerID	DT_I4
DeclarationMonth	DT_I4
DeclarationYear	DT_I4
GrossRevenue	DT_NUMERIC (18,2)
TaxableRevenue	DT_NUMERIC (18,2)
TaxAmount	DT_NUMERIC (18,2)
PenaltyAmount	DT_NUMERIC (18,2)
InterestAmount	DT_NUMERIC (18,2)
TotalAmount	DT_NUMERIC (18,2)
DeclarationDate	DT_DBDATE
DueDate	DT_DBDATE
Status	DT_STR (50)
OfficerID	DT_I4
IsActive	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_MonthlyDecl, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_MonthlyDecl'.

CONFIDENTIAL

4.8.8 BRZ_SAT_Payment.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	Payment

All expressions auto-update: v_StepName → Load_SAT_Payment, v_DestTable → SAT_Payment, v_LookupSQL → SELECT HUB_Payment_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_Payment WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.PaymentID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Payment_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(CAST(stg.TaxpayerID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.DeclarationID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.AnnualDeclarationID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.PaymentAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.PaymentDate AS NVARCHAR(30)),''), '|',
        ISNULL(stg.PaymentMethod,''), '|',
        ISNULL(stg.ReferenceNumber,''), '|',
        ISNULL(stg.Status,''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)),'')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.TaxpayerID,
    stg.DeclarationID,
    stg.AnnualDeclarationID,
    stg.PaymentAmount,
    stg.PaymentDate,
    stg.PaymentMethod,
    stg.ReferenceNumber,
    stg.Status,
    stg.IsActive
FROM DV_Staging.dbo.STG_Payment stg
```

Output Columns:

Column	Data Type
HUB_Payment_HK	DT_BYTES (32)
SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP
SAT_RecordSource	DT_WSTR (50)
TaxpayerID	DT_I4
DeclarationID	DT_I4
AnnualDeclarationID	DT_I4
PaymentAmount	DT_NUMERIC (18,2)
PaymentDate	DT_DBDATE
PaymentMethod	DT_STR (50)
ReferenceNumber	DT_STR (100)

© 2026 Mr. Sot So — Royal University of Phnom Penh

`Status`	DT_STR (50)
`IsActive`	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_Payment, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_Payment'.

CONFIDENTIAL

4.8.9 BRZ_SAT_AnnualDecl.dtsx, Clone from Category

Variable Change:

Variable	New Value
v_TableName	AnnualDecl

All expressions auto-update: v_StepName → Load_SAT_AnnualDecl, v_DestTable → SAT_AnnualDecl, v_LookupSQL → SELECT HUB_AnnualDecl_HK, SAT_HashDiff FROM DV_Bronze.dbo.SAT_AnnualDecl WHERE SAT_EndDate IS NULL, v_EndDateSQL auto-updates table/column names.

OLE_SRC SQL Command (direct input, not expression-driven):

```
SELECT
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.AnnualDeclarationID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_AnnualDecl_HK,
    CAST(HASHBYTES('SHA2_256', CONCAT(
        ISNULL(CAST(stg.TaxpayerID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.DeclarationYear AS NVARCHAR(4)),''), '|',
        ISNULL(CAST(stg.GrossRevenue AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.TaxableRevenue AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.TaxAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.PenaltyAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.InterestAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.TotalAmount AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.DeclarationDate AS NVARCHAR(30)),''), '|',
        ISNULL(CAST(stg.DueDate AS NVARCHAR(30)),''), '|',
        ISNULL(stg.Status,''), '|',
        ISNULL(CAST(stg.OfficerID AS NVARCHAR(20)),''), '|',
        ISNULL(CAST(stg.IsActive AS NVARCHAR(1)),'')
    )) AS VARBINARY(32)) AS SAT_HashDiff,
    GETDATE()              AS SAT_LoadDate,
    stg.RecordSource       AS SAT_RecordSource,
    stg.TaxpayerID,
    stg.DeclarationYear,
    stg.GrossRevenue,
    stg.TaxableRevenue,
    stg.TaxAmount,
    stg.PenaltyAmount,
    stg.InterestAmount,
    stg.TotalAmount,
    stg.DeclarationDate,
    stg.DueDate,
    stg.Status,
    stg.OfficerID,
    stg.IsActive
FROM DV_Staging.dbo.STG_AnnualDeclaration stg
```

Output Columns:

Column	Data Type
HUB_AnnualDecl_HK	DT_BYTES (32)
SAT_HashDiff	DT_BYTES (32)
SAT_LoadDate	DT_DBTIMESTAMP

SAT_RecordSource	DT_WSTR (50)
TaxpayerID	DT_I4
DeclarationYear	DT_I4
GrossRevenue	DT_NUMERIC (18,2)
TaxableRevenue	DT_NUMERIC (18,2)
TaxAmount	DT_NUMERIC (18,2)
PenaltyAmount	DT_NUMERIC (18,2)
InterestAmount	DT_NUMERIC (18,2)
TotalAmount	DT_NUMERIC (18,2)
DeclarationDate	DT_DBDATE
DueDate	DT_DBDATE
Status	DT_STR (50)
OfficerID	DT_I4
IsActive	DT_BOOL

OLE_DST: CM_DV_Bronze, dbo.SAT_AnnualDecl, fast load, table lock. SAT_EndDate is NOT mapped (defaults to NULL). All other columns auto-match.
OnError: Change @PackageName to 'BRZ_SAT_AnnualDecl'.

CONFIDENTIAL

4.9 Satellite Attribute Reference

Use this table to configure the source SQL for each Satellite package. The HashDiff CONCAT must include ALL descriptive attributes.

Satellite Descriptive Attributes by Table

Satellite Table	Staging Source	Descriptive Attributes (in HashDiff)
SAT_Category	STG_Category	CategoryName, CategoryDescription, IsActive
SAT_Structure	STG_Structure	StructureName, StructureDescription, IsActive
SAT_Activity	STG_Activity	ActivityName, ActivityDescription, IsActive
SAT_Taxpayer	STG_Taxpayer	LegalBusinessName, TradingName, CategoryID, StructureID, RegistrationDate, EstimatedAnnualRevenue, IsActive
SAT_Owner	STG_Owner	TaxpayerID, OwnerName, OwnerType, OwnershipPercentage, IsActive
SAT_Officer	STG_Officer	OfficerID, FirstName, LastName, Department, IsActive
SAT_MonthlyDecl	STG_MonthlyDeclaration	TaxpayerID, DeclarationMonth, DeclarationYear, GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount, DeclarationDate, DueDate, Status, OfficerID, IsActive
SAT_Payment	STG_Payment	TaxpayerID, DeclarationID, AnnualDeclarationID, PaymentAmount, PaymentDate, PaymentMethod, ReferenceNumber, Status, IsActive
SAT_AnnualDecl	STG_AnnualDeclaration	TaxpayerID, DeclarationYear, GrossRevenue, TaxableRevenue, TaxAmount, PenaltyAmount, InterestAmount, TotalAmount, DeclarationDate, DueDate, Status, OfficerID, IsActive

⚠ **IMPORTANT:** HashDiff ISNULL Wrapping Is Critical. Every attribute in the CONCAT must be wrapped with ISNULL(..., ''). Without this, a single NULL causes the entire CONCAT to return NULL, making all rows appear 'changed' on every load.

Cloning Satellite Packages — 1-Variable + Source SQL:
Build BRZ_SAT_Category.dtsx first as the template. To clone: 1) Copy package → rename. 2) Change v_TableName to the new entity. All expressions auto-update (v_LookupSQL, v_EndDateSQL, v_DestTable). 3) Open DFT → OLE_SRC → replace the SQL command with the new entity's HASHBYTES query. 4) Re-run metadata setup (Columns tab). 5) Re-verify OLE_DST column mappings. 6) Update @PackageName in OnError.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.10 Link Child Packages

All 5 Link child packages follow the same pattern as Hubs: a 3-step Control Flow (SQL_StartStep → DFT_Load_Link → SQL_EndStep) with a 4-component DFT (OLE_SRC → LKP_LNK → ROW_CNT → OLE_DST). The key difference: the source SQL computes a composite hash key (from two or more FK columns) plus individual Hub HKs for each foreign key.

> ❌ **COMMON ERROR, LKP_LNK must use "Redirect rows to no match output":** Same as Satellite packages. On fresh load the Link table is empty, so every row is No Match. If LKP_LNK General tab is set to "Fail component" or "Redirect rows to error output", the package fails or inserts 0 rows. Must be "Redirect rows to no match output".

BRZ_LNK_TaxpayerDeclaration.dtsx is the reference template. Remaining Link packages document only the differences.

CONFIDENTIAL

4.10.1 BRZ_LNK_TaxpayerDeclaration.dtsx, Reference Template

Package Parameters

Parameter	Data Type	Description
p_BatchID	Int32	Current ETL batch ID from orchestrator

Package Variables

Variable Name	Data Type	Default Value	EvaluateAsExpression?
v_TableName	String	TaxpayerDeclaration	No
v_StepName	String	(expression)	Yes
v_LookupSQL	String	(expression)	Yes
v_DestTable	String	(expression)	Yes
v_RowCount	Int32	0	No
v_StepID	Int32	0	No
v_ErrorMessage	String	(empty)	No
v_ErrorSource	String	(expression)	Yes

Expression Definitions:

```
v_StepName:   "Load_LNK_" + @[User::v_TableName]
v_DestTable:  "LNK_" + @[User::v_TableName]
v_LookupSQL:  "SELECT LNK_" + @[User::v_TableName] + "_HK FROM DV_Bronze.dbo.LNK_" +
@[User::v_TableName]
```

Connection Managers Used

Connection Manager	Database	Used By
CM_DV_Staging	DV_Staging	OLE_SRC (reads staging + joins for FK resolution)
CM_DV_Bronze	DV_Bronze	LKP_LNK + OLE_DST
CM_ETL_Control	ETL_Control	SQL_StartStep, SQL_EndStep
CM_ETL_Control_OnError	ETL_Control	OnError handler tasks

CONFIDENTIAL

Control Flow (3 Steps), same as Hub packages

```
SQL_StartStep  →  DFT_Load_Link  →  SQL_EndStep
```

Step 2: DFT_Load_Link (Data Flow Task)

Same 4-component pattern as Hub DFTs: OLE_SRC → LKP_LNK → ROW_CNT → OLE_DST. The source SQL computes THREE hashes (composite LNK_HK + two individual HUB_HKs).

Data Flow Diagram

```
OLE_SRC_STG_MonthlyDeclaration
    │  (rows with composite LNK_HK + individual HUB_HKs)
    ▼
LKP_LNK_TaxpayerDeclaration
    │
    ├── Match Output ──────────→ (discarded, Link already exists)
    │
    └── No Match Output —→ ROW_CNT —→ OLE_DST_LNK_TaxpayerDeclaration
```

OLE_SRC SQL Command (BRZ_LNK_TaxpayerDeclaration):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        CONCAT(
            UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
            UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX))))
        )) AS VARBINARY(32)) AS LNK_TaxpayerDeclaration_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Taxpayer_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Declaration_HK,
    GETDATE()          AS LNK_LoadDate,
    stg.RecordSource   AS LNK_RecordSource
FROM DV_Staging.dbo.STG_MonthlyDeclaration stg
INNER JOIN DV_Staging.dbo.STG_Taxpayer stg_tp ON stg.TaxpayerID = stg_tp.TaxpayerID
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT DISTINCT CAST(HASHBYTES('SHA2_256', CONCAT(UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX)))))) AS VARBINARY(32)) AS LNK_TaxpayerDeclaration_HK,
CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX))))) AS VARBINARY(32)) AS
HUB_Taxpayer_HK, CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX))))) AS
VARBINARY(32)) AS HUB_Declaration_HK, GETDATE() AS LNK_LoadDate, stg.RecordSource AS LNK_RecordSource FROM
DV_Staging.dbo.STG_MonthlyDeclaration stg INNER JOIN DV_Staging.dbo.STG_Taxpayer stg_tp ON stg.TaxpayerID =
stg_tp.TaxpayerID
```

⚠ **IMPORTANT:** Link TaxpayerID resolves via HUB_Taxpayer.TaxID, not the source TaxpayerID INT. The staging INNER JOIN resolves TaxpayerID to TaxID, then HASHBYTES computes HUB_Taxpayer_HK from TaxID. This is because HUB_Taxpayer uses TaxID as its business key, not TaxpayerID.

Output Columns:

Column	Data Type	Source
LNK_TaxpayerDeclaration_HK	DT_BYTES (32)	Composite hash (TaxID|DeclarationID)
HUB_Taxpayer_HK	DT_BYTES (32)	HASHBYTES on TaxID
HUB_Declaration_HK	DT_BYTES (32)	HASHBYTES on DeclarationID

© 2026 Mr. Sot So — Royal University of Phnom Penh

LNK_LoadDate	DT_DBTIMESTAMP	GETDATE()
LNK_RecordSource	DT_WSTR (50)	stg.RecordSource

LKP_LNK: CM_DV_Bronze, Full cache, redirect no match. v_LookupSQL evaluates to: SELECT LNK_TaxpayerDeclaration_HK FROM DV_Bronze.dbo.LNK_TaxpayerDeclaration. Match-only lookup (no output columns needed).

OLE_DST: CM_DV_Bronze, dbo.LNK_TaxpayerDeclaration, fast load, table lock. All 5 columns auto-match.

Complete Link DFT Property Summary

Component	Type	Connection	Key Setting
OLE_SRC	OLE DB Source	CM_DV_Staging	SQL command (direct), 3 HASHBYTES
LKP_LNK	Lookup	CM_DV_Bronze	Full cache, redirect no match, v_LookupSQL
ROW_CNT	Row Count	-	User::v_RowCount
OLE_DST	OLE DB Destination	CM_DV_Bronze	Fast load, v_DestTable

CONFIDENTIAL

4.10.2 BRZ_LNK_DeclarationPayment.dtsx, Clone from TaxpayerDeclaration

Variable Change:

Variable	New Value
v_TableName	DeclarationPayment

Expressions auto-update: v_StepName → Load_LNK_DeclarationPayment, v_DestTable → LNK_DeclarationPayment, v_LookupSQL → SELECT LNK_DeclarationPayment_HK FROM DV_Bronze.dbo.LNK_DeclarationPayment

OLE_SRC SQL Command (direct input):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        CONCAT(
            UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX)))), '|',
            UPPER(TRIM(CAST(stg.PaymentID AS NVARCHAR(MAX))))
        )) AS VARBINARY(32)) AS LNK_DeclarationPayment_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Declaration_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.PaymentID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Payment_HK,
    GETDATE()          AS LNK_LoadDate,
    stg.RecordSource   AS LNK_RecordSource
FROM DV_Staging.dbo.STG_Payment stg
WHERE stg.DeclarationID IS NOT NULL
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT DISTINCT CAST(HASHBYTES('SHA2_256', CONCAT(UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX)))),
'|', UPPER(TRIM(CAST(stg.PaymentID AS NVARCHAR(MAX)))))) AS VARBINARY(32)) AS LNK_DeclarationPayment_HK,
CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg.DeclarationID AS NVARCHAR(MAX))))) AS VARBINARY(32)) AS
HUB_Declaration_HK, CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg.PaymentID AS NVARCHAR(MAX))))) AS
VARBINARY(32)) AS HUB_Payment_HK, GETDATE() AS LNK_LoadDate, stg.RecordSource AS LNK_RecordSource FROM
DV_Staging.dbo.STG_Payment stg WHERE stg.DeclarationID IS NOT NULL
```

Output Columns:

Column	Data Type
LNK_DeclarationPayment_HK	DT_BYTES (32)
HUB_Declaration_HK	DT_BYTES (32)
HUB_Payment_HK	DT_BYTES (32)
LNK_LoadDate	DT_DBTIMESTAMP
LNK_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.LNK_DeclarationPayment, fast load, table lock. All 5 columns auto-match.
OnError: Change @PackageName to 'BRZ_LNK_DeclarationPayment'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.10.3 BRZ_LNK_TaxpayerOfficer.dtsx, Clone from TaxpayerDeclaration

Variable Change:

Variable	New Value
v_TableName	TaxpayerOfficer

Expressions auto-update: v_StepName → Load_LNK_TaxpayerOfficer, v_DestTable → LNK_TaxpayerOfficer, v_LookupSQL → SELECT LNK_TaxpayerOfficer_HK FROM DV_Bronze.dbo.LNK_TaxpayerOfficer

OLE_SRC SQL Command (direct input):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        CONCAT(
            UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
            UPPER(TRIM(CAST(stg_ofc.OfficerCode AS NVARCHAR(MAX))))
        )) AS VARBINARY(32)) AS LNK_TaxpayerOfficer_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Taxpayer_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg_ofc.OfficerCode AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Officer_HK,
    GETDATE()          AS LNK_LoadDate,
    stg.RecordSource   AS LNK_RecordSource
FROM DV_Staging.dbo.STG_MonthlyDeclaration stg
INNER JOIN DV_Staging.dbo.STG_Taxpayer stg_tp ON stg.TaxpayerID = stg_tp.TaxpayerID
INNER JOIN DV_Staging.dbo.STG_Officer stg_ofc ON stg.OfficerID = stg_ofc.OfficerID
WHERE stg.OfficerID IS NOT NULL
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT DISTINCT CAST(HASHBYTES('SHA2_256', CONCAT(UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
UPPER(TRIM(CAST(stg_ofc.OfficerCode AS NVARCHAR(MAX)))))) AS VARBINARY(32)) AS LNK_TaxpayerOfficer_HK,
CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX))))) AS VARBINARY(32)) AS
HUB_Taxpayer_HK, CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg_ofc.OfficerCode AS NVARCHAR(MAX))))) AS
VARBINARY(32)) AS HUB_Officer_HK, GETDATE() AS LNK_LoadDate, stg.RecordSource AS LNK_RecordSource FROM
DV_Staging.dbo.STG_MonthlyDeclaration stg INNER JOIN DV_Staging.dbo.STG_Taxpayer stg_tp ON stg.TaxpayerID =
stg_tp.TaxpayerID INNER JOIN DV_Staging.dbo.STG_Officer stg_ofc ON stg.OfficerID = stg_ofc.OfficerID WHERE
stg.OfficerID IS NOT NULL
```

Output Columns:

Column	Data Type
LNK_TaxpayerOfficer_HK	DT_BYTES (32)
HUB_Taxpayer_HK	DT_BYTES (32)
HUB_Officer_HK	DT_BYTES (32)
LNK_LoadDate	DT_DBTIMESTAMP
LNK_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.LNK_TaxpayerOfficer, fast load, table lock. All 5 columns auto-match.
OnError: Change @PackageName to 'BRZ_LNK_TaxpayerOfficer'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.10.4 BRZ_LNK_TaxpayerOwner.dtsx, Clone from TaxpayerDeclaration

Variable Change:

Variable	New Value
v_TableName	TaxpayerOwner

Expressions auto-update: v_StepName → Load_LNK_TaxpayerOwner, v_DestTable → LNK_TaxpayerOwner, v_LookupSQL → SELECT LNK_TaxpayerOwner_HK FROM DV_Bronze.dbo.LNK_TaxpayerOwner

OLE_SRC SQL Command (direct input):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        CONCAT(
            UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
            UPPER(TRIM(CAST(stg.OwnerID AS NVARCHAR(MAX))))
        )) AS VARBINARY(32)) AS LNK_TaxpayerOwner_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Taxpayer_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.OwnerID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Owner_HK,
    GETDATE()          AS LNK_LoadDate,
    stg.RecordSource   AS LNK_RecordSource
FROM DV_Staging.dbo.STG_Owner stg
INNER JOIN DV_Staging.dbo.STG_Taxpayer stg_tp ON stg.TaxpayerID = stg_tp.TaxpayerID
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT DISTINCT CAST(HASHBYTES('SHA2_256', CONCAT(UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
UPPER(TRIM(CAST(stg.OwnerID AS NVARCHAR(MAX)))))) AS VARBINARY(32)) AS LNK_TaxpayerOwner_HK,
CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX))))) AS VARBINARY(32)) AS
HUB_Taxpayer_HK, CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg.OwnerID AS NVARCHAR(MAX))))) AS
VARBINARY(32)) AS HUB_Owner_HK, GETDATE() AS LNK_LoadDate, stg.RecordSource AS LNK_RecordSource FROM
DV_Staging.dbo.STG_Owner stg INNER JOIN DV_Staging.dbo.STG_Taxpayer stg_tp ON stg.TaxpayerID =
stg_tp.TaxpayerID
```

Output Columns:

Column	Data Type
LNK_TaxpayerOwner_HK	DT_BYTES (32)
HUB_Taxpayer_HK	DT_BYTES (32)
HUB_Owner_HK	DT_BYTES (32)
LNK_LoadDate	DT_DBTIMESTAMP
LNK_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.LNK_TaxpayerOwner, fast load, table lock. All 5 columns auto-match.
OnError: Change @PackageName to 'BRZ_LNK_TaxpayerOwner'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.10.5 BRZ_LNK_TaxpayerAnnualDecl.dtsx, Clone from TaxpayerDeclaration

Variable Change:

Variable	New Value
v_TableName	TaxpayerAnnualDecl

Expressions auto-update: v_StepName → Load_LNK_TaxpayerAnnualDecl, v_DestTable → LNK_TaxpayerAnnualDecl, v_LookupSQL → SELECT LNK_TaxpayerAnnualDecl_HK FROM DV_Bronze.dbo.LNK_TaxpayerAnnualDecl

OLE_SRC SQL Command (direct input):

```
SELECT DISTINCT
    CAST(HASHBYTES('SHA2_256',
        CONCAT(
            UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
            UPPER(TRIM(CAST(stg.AnnualDeclarationID AS NVARCHAR(MAX))))
        )) AS VARBINARY(32)) AS LNK_TaxpayerAnnualDecl_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_Taxpayer_HK,
    CAST(HASHBYTES('SHA2_256',
        UPPER(TRIM(CAST(stg.AnnualDeclarationID AS NVARCHAR(MAX)))))
        AS VARBINARY(32)) AS HUB_AnnualDecl_HK,
    GETDATE()          AS LNK_LoadDate,
    stg.RecordSource   AS LNK_RecordSource
FROM DV_Staging.dbo.STG_AnnualDeclaration stg
INNER JOIN DV_Staging.dbo.STG_Taxpayer stg_tp ON stg.TaxpayerID = stg_tp.TaxpayerID
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT DISTINCT CAST(HASHBYTES('SHA2_256', CONCAT(UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX)))), '|',
UPPER(TRIM(CAST(stg.AnnualDeclarationID AS NVARCHAR(MAX))))))) AS VARBINARY(32)) AS
LNK_TaxpayerAnnualDecl_HK, CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg_tp.TaxID AS NVARCHAR(MAX))))) AS
VARBINARY(32)) AS HUB_Taxpayer_HK, CAST(HASHBYTES('SHA2_256', UPPER(TRIM(CAST(stg.AnnualDeclarationID AS
NVARCHAR(MAX))))) AS VARBINARY(32)) AS HUB_AnnualDecl_HK, GETDATE() AS LNK_LoadDate, stg.RecordSource AS
LNK_RecordSource FROM DV_Staging.dbo.STG_AnnualDeclaration stg INNER JOIN DV_Staging.dbo.STG_Taxpayer
stg_tp ON stg.TaxpayerID = stg_tp.TaxpayerID
```

Output Columns:

Column	Data Type
LNK_TaxpayerAnnualDecl_HK	DT_BYTES (32)
HUB_Taxpayer_HK	DT_BYTES (32)
HUB_AnnualDecl_HK	DT_BYTES (32)
LNK_LoadDate	DT_DBTIMESTAMP
LNK_RecordSource	DT_WSTR (50)

OLE_DST: CM_DV_Bronze, dbo.LNK_TaxpayerAnnualDecl, fast load, table lock. All 5 columns auto-match.
OnError: Change @PackageName to 'BRZ_LNK_TaxpayerAnnualDecl'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4.11 Link FK Mapping Reference

Use this table to configure the source SQL for each Link package. Each Link connects two Hubs via foreign key columns from the staging source.

Link Foreign Key Mapping by Table

Link Table	Hub Side A (FK Column)	Hub Side B (FK Column)	Staging Source
LNK_TaxpayerDeclaration	TaxpayerID → HUB_Taxpayer	DeclarationID → HUB_Declaration	STG_MonthlyDeclaration
LNK_DeclarationPayment	DeclarationID → HUB_Declaration	PaymentID → HUB_Payment	STG_Payment
LNK_TaxpayerOfficer	TaxpayerID → HUB_Taxpayer	OfficerID → HUB_Officer	STG_MonthlyDeclaration
LNK_TaxpayerOwner	TaxpayerID → HUB_Taxpayer	OwnerID → HUB_Owner	STG_Owner
LNK_TaxpayerAnnualDecl	TaxpayerID → HUB_Taxpayer	AnnualDeclarationID → HUB_AnnualDecl	STG_AnnualDeclaration

⚠ **IMPORTANT:** Link TaxpayerID Resolves via HUB_Taxpayer_HK, Not HUB_Taxpayer.TaxID. The staging TaxpayerID column is a surrogate key. To compute HUB_Taxpayer_HK, you must JOIN to STG_Taxpayer to get TaxID (the business key), then HASHBYTES on TaxID. Similarly, OfficerID resolves via STG_Officer.OfficerCode.

Cloning Link Packages — 1-Variable + Source SQL:
Build BRZ_LNK_TaxpayerDeclaration.dtsx first. To clone: 1) Copy package → rename. 2) Change v_TableName to new entity. Expressions auto-update. 3) Replace OLE_SRC SQL command with new entity's HASHBYTES query (note different staging joins and FK columns). 4) Re-run metadata setup. 5) Re-verify OLE_DST column mappings. 6) Update @PackageName in OnError.

4.12 Stored Procedures (Alternative Approach)

The DFT-based approach (Sections 4.6–11) is the primary implementation. As an alternative, the same logic can be implemented as stored procedures called from Execute SQL Tasks. This is useful for teams who prefer SQL-based ETL over SSIS Data Flow Tasks.

When to Consider Stored Procedures Instead of DFT

Very large tables (millions of rows): SP with set-based INSERT WHERE NOT EXISTS can outperform Lookup Transform.

Complex multi-step logic: Satellite end-dating + insert is a single transaction in SP vs two steps in DFT.

DBA-managed environments: SPs are easier to modify without opening Visual Studio.

If using SPs, replace DFT_Load_Hub/Satellite/Link with an Execute SQL Task calling the SP, and add v_SPName variable with expression-driven SqlStatementSource (same pattern as staging SQL_LoadSP).

4.13 OnError Handler (Bronze Package Level)

Every Bronze child package (Hub, Satellite, Link) has an OnError event handler at the package level. The pattern is identical to staging child packages.

OnError Event Handler Tasks (3 steps)

Step	Task	Description
1	SCR_CaptureError (Script Task)	Captures System::ErrorDescription into User::v_ErrorMessage
2	SQL_EndStepOnError (Execute SQL Task)	Calls usp_EndStep(@StepID, 'Failed', 0) to close the step as failed.
3	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError(@BatchID, 'Error', @ErrorSource, @ErrorMessage) to record error details.

Critical: OnError Tasks Require Separate Connection + NotSupported

1. Connection: Use CM_ETL_Control_OnError (not CM_ETL_Control) for SQL_EndStepOnError and SQL_LogErrorOnError.
2. TransactionOption: Set to NotSupported on all 3 OnError tasks via F4 Properties.

Without BOTH settings, OnError writes execute successfully but get rolled back by the parent's transaction.

Step 1: SCR_CaptureError (Script Task)

Step-by-step Configuration:

- 1. In the Control Flow, click the package background → Event Handlers tab.
- 2. In the Event handler dropdown, select OnError.
- 3. Click the link 'Click here to create an OnError event handler for executable BRZ_HUB_Category'.
- 4. Drag a Script Task onto the OnError canvas. Name it SCR_CaptureError.
- 5. Double-click SCR_CaptureError to open the Script Task Editor.
- 6. Set ReadOnlyVariables: System::ErrorDescription
- 7. Set ReadWriteVariables: User::v_ErrorMessage
- 8. Click Edit Script... and enter the code below.
- 9. Click OK.

Script Task Code (C#)

```
public void Main()
{
    string errorDesc = Dts.Variables["System::ErrorDescription"].Value.ToString();
    Dts.Variables["User::v_ErrorMessage"].Value = errorDesc;
    Dts.TaskResult = (int)ScriptResults.Success;
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
}
```

Why Script Task Instead of Direct Expression?

System::ErrorDescription is a read-only system variable available only in event handler scope.
SSIS does not allow direct expression assignment from System variables to User variables.
The Script Task bridges this gap: it reads System::ErrorDescription and writes to User::v_ErrorMessage.
This is the standard SSIS pattern (Option 2 per Sot So's preferred error handling approach).

Step 2: SQL_EndStepOnError (Execute SQL Task)

General Tab

Property	Value
Name	SQL_EndStepOnError
Connection	CM_ETL_Control_OnError
SQLSourceType	Direct input
ResultSet	None

SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_EndStep
    @StepID       = ?,
    @Status       = 'Failed',
    @RowCount     = 0,
    @ErrorMessage = ?;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
User::v_StepID	Input	LONG	0	@StepID
User::v_ErrorMessage	Input	NVARCHAR	1	@ErrorMessage

Step 3: SQL_LogErrorOnError (Execute SQL Task)

General Tab

Property	Value

CONFIDENTIAL

Name	SQL_LogErrorOnError
Connection	CM_ETL_Control_OnError
SQLSourceType	Direct input
ResultSet	None

SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_LogError
    @BatchID       = ?,
    @StepLogID     = ?,
    @ErrorSeverity = 'Error',
    @ErrorSource   = ?,
    @ErrorMessage  = ?;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_StepID	Input	LONG	1	@StepLogID
User::v_ErrorSource	Input	NVARCHAR	2	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	3	@ErrorMessage

Error Propagation After OnError

After the OnError handler completes, the error propagates UP to the parent package:
Child OnError → Orchestrator OnError → Master OnError.
The child package does NOT stop the error, it only logs it at the step level.
The orchestrator (BRZ_Load_All_Hubs.dtsx) decides whether to continue or stop based on its own OnError handler.
Default behavior: If a child fails, the orchestrator stops executing subsequent children.
To continue despite errors, set MaximumErrorCount = 0 on the orchestrator's Execute Package Task.

4.14 OnError Handler (Orchestrator Level, Bronze Layer)

The three Bronze orchestrators (BRZ_Load_All_Hubs.dtsx, BRZ_Load_All_Satellites.dtsx, BRZ_Load_All_Links.dtsx) each have their own OnError handler. Like the staging orchestrator, they call usp_LogError only (no usp_EndStep) because they do not own any steps.

OnError Event Handler Tasks (2 steps)

© 2026 Mr. Sot So — Royal University of Phnom Penh

#	Task	Description
1	SCR_CaptureError (Script Task)	Reads System::ErrorDescription into User::v_ErrorMessage.
2	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError only. No usp_EndStep, orchestrator does not own steps.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on both OnError tasks.

Step 1: SCR_CaptureError (Script Task)

Identical to Section 3.11 and Section 4.13 Step 1. Reads System::ErrorDescription into User::v_ErrorMessage.

Step 2: SQL_LogErrorOnError (Execute SQL Task)

General Tab

Property	Value
Name	SQL_LogErrorOnError
Connection	CM_ETL_Control_OnError
SQLSourceType	Direct input
ResultSet	None

SQLStatement, BRZ_Load_All_Hubs.dtsx

```
SET NOCOUNT ON;

EXEC dbo.usp_LogError
    @BatchID       = ?,
    @ErrorSeverity = 'Error',
    @ErrorSource   = ?,
    @ErrorMessage  = ?;
```

v_ErrorSource, Automatic via Expression

v_ErrorSource uses EvaluateAsExpression = True with expression: @[System::PackageName].
At runtime, SSIS resolves this to the current package name (e.g., BRZ_Load_All_Hubs).
No manual change needed when cloning orchestrator packages.

CONFIDENTIAL

Parameter Mapping Tab (same for all 3)

Variable Name	Direction	Data Type	Param #	Maps To
$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_ErrorSource	Input	NVARCHAR	1	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	2	@ErrorMessage

Bronze Orchestrator Sibling Behavior

BRZ_Load_All_Hubs has 9 EPT tasks running sequentially. If HUB_Category fails, the remaining 8 Hub packages stop by default (MaximumErrorCount = 1).

However, Hub packages are independent of each other, HUB_Taxpayer does not depend on HUB_Category.
To load as many Hubs as possible despite failures, use Completion constraints (blue arrows) instead of Success.

Satellite and Link orchestrators follow the same logic. Satellite packages depend on their parent Hub existing, but they are independent of each other (SAT_Taxpayer does not depend on SAT_Category).

Package Variable Requirements

Each Bronze orchestrator needs two variables for OnError:
User::v_ErrorMessage (String, default empty)
User::v_ErrorSource (String, Expression: @[System::PackageName])

No v_StepID is needed because orchestrators do not call usp_StartStep / usp_EndStep.
p_BatchID is already available as a package parameter (received from Master_Full_Load.dtsx).

© 2026 Mr. Sot So — Royal University of Phnom Penh

5. Silver Packages, Detailed Implementation

The Silver layer implements the Business Vault: Point-in-Time (PIT) tables for efficient querying, Bridge tables for pre-joined relationships, and Business Vault computed tables. It reads from DV_Bronze and writes to DV_Silver. One orchestrator package (SLV_Load_All.dtsx) coordinates 6 target objects.

5.1 Package Inventory

Orchestrator Package (03_Silver folder)

Package	Called By	Purpose
SLV_Load_All.dtsx	Master_Full_Load / Master_Incremental_Load	Loads all 6 Silver objects from Bronze

Child Packages (03_Silver folder)

Package	Type	Target	Source(s)
SLV_PIT_Taxpayer.dtsx	PIT	PIT_Taxpayer	HUB_Taxpayer + SAT_Taxpayer
SLV_PIT_Declaration.dtsx	PIT	PIT_Declaration	HUB_Declaration + SAT_MonthlyDecl
SLV_PIT_Payment.dtsx	PIT	PIT_Payment	HUB_Payment + SAT_Payment
SLV_BRG_Taxpayer_Owner.dtsx	Bridge	BRG_Taxpayer_Owner	LNK_TaxpayerOwner + HUB_Taxpayer + HUB_Owner
SLV_BUS_ComplianceScore.dtsx	Business	BUS_ComplianceScore	SAT_Taxpayer + SAT_MonthlyDecl + SAT_Payment
SLV_BUS_MonthlyMetrics.dtsx	Business	BUS_MonthlyMetrics	SAT_MonthlyDecl + SAT_Payment

5.2 Silver Table Design

PIT Table Template: PIT_Taxpayer

```
CREATE TABLE DV_Silver.dbo.PIT_Taxpayer (
    PIT_Taxpayer_HK     VARBINARY(32)  NOT NULL,  -- Same as HUB_Taxpayer_HK
    PIT_SnapshotDate    DATETIME       NOT NULL,   -- Point-in-time snapshot
    SAT_Taxpayer_LoadDate DATETIME     NULL,        -- Latest SAT_Taxpayer as-of snapshot
    PIT_LoadDate         DATETIME      NOT NULL DEFAULT GETDATE(),
    PIT_BatchID          INT           NOT NULL,
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
    CONSTRAINT PK_PIT_Taxpayer PRIMARY KEY (PIT_Taxpayer_HK, PIT_SnapshotDate)
);
```

Bridge Table Template: BRG_Taxpayer_Owner

```
CREATE TABLE DV_Silver.dbo.BRG_Taxpayer_Owner (
    HUB_Taxpayer_HK     VARBINARY(32)  NOT NULL,
    HUB_Owner_HK        VARBINARY(32)  NOT NULL,
    BRG_SnapshotDate    DATETIME       NOT NULL,
    BRG_LoadDate        DATETIME       NOT NULL DEFAULT GETDATE(),
    BRG_BatchID         INT            NOT NULL,

    -- Denormalized for query convenience
    TaxID               VARCHAR(20)    NULL,
    OwnerName           NVARCHAR(200)  NULL,

    CONSTRAINT PK_BRG_Taxpayer_Owner PRIMARY KEY
        (HUB_Taxpayer_HK, HUB_Owner_HK, BRG_SnapshotDate)
);
```

Business Vault Table Template: BUS_ComplianceScore

```
CREATE TABLE DV_Silver.dbo.BUS_ComplianceScore (
    HUB_Taxpayer_HK     VARBINARY(32)  NOT NULL,
    SnapshotDate        DATETIME       NOT NULL,
    ComplianceScore     DECIMAL(5,2)   NULL,  -- Computed metric
    FilingOnTimeRate    DECIMAL(5,2)   NULL,
    PaymentOnTimeRate   DECIMAL(5,2)   NULL,
    PenaltyCount        INT            NULL,
    BUS_LoadDate        DATETIME       NOT NULL DEFAULT GETDATE(),
    BUS_BatchID         INT            NOT NULL,

    CONSTRAINT PK_BUS_ComplianceScore PRIMARY KEY
        (HUB_Taxpayer_HK, SnapshotDate)
);
```

5.3 SLV_Load_All.dtsx, Orchestrator

Package Parameters

Parameter	Data Type	Passed From	Purpose
$Package::p_BatchID	Int32	User::v_BatchID (parent)	Current batch identifier
$Package::p_SnapshotDate	DateTime	User::v_SnapshotDate (parent)	PIT/Bridge snapshot timestamp

CONFIDENTIAL

Control Flow, 6 Execute Package Tasks (Sequential)

Step	Task	Parameters Passed to Child
1	EPT_SLV_PIT_Taxpayer	p_BatchID, p_SnapshotDate
2	EPT_SLV_PIT_Declaration	p_BatchID, p_SnapshotDate
3	EPT_SLV_PIT_Payment	p_BatchID, p_SnapshotDate
4	EPT_SLV_BRG_Taxpayer_Owner	p_BatchID, p_SnapshotDate
5	EPT_SLV_BUS_ComplianceScore	p_BatchID, p_SnapshotDate
6	EPT_SLV_BUS_MonthlyMetrics	p_BatchID, p_SnapshotDate

Why Silver Needs SnapshotDate

PIT tables capture the state of satellites as of a specific point in time (the SnapshotDate).
Bridge tables also stamp their records with the snapshot date for temporal consistency.
Business Vault tables use SnapshotDate as the period reference for computed metrics.
SnapshotDate comes from Master_Complete_Pipeline.dtsx → SCR_ValidateAndInitialize.

CONFIDENTIAL

5.4 PIT Child Packages

All 3 PIT child packages follow the same pattern. PIT tables capture the state of Satellite data as-of a specific SnapshotDate. This enables efficient temporal queries without scanning full Satellite history. Each PIT package uses a 4-step control flow: SQL_StartStep → SQL_DeleteSnapshot → DFT_Load_PIT → SQL_EndStep. The DFT is a simple 3-component straight-through pipeline (no Lookup needed — the DELETE step ensures idempotency).
This section provides the complete step-by-step implementation guide. SLV_PIT_Taxpayer.dtsx is the reference template; the remaining two packages document only the differences.

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

5.4.1 SLV_PIT_Taxpayer.dtsx, Reference Template

This is the master template for all PIT child packages. Build this package first, then clone for PIT_Declaration and PIT_Payment.

Package Parameters

Parameter	Data Type	Passed From	Purpose
`p_BatchID`	Int32	User::v_BatchID (parent)	Current ETL batch identifier
`p_SnapshotDate`	DateTime	User::v_SnapshotDate (parent)	Point-in-time snapshot date

Package Variables

Variable Name	Data Type	Default Value	EvaluateAsExpression?
`v_TableName`	String	`PIT_Taxpayer`	No
`v_StepName`	String	(expression)	Yes
`v_DeleteSQL`	String	(expression)	Yes
`v_SourceSQL`	String	(expression)	Yes
`v_DestTable`	String	(expression)	Yes
`v_RowCount`	Int32	`0`	No
`v_ErrorMessage`	String	(empty)	No
`v_ErrorSource`	String	(expression)	Yes

Expression Definitions (F4 → Properties → EvaluateAsExpression = True)

```
// v_StepName expression:
"Load_" + @[User::v_TableName]                                  → Load_PIT_Taxpayer

// v_DestTable expression:
@[User::v_TableName]                                            → PIT_Taxpayer

// v_ErrorSource expression:
@[System::PackageName]                                          → SLV_PIT_Taxpayer
```

v_DeleteSQL Expression:

```
"SET NOCOUNT ON; DECLARE @SD DATETIME = GETDATE();"
+ " DELETE FROM DV_Silver.dbo." + @[User::v_TableName]
+ " WHERE PIT_SnapshotDate = @SD;"
```

v_SourceSQL Expression:

```
"DECLARE @SD DATETIME = '" + (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate] + "';"
+ " DECLARE @BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + ";"
+ " SELECT hub.HUB_Taxpayer_HK AS PIT_Taxpayer_HK,"
+ "   @SD AS PIT_SnapshotDate,"
+ "   sat.SAT_LoadDate AS SAT_Taxpayer_LoadDate,"
+ "   GETDATE() AS PIT_LoadDate,"
+ "   @BatchID AS PIT_BatchID"
+ " FROM DV_Bronze.dbo.HUB_Taxpayer hub"
+ " CROSS APPLY ("
+ "   SELECT TOP 1 s.SAT_LoadDate FROM DV_Bronze.dbo.SAT_Taxpayer s"
+ "   WHERE s.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK AND s.SAT_LoadDate <= @SD"
+ "   ORDER BY s.SAT_LoadDate DESC ) sat"
```

Connection Managers Used

Name	Type	Database	Used By

CM_DV_Bronze	OLE DB	DV_Bronze	OLE_SRC (Data Flow source)
CM_DV_Silver	OLE DB	DV_Silver	SQL_DeleteSnapshot + OLE_DST
CM_ETL_Control	OLE DB	ETL_Control	SQL_StartStep, SQL_EndStep
CM_ETL_Control_OnError	OLE DB	ETL_Control	OnError handler tasks

Control Flow (4 Steps)

Step	Task	Description
1	SQL_StartStep	Calls usp_StartStep. See Section 4.6 for full configuration.
2	SQL_DeleteSnapshot	DELETE FROM PIT_Taxpayer WHERE PIT_SnapshotDate = @SD.
3	DFT_Load_PIT	Hub + SAT via CROSS APPLY → ROW_CNT → OLE_DST.
4	SQL_EndStep	Calls usp_EndStep('Success', @RowCount).

Control Flow Diagram

```
SQL_StartStep
    │  (Success)
    ▼
SQL_DeleteSnapshot     ← DELETE FROM PIT_Taxpayer WHERE PIT_SnapshotDate = @SD
    │  (Success)
    ▼
DFT_Load_PIT           ← OLE_SRC (CROSS APPLY) → ROW_CNT → OLE_DST
    │  (Success)
    ▼
SQL_EndStep
```

CONFIDENTIAL

Step 1: SQL_StartStep (Execute SQL Task)

Identical to Bronze packages (Section 4.6 Step 1).

Property	Value
Connection	`CM_ETL_Control`
SQLSourceType	Direct input
ResultSet	None

SQL Statement:

```
EXEC ETL_Control.dbo.usp_StartStep @BatchID = ?, @StepName = ?, @PackageName =
'SLV_PIT_Taxpayer'
```

Parameter Mapping Tab:

Variable Name	Direction	Data Type	Param Name	Size
`$Package::p_BatchID`	Input	LONG	`0`	-1
`User::v_StepName`	Input	VARCHAR	`1`	200

CONFIDENTIAL

Step 2: SQL_DeleteSnapshot (Execute SQL Task)

Property	Value
Connection	`CM_DV_Silver`
SQLSourceType	Direct input (expression override)
ResultSet	None

Setting SQL via Expression (Dynamic v_DeleteSQL):
1. Click SQL_DeleteSnapshot in the Control Flow.
2. Press F4 to open Properties window.
3. Click Expressions row → click '...' button.
4. In the Property Expressions Editor: Property = SqlStatementSource, Expression = @[User::v_DeleteSQL].
5. Click OK.

At runtime with v_TableName = 'PIT_Taxpayer', v_DeleteSQL evaluates to:

```
SET NOCOUNT ON;
DECLARE @SD DATETIME = GETDATE();
DELETE FROM DV_Silver.dbo.PIT_Taxpayer WHERE PIT_SnapshotDate = @SD;
```

No parameter mapping needed, the DECLARE @SD approach embeds the date directly.

✓ **TIP:** The DELETE + INSERT pattern ensures idempotent re-runs. If the package executes twice on the same day, the second run deletes the first run's data before re-inserting.

⚠ **IMPORTANT:** Expression-Driven DELETE Enables Cloning: When v_TableName changes (e.g., to PIT_Declaration), the v_DeleteSQL expression auto-updates the table name in the DELETE statement.

CONFIDENTIAL

Step 3: DFT_Load_PIT (Data Flow Task)

The PIT DFT is a straight-through pipeline: complex temporal source SQL feeds directly into the destination. No Lookup or Conditional Split needed, the DELETE step already cleared existing rows for this SnapshotDate.

Data Flow Diagram

```
OLE_SRC_PIT_Taxpayer
    │  (Hub CROSS APPLY Satellite, complex temporal SQL)
    ▼
ROW_CNT ─→ OLE_DST_PIT_Taxpayer
```

Creating the Data Flow Task

In the Control Flow tab, drag a Data Flow Task from the SSIS Toolbox onto the design surface between SQL_DeleteSnapshot and SQL_EndStep.

Step-by-Step: Create DFT_Load_PIT

1. Drag Data Flow Task from SSIS Toolbox → Control Flow design surface.
2. Right-click the new Data Flow Task → Rename → type DFT_Load_PIT.
3. Drag green arrow from SQL_DeleteSnapshot to DFT_Load_PIT.
4. Drag green arrow from DFT_Load_PIT to SQL_EndStep.

Adding Components to the Data Flow

Double-click DFT_Load_PIT to enter the Data Flow designer. Add components from the SSIS Toolbox (left panel) in this order:

#	Toolbox Section	Component	Rename To
1	Other Sources	OLE DB Source	`OLE_SRC_PIT_Taxpayer`
2	Common	Row Count	`ROW_CNT`
3	Other Destinations	OLE DB Destination	`OLE_DST_PIT_Taxpayer`

Connecting the Data Flow Paths (Green/Blue Arrows)

#	From	To	Output Type
1	`OLE_SRC_PIT_Taxpayer`	`ROW_CNT`	Default Output (green arrow)
2	`ROW_CNT`	`OLE_DST_PIT_Taxpayer`	Default Output (green arrow)

Drag the green arrow from OLE_SRC to ROW_CNT, then from ROW_CNT to OLE_DST.

CONFIDENTIAL

Component 1: OLE_SRC_PIT_Taxpayer (OLE DB Source), Full Configuration

Double-click OLE_SRC_PIT_Taxpayer to open the OLE DB Source Editor. Configure each tab as follows:

Connection Manager Tab

Property	Value	Notes
OLE DB connection manager	`CM_DV_Bronze`	Select from dropdown, Bronze database
Data access mode	SQL command from variable	Select from dropdown
Variable name	`User::v_SourceSQL`	Select from dropdown

Runtime SQL (v_TableName = 'PIT_Taxpayer')

At runtime, the v_SourceSQL expression evaluates to:

```
DECLARE @SD DATETIME = '2026-02-22';
DECLARE @BatchID INT = 1;
SELECT
    hub.HUB_Taxpayer_HK      AS PIT_Taxpayer_HK,
    @SD                       AS PIT_SnapshotDate,
    sat.SAT_LoadDate          AS SAT_Taxpayer_LoadDate,
    GETDATE()                 AS PIT_LoadDate,
    @BatchID                  AS PIT_BatchID
FROM DV_Bronze.dbo.HUB_Taxpayer hub
CROSS APPLY (
    SELECT TOP 1 s.SAT_LoadDate
    FROM DV_Bronze.dbo.SAT_Taxpayer s
    WHERE s.HUB_Taxpayer_HK = hub.HUB_Taxpayer_HK
      AND s.SAT_LoadDate <= @SD
    ORDER BY s.SAT_LoadDate DESC
) sat
```

⚠ **IMPORTANT:** CROSS APPLY returns the most recent SAT_LoadDate on or before the snapshot date. If a Hub record has no Satellite entries before the snapshot, CROSS APPLY excludes it (unlike OUTER APPLY which would return NULL).

Columns Tab, Metadata Setup

When using 'SQL command from variable', the Columns tab shows a warning: 'Validation would fail because the variable value is not available at design time.' You must manually set up the metadata:

Metadata Setup Procedure (one-time per entity):

1. Temporarily switch Data access mode to 'SQL command'.
2. Paste the full evaluated runtime SQL (above) into the SQL command text box.
3. Click Columns tab, SSIS reads the column metadata from the SQL.
4. Click OK to accept.
5. Re-open OLE_SRC → switch back to 'SQL command from variable' → select User::v_SourceSQL.
6. Click OK. The metadata is now cached and persists across sessions.

Output Columns After Setup:

Column Name	Data Type	Length	Purpose
`PIT_Taxpayer_HK`	DT_BYTES	32	Hub hash key, pointer to HUB_Taxpayer
`PIT_SnapshotDate`	DT_DBTIMESTAMP		Snapshot timestamp from @SD parameter

© 2026 Mr. Sot So — Royal University of Phnom Penh

SAT_Taxpayer_LoadDate	DT_DBTIMESTAMP		Latest SAT_LoadDate as-of snapshot
PIT_LoadDate	DT_DBTIMESTAMP		Processing timestamp from GETDATE()
PIT_BatchID	DT_I4		Batch identifier from @BatchID parameter

Error Output Tab

Leave default. If the source SQL fails (bad connection, syntax error), the DFT should fail and trigger the OnError handler.

✓ **TIP:** No parameter mapping needed on OLE_SRC. The DECLARE approach embeds p_SnapshotDate and p_BatchID directly into the SQL string via the v_SourceSQL expression.

CONFIDENTIAL

Component 2: ROW_CNT (Row Count Transform), Full Configuration

The Row Count transform counts the rows passing through and stores the count in a variable. It does not modify the data.

Configuration (1 property):

Select ROW_CNT in the Data Flow designer → press F4 (Properties window) → set VariableName to User::v_RowCount.

Property	Value
VariableName	`User::v_RowCount`

Alternatively: double-click ROW_CNT. In some SSIS versions this opens a Custom Properties dialog where you can type the variable name directly.

Component 3: OLE_DST_PIT_Taxpayer (OLE DB Destination), Full Configuration

Double-click OLE_DST_PIT_Taxpayer to open the OLE DB Destination Editor. Configure each tab:

Connection Manager Tab

Property	Value	Notes
OLE DB connection manager	`CM_DV_Silver`	Select from dropdown, Silver database
Data access mode	Table or view — fast load	Bulk insert mode
Table name	`dbo.PIT_Taxpayer`	Select from dropdown (metadata setup)

Setting Destination Table via Expression:

Data Flow components do NOT have their own Expressions property. Instead, set on the parent DFT from the Control Flow:

1. Switch to Control Flow tab.
2. Select DFT_Load_PIT → press F4.
3. Expressions → '...' → Property = [OLE_DST_PIT_Taxpayer].[OpenRowset] → Expression = @[User::v_DestTable].
4. Click OK.

At runtime with v_TableName = 'PIT_Taxpayer', v_DestTable evaluates to 'PIT_Taxpayer'.

Fast Load Options

Option	Setting	Notes
Table lock	Checked	Faster bulk insert, acquires exclusive lock
Check constraints	Checked	Validates PK constraints on insert
Rows per batch	(empty)	SSIS default
Maximum insert commit size	`0`	All rows in one commit

Mappings Tab

The Mappings tab shows Available Input Columns (from the pipeline) on the left and Available Destination Columns (from PIT_Taxpayer) on the right.

Input Column	Destination Column	Auto-Matched?
`PIT_Taxpayer_HK`	`PIT_Taxpayer_HK`	Yes
`PIT_SnapshotDate`	`PIT_SnapshotDate`	Yes
`SAT_Taxpayer_LoadDate`	`SAT_Taxpayer_LoadDate`	Yes
`PIT_LoadDate`	`PIT_LoadDate`	Yes
`PIT_BatchID`	`PIT_BatchID`	Yes

✓ **TIP:** All 5 columns auto-match by name. No IDENTITY or DEFAULT columns to ignore — the PIT table PK is a composite of PIT_Taxpayer_HK + PIT_SnapshotDate (no surrogate key).

© 2026 Mr. Sot So — Royal University of Phnom Penh

Error Output Tab
Leave default. Primary key violations (duplicate HK + SnapshotDate) cause the DFT to fail, which triggers OnError.

Debugging: Data Viewer Placements
Data Viewers let you inspect rows flowing through the pipeline during debug execution. Right-click a data flow path (green arrow) → Enable Data Viewer.

Location	What to Verify
OLE_SRC → ROW_CNT	All Hub records have SAT_LoadDate values (CROSS APPLY excludes orphans)
ROW_CNT → OLE_DST	Same data, verify row count matches expected Hub count

CONFIDENTIAL

Step 4: SQL_EndStep (Execute SQL Task)

Identical to Bronze packages (Section 4.6 Step 3).

Property	Value
Connection	`CM_ETL_Control`
SQLSourceType	Direct input

SQL Statement:

```
EXEC ETL_Control.dbo.usp_EndStep @BatchID = ?, @StepName = ?, @RowCount = ?, @Status =
'Success'
```

Parameter Mapping Tab:

Variable Name	Direction	Data Type	Param Name	Size
`$Package::p_BatchID`	Input	LONG	`0`	-1
`User::v_StepName`	Input	VARCHAR	`1`	200
`User::v_RowCount`	Input	LONG	`2`	-1

CONFIDENTIAL

OnError Event Handler (SLV_PIT_Taxpayer.dtsx)

Every Silver child package has an OnError handler at the package level. The pattern is identical to Bronze child packages (Section 4.13). All OnError tasks MUST use CM_ETL_Control_OnError connection and set TransactionOption = NotSupported.

OnError Event Handler Tasks (3 steps)

Step	Task	Connection	TransactionOption
1	`SCR_CaptureError`	— (Script Task)	NotSupported
2	`SQL_EndStepOnError`	`CM_ETL_Control_OnError`	NotSupported
3	`SQL_LogErrorOnError`	`CM_ETL_Control_OnError`	NotSupported

Step 1: SCR_CaptureError (Script Task)

Step-by-step Configuration:

1. In the Control Flow, click the package background → Event Handlers tab.
2. In the Event handler dropdown, select OnError.
3. Click the link 'Click here to create an OnError event handler for executable SLV_PIT_Taxpayer'.
4. Drag a Script Task onto the OnError canvas. Name it SCR_CaptureError.
5. Double-click SCR_CaptureError to open the Script Task Editor.
6. Set ReadOnlyVariables: System::ErrorDescription
7. Set ReadWriteVariables: User::v_ErrorMessage
8. Click Edit Script... and enter the code below.
9. Click OK.

Script Task Code (C#):

```
public void Main()
{
    string errorDesc = Dts.Variables["System::ErrorDescription"].Value.ToString();
    Dts.Variables["User::v_ErrorMessage"].Value = errorDesc;
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

Step 2: SQL_EndStepOnError (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control_OnError`
TransactionOption	NotSupported
SQLSourceType	Direct input

```
EXEC ETL_Control.dbo.usp_EndStep @BatchID = ?, @StepName = ?, @RowCount = 0, @Status = 'Failed'
```

Parameter Mapping:

Variable Name	Direction	Data Type	Param Name
`$Package::p_BatchID`	Input	LONG	`0`
`User::v_StepName`	Input	VARCHAR	`1`

Step 3: SQL_LogErrorOnError (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control_OnError`
TransactionOption	NotSupported
SQLSourceType	Direct input

```
EXEC ETL_Control.dbo.usp_LogError @BatchID = ?, @StepName = ?, @ErrorMessage = ?, @PackageName
= 'SLV_PIT_Taxpayer'
```

Parameter Mapping:

Variable Name	Direction	Data Type	Param Name	Size
`$Package::p_BatchID`	Input	LONG	`0`	-1
`User::v_StepName`	Input	VARCHAR	`1`	200
`User::v_ErrorMessage`	Input	VARCHAR	`2`	4000

CONFIDENTIAL

Complete PIT DFT Property Summary (SLV_PIT_Taxpayer)

Component	Key Property	Value
OLE_SRC	Connection	CM_DV_Bronze
OLE_SRC	Data access mode	SQL command from variable (User::v_SourceSQL)
OLE_SRC	Parameters	No mapping, DECLARE approach
OLE_SRC	Key SQL pattern	Hub CROSS APPLY (SELECT TOP 1 SAT_LoadDate <= @SD)
ROW_CNT	VariableName	User::v_RowCount
OLE_DST	Connection	CM_DV_Silver
OLE_DST	Table (expression)	@[User::v_DestTable] → PIT_Taxpayer
OLE_DST	Fast load	Table lock checked, all rows in one commit

PIT Tables and Their Satellite Sources

PIT Table	Hub Source	Satellite (CROSS APPLY)	PIT_HK Column	SAT_LoadDate Column
PIT_Taxpayer	HUB_Taxpayer	SAT_Taxpayer	PIT_Taxpayer_HK	SAT_Taxpayer_LoadDate
PIT_Declaration	HUB_Declaration	SAT_MonthlyDecl	PIT_Declaration_HK	SAT_MonthlyDecl_LoadDate
PIT_Payment	HUB_Payment	SAT_Payment	PIT_Payment_HK	SAT_Payment_LoadDate

CONFIDENTIAL

5.4.2 SLV_PIT_Declaration.dtsx, Clone from PIT_Taxpayer

Identical pattern to SLV_PIT_Taxpayer.dtsx. Build PIT_Taxpayer first, then copy package in Solution Explorer → Paste → rename. Change only the following:

Variable Change:

Variable	New Value	Effect on Expressions
v_TableName	PIT_Declaration	v_StepName → Load_PIT_Declaration, v_DestTable → PIT_Declaration, v_DeleteSQL auto-updates

OLE_SRC: Update v_SourceSQL Expression
Change the v_SourceSQL expression to reference HUB_Declaration and SAT_MonthlyDecl. At runtime it evaluates to:

```
SELECT
    hub.HUB_Declaration_HK    AS PIT_Declaration_HK,
    @SD                        AS PIT_SnapshotDate,
    sat.SAT_LoadDate           AS SAT_MonthlyDecl_LoadDate,
    GETDATE()                  AS PIT_LoadDate,
    @BatchID                   AS PIT_BatchID
FROM DV_Bronze.dbo.HUB_Declaration hub
CROSS APPLY (
    SELECT TOP 1 s.SAT_LoadDate
    FROM DV_Bronze.dbo.SAT_MonthlyDecl s
    WHERE s.HUB_Declaration_HK = hub.HUB_Declaration_HK
      AND s.SAT_LoadDate <= @SD
    ORDER BY s.SAT_LoadDate DESC
) sat
```

OLE_SRC: Re-do Metadata Setup
After changing v_SourceSQL, the OLE_SRC metadata is stale. Follow the same Metadata Setup Procedure (Section 5.4.1): temporarily switch to 'SQL command' mode, paste the new runtime SQL, click Columns to refresh metadata, then switch back to 'SQL command from variable'.

OLE_DST: Re-map Columns
Change destination table to PIT_Declaration. Column mappings:

Input Column	Destination Column
PIT_Declaration_HK	PIT_Declaration_HK
PIT_SnapshotDate	PIT_SnapshotDate
SAT_MonthlyDecl_LoadDate	SAT_MonthlyDecl_LoadDate
PIT_LoadDate	PIT_LoadDate
PIT_BatchID	PIT_BatchID

OnError: Change @PackageName string literal to 'SLV_PIT_Declaration' in SQL_LogErrorOnError.

© 2026 Mr. Sot So — Royal University of Phnom Penh

5.4.3 SLV_PIT_Payment.dtsx, Clone from PIT_Taxpayer

Identical pattern. Change only the following:

Variable Change:

Variable	New Value
v_TableName	PIT_Payment

OLE_SRC: v_SourceSQL Runtime SQL:

```
SELECT
    hub.HUB_Payment_HK        AS PIT_Payment_HK,
    @SD                        AS PIT_SnapshotDate,
    sat.SAT_LoadDate           AS SAT_Payment_LoadDate,
    GETDATE()                  AS PIT_LoadDate,
    @BatchID                   AS PIT_BatchID
FROM DV_Bronze.dbo.HUB_Payment hub
CROSS APPLY (
    SELECT TOP 1 s.SAT_LoadDate
    FROM DV_Bronze.dbo.SAT_Payment s
    WHERE s.HUB_Payment_HK = hub.HUB_Payment_HK
      AND s.SAT_LoadDate <= @SD
    ORDER BY s.SAT_LoadDate DESC
) sat
```

OLE_SRC: Re-do Metadata Setup (same procedure as Section 5.4.2).

OLE_DST Column Mappings:

Input Column	Destination Column
PIT_Payment_HK	PIT_Payment_HK
PIT_SnapshotDate	PIT_SnapshotDate
SAT_Payment_LoadDate	SAT_Payment_LoadDate
PIT_LoadDate	PIT_LoadDate
PIT_BatchID	PIT_BatchID

OnError: Change @PackageName to 'SLV_PIT_Payment'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

5.5 Bridge Child Package (SLV_BRG_Taxpayer_Owner.dtsx)

Bridge tables pre-join multiple Hub and Link tables to flatten many-to-many relationships for efficient querying. The DFT uses the same 4-step pattern as PIT: SQL_StartStep → SQL_DeleteSnapshot → DFT_Load_Bridge → SQL_EndStep. There is only one Bridge package in this project.

Package Parameters

Parameter	Data Type	Passed From	Purpose
`p_BatchID`	Int32	User::v_BatchID (parent)	Current ETL batch identifier
`p_SnapshotDate`	DateTime	User::v_SnapshotDate (parent)	Bridge snapshot date

Package Variables

Variable Name	Data Type	Default / Expression	Purpose
`v_TableName`	String	`BRG_Taxpayer_Owner`	CHANGE PER PACKAGE
`v_StepName`	String	(Expr) "Load_" + v_TableName	Auto: Load_BRG_Taxpayer_Owner
`v_DeleteSQL`	String	(Expr) DELETE + BRG_SnapshotDate	Expression-driven DELETE
`v_SourceSQL`	String	(Expr) DECLARE + SELECT	Bridge join query
`v_DestTable`	String	(Expr) v_TableName	Auto: BRG_Taxpayer_Owner
`v_RowCount`	Int32	`0`	Rows inserted
`v_ErrorMessage`	String	(empty)	Captured by OnError
`v_ErrorSource`	String	(Expr) @[System::PackageName]	SLV_BRG_Taxpayer_Owner

v_DeleteSQL Expression:

```
"SET NOCOUNT ON; DELETE FROM DV_Silver.dbo." + @[User::v_TableName]
+ " WHERE BRG_SnapshotDate = @SD;"
```

⚠ **IMPORTANT:** Bridge uses BRG_SnapshotDate (not PIT_SnapshotDate). The column name in the DELETE must match the Bridge table DDL.

Connection Managers Used

Name	Type	Database	Used By
`CM_DV_Bronze`	OLE DB	DV_Bronze	OLE_SRC (Link + Hub + SAT joins)
`CM_DV_Silver`	OLE DB	DV_Silver	SQL_DeleteSnapshot + OLE_DST
`CM_ETL_Control`	OLE DB	ETL_Control	SQL_StartStep, SQL_EndStep
`CM_ETL_Control_OnError`	OLE DB	ETL_Control	OnError handler tasks

Control Flow (4 Steps)

Step	Task	Description
1	`SQL_StartStep`	Calls usp_StartStep with v_StepName = 'Load_BRG_Taxpayer_Owner'.
2	`SQL_DeleteSnapshot`	DELETE FROM BRG_Taxpayer_Owner WHERE BRG_SnapshotDate = @SD.
3	`DFT_Load_Bridge`	Reads Link+Hub+SAT via complex join, inserts Bridge rows.
4	`SQL_EndStep`	Calls usp_EndStep('Success', @RowCount).

Control Flow Diagram

```
SQL_StartStep
    │  (Success)
    ▼
SQL_DeleteSnapshot     ← DELETE FROM BRG_Taxpayer_Owner WHERE BRG_SnapshotDate = @SD
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
    │  (Success)
    ▼
DFT_Load_Bridge        ← OLE_SRC (LNK+HUB+SAT join) → ROW_CNT → OLE_DST
    │  (Success)
    ▼
SQL_EndStep
```

CONFIDENTIAL

Step 3: DFT_Load_Bridge (Data Flow Task)

3-component straight-through pipeline: OLE_SRC → ROW_CNT → OLE_DST. No Lookup needed, SQL_DeleteSnapshot already cleared existing rows.

Creating the DFT

Drag Data Flow Task between SQL_DeleteSnapshot and SQL_EndStep. Rename to DFT_Load_Bridge. Connect precedence constraints.

Adding Components to the Data Flow

#	Toolbox Section	Component	Rename To
1	Other Sources	OLE DB Source	`OLE_SRC_BRG_Taxpayer_Owner`
2	Common	Row Count	`ROW_CNT`
3	Other Destinations	OLE DB Destination	`OLE_DST_BRG_Taxpayer_Owner`

Connecting the Data Flow Paths

#	From	To	Output Type
1	`OLE_SRC_BRG_Taxpayer_Owner`	`ROW_CNT`	Default Output
2	`ROW_CNT`	`OLE_DST_BRG_Taxpayer_Owner`	Default Output

Component 1: OLE_SRC (OLE DB Source), Full Configuration

Property	Value
Connection	`CM_DV_Bronze`
Data access mode	SQL command from variable
Variable name	`User::v_SourceSQL`

v_SourceSQL Runtime SQL:

```
SELECT
    lnk.HUB_Taxpayer_HK,
    lnk.HUB_Owner_HK,
    @SD                        AS BRG_SnapshotDate,
    GETDATE()                  AS BRG_LoadDate,
    @BatchID                   AS BRG_BatchID,
    hub_tp.TaxID,
    sat_ow.OwnerName
FROM DV_Bronze.dbo.LNK_TaxpayerOwner lnk
INNER JOIN DV_Bronze.dbo.HUB_Taxpayer hub_tp
    ON lnk.HUB_Taxpayer_HK = hub_tp.HUB_Taxpayer_HK
INNER JOIN DV_Bronze.dbo.HUB_Owner hub_ow
    ON lnk.HUB_Owner_HK = hub_ow.HUB_Owner_HK
CROSS APPLY (
    SELECT TOP 1 s.OwnerName
    FROM DV_Bronze.dbo.SAT_Owner s
    WHERE s.HUB_Owner_HK = hub_ow.HUB_Owner_HK
      AND s.SAT_LoadDate <= @SD
      AND (s.SAT_EndDate IS NULL OR s.SAT_EndDate > @SD)
    ORDER BY s.SAT_LoadDate DESC
) sat_ow
```

Columns Tab, Metadata Setup

Same procedure as PIT (Section 5.4.1): temporarily switch to 'SQL command', paste runtime SQL, click Columns, switch back to variable mode.

© 2026 Mr. Sot So — Royal University of Phnom Penh

Output Columns:

Column Name	Data Type	Length	Source
HUB_Taxpayer_HK	DT_BYTES	32	LNK_TaxpayerOwner
HUB_Owner_HK	DT_BYTES	32	LNK_TaxpayerOwner
BRG_SnapshotDate	DT_DBTIMESTAMP		@SD parameter
BRG_LoadDate	DT_DBTIMESTAMP		GETDATE()
BRG_BatchID	DT_I4		@BatchID parameter
TaxID	DT_STR	20	HUB_Taxpayer (denormalized)
OwnerName	DT_WSTR	200	SAT_Owner via CROSS APPLY

Component 2: ROW_CNT, VariableName = User::v_RowCount

Component 3: OLE_DST (OLE DB Destination), Full Configuration

Property	Value
Connection	CM_DV_Silver
Data access mode	Table or view — fast load
Table name	dbo.BRG_Taxpayer_Owner
Table lock	Checked

Mappings Tab, All 7 columns auto-match by name.

Input Column	Destination Column
HUB_Taxpayer_HK	HUB_Taxpayer_HK
HUB_Owner_HK	HUB_Owner_HK
BRG_SnapshotDate	BRG_SnapshotDate
BRG_LoadDate	BRG_LoadDate
BRG_BatchID	BRG_BatchID
TaxID	TaxID
OwnerName	OwnerName

Debugging: Data Viewer Placements

Location	What to Verify
OLE_SRC → ROW_CNT	All Link records resolve to Hub + SAT (no NULLs in denormalized columns)
ROW_CNT → OLE_DST	Row count matches expected Link count

OnError: Same 3-step pattern as PIT. Change @PackageName to 'SLV_BRG_Taxpayer_Owner'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

5.6 Business Vault Child Packages

Business Vault tables contain computed/derived metrics that do not exist in the source system. They apply business rules to Raw Vault data. Same 4-step pattern: SQL_StartStep → SQL_DeleteSnapshot → DFT_Load_BUS → SQL_EndStep. Two packages: BUS_ComplianceScore and BUS_MonthlyMetrics.

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

5.6.1 SLV_BUS_ComplianceScore.dtsx, Reference Template

Package Variables

Variable Name	Data Type	Default / Expression	Purpose
`v_TableName`	String	`BUS_ComplianceScore`	CHANGE PER PACKAGE
`v_StepName`	String	(Expr) "Load_" + v_TableName	Auto: Load_BUS_ComplianceScore
`v_DeleteSQL`	String	(Expr) DELETE + SnapshotDate	Expression-driven DELETE
`v_SourceSQL`	String	(Expr) DECLARE + complex SQL	3 OUTER APPLY subqueries
`v_DestTable`	String	(Expr) v_TableName	Auto: BUS_ComplianceScore
`v_RowCount`	Int32	`0`	Rows inserted
`v_ErrorMessage`	String	(empty)	Captured by OnError
`v_ErrorSource`	String	(Expr) @[System::PackageName]	SLV_BUS_ComplianceScore

v_DeleteSQL Expression:

```
"SET NOCOUNT ON; DELETE FROM DV_Silver.dbo." + @[User::v_TableName]
+ " WHERE SnapshotDate = @SD;"
```

⚠ **IMPORTANT:** BUS_ComplianceScore uses SnapshotDate (not PIT_SnapshotDate or BRG_SnapshotDate). Each Silver table type has its own snapshot column name matching its DDL.

Control Flow (4 Steps)

Step	Task	Description
1	`SQL_StartStep`	Calls usp_StartStep.
2	`SQL_DeleteSnapshot`	DELETE FROM BUS_ComplianceScore WHERE SnapshotDate = @SD.
3	`DFT_Load_BUS`	Reads Hub + OUTER APPLY metrics, inserts computed scores.
4	`SQL_EndStep`	Calls usp_EndStep('Success', @RowCount).

DFT_Load_BUS: 3-component pipeline

Creating: Drag Data Flow Task between SQL_DeleteSnapshot and SQL_EndStep. Rename to DFT_Load_BUS.

Adding Components:

#	Toolbox Section	Component	Rename To
1	Other Sources	OLE DB Source	`OLE_SRC_BUS_ComplianceScore`
2	Common	Row Count	`ROW_CNT`
3	Other Destinations	OLE DB Destination	`OLE_DST_BUS_ComplianceScore`

Connect: OLE_SRC → ROW_CNT → OLE_DST (green arrows).

Component 1: OLE_SRC SQL Command (the most complex query in the Silver layer):

```
SELECT
    hub.HUB_Taxpayer_HK,
    @SD  AS SnapshotDate,
    CAST(
        (ISNULL(filing.OnTimeRate, 0) * 0.5 +
         ISNULL(payment.OnTimeRate, 0) * 0.5) AS DECIMAL(5,2)
    ) AS ComplianceScore,
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
    filing.OnTimeRate    AS FilingOnTimeRate,
    payment.OnTimeRate   AS PaymentOnTimeRate,
    ISNULL(penalties.PenaltyCount, 0) AS PenaltyCount,
    GETDATE()            AS BUS_LoadDate,
    @BatchID             AS BUS_BatchID
FROM DV_Bronze.dbo.HUB_Taxpayer hub
-- Filing on-time rate
OUTER APPLY (
    SELECT CAST(
        SUM(CASE WHEN sat.DeclarationDate <= sat.DueDate THEN 1.0 ELSE 0.0 END) /
        NULLIF(COUNT(*), 0) * 100 AS DECIMAL(5,2)
    ) AS OnTimeRate
    FROM DV_Bronze.dbo.SAT_MonthlyDecl sat
    INNER JOIN DV_Bronze.dbo.HUB_Declaration hd ON sat.HUB_Declaration_HK =
hd.HUB_Declaration_HK
    WHERE sat.SAT_EndDate IS NULL
) filing
-- Payment on-time rate
OUTER APPLY (
    SELECT CAST(
        SUM(CASE WHEN sat.Status = 'Paid' THEN 1.0 ELSE 0.0 END) /
        NULLIF(COUNT(*), 0) * 100 AS DECIMAL(5,2)
    ) AS OnTimeRate
    FROM DV_Bronze.dbo.SAT_Payment sat
    INNER JOIN DV_Bronze.dbo.HUB_Payment hp ON sat.HUB_Payment_HK = hp.HUB_Payment_HK
    WHERE sat.SAT_EndDate IS NULL
) payment
-- Penalty count
OUTER APPLY (
    SELECT COUNT(*) AS PenaltyCount
    FROM DV_Bronze.dbo.SAT_MonthlyDecl sat
    WHERE sat.SAT_EndDate IS NULL AND sat.PenaltyAmount > 0
) penalties
```

Output Columns:

Column Name	Data Type	Source
HUB_Taxpayer_HK	DT_BYTES (32)	HUB_Taxpayer
SnapshotDate	DT_DBTIMESTAMP	@SD parameter
ComplianceScore	DT_NUMERIC (5,2)	Computed: 50% filing + 50% payment
FilingOnTimeRate	DT_NUMERIC (5,2)	OUTER APPLY filing
PaymentOnTimeRate	DT_NUMERIC (5,2)	OUTER APPLY payment
PenaltyCount	DT_I4	OUTER APPLY penalties
BUS_LoadDate	DT_DBTIMESTAMP	GETDATE()
BUS_BatchID	DT_I4	@BatchID parameter

Component 3: OLE_DST — CM_DV_Silver, dbo.BUS_ComplianceScore, fast load, table lock. All 8 columns auto-match.

Debugging: Data Viewer Placements

Location	What to Verify
OLE_SRC → ROW_CNT	ComplianceScore values 0–100 (not NULL). OUTER APPLY returns NULL for taxpayers with no declarations.
ROW_CNT → OLE_DST	Row count matches HUB_Taxpayer count (one row per taxpayer)

© 2026 Mr. Sot So — Royal University of Phnom Penh

OnError: Change @PackageName to 'SLV_BUS_ComplianceScore'.

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

5.6.2 SLV_BUS_MonthlyMetrics.dtsx, Clone from ComplianceScore

Aggregates monthly declaration and payment data per taxpayer. Same 4-step control flow. Clone from ComplianceScore.

Variable Change:

Variable	New Value
v_TableName	BUS_MonthlyMetrics

OLE_SRC: Update v_SourceSQL. Runtime SQL:

```
SELECT
    lnk.HUB_Taxpayer_HK,
    sd.DeclarationMonth,
    sd.DeclarationYear,
    @SD AS SnapshotDate,
    COUNT(DISTINCT sd.HUB_Declaration_HK)          AS TotalDeclarations,
    SUM(sd.GrossRevenue)                            AS TotalGrossRevenue,
    SUM(sd.TaxAmount)                               AS TotalTaxAmount,
    pay_agg.TotalPayments,
    pay_agg.TotalPaymentAmount,
    CASE WHEN SUM(sd.TotalAmount) = 0 THEN 0
         ELSE ISNULL(pay_agg.TotalPaymentAmount, 0) / SUM(sd.TotalAmount) * 100
    END                                             AS PaymentCoverageRate,
    GETDATE()                                       AS BUS_LoadDate,
    @BatchID                                        AS BUS_BatchID
FROM DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk
INNER JOIN DV_Bronze.dbo.SAT_MonthlyDecl sd
    ON lnk.HUB_Declaration_HK = sd.HUB_Declaration_HK AND sd.SAT_EndDate IS NULL
OUTER APPLY (
    SELECT COUNT(DISTINCT sp.HUB_Payment_HK) AS TotalPayments,
           SUM(sp.PaymentAmount) AS TotalPaymentAmount
    FROM DV_Bronze.dbo.LNK_DeclarationPayment lnk_dp
    INNER JOIN DV_Bronze.dbo.SAT_Payment sp ON lnk_dp.HUB_Payment_HK = sp.HUB_Payment_HK AND
sp.SAT_EndDate IS NULL
    INNER JOIN DV_Bronze.dbo.SAT_MonthlyDecl sd2 ON lnk_dp.HUB_Declaration_HK =
sd2.HUB_Declaration_HK
        AND sd2.SAT_EndDate IS NULL AND sd2.DeclarationMonth = sd.DeclarationMonth AND
sd2.DeclarationYear = sd.DeclarationYear
    INNER JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk_td2 ON lnk_dp.HUB_Declaration_HK =
lnk_td2.HUB_Declaration_HK
        AND lnk_td2.HUB_Taxpayer_HK = lnk.HUB_Taxpayer_HK
) pay_agg
GROUP BY lnk.HUB_Taxpayer_HK, sd.DeclarationMonth, sd.DeclarationYear,
         pay_agg.TotalPayments, pay_agg.TotalPaymentAmount
```

OLE_SRC: Re-do Metadata Setup (same procedure as Section 5.4.1).

OLE_DST Column Mappings (12 columns):

Input Column	Destination Column
HUB_Taxpayer_HK	HUB_Taxpayer_HK
DeclarationMonth	MetricMonth
DeclarationYear	MetricYear
SnapshotDate	SnapshotDate
TotalDeclarations	TotalDeclarations

© 2026 Mr. Sot So — Royal University of Phnom Penh

TotalGrossRevenue	TotalGrossRevenue
TotalTaxAmount	TotalTaxAmount
TotalPayments	TotalPayments
TotalPaymentAmount	TotalPaymentAmount
PaymentCoverageRate	PaymentCoverageRate
BUS_LoadDate	BUS_LoadDate
BUS_BatchID	BUS_BatchID

⚠ **IMPORTANT:** DeclarationMonth maps to MetricMonth and DeclarationYear maps to MetricYear, these are NOT auto-matched because the column names differ. You must manually drag these mappings in the Mappings tab.

OnError: Change @PackageName to 'SLV_BUS_MonthlyMetrics'.

CONFIDENTIAL

5.7 Silver Package Cloning, Quick Reference

All 6 Silver packages share the same 4-step control flow. Build SLV_PIT_Taxpayer.dtsx first as the template. Detailed component configurations are in Sections 5.4–5.6 above.

PIT Package Cloning (3 packages)

Package	v_TableName	Hub Source	SAT Source	Key Changes
SLV_PIT_Taxpayer	`PIT_Taxpayer`	HUB_Taxpayer	SAT_Taxpayer	TEMPLATE, build first
SLV_PIT_Declaration	`PIT_Declaration`	HUB_Declaration	SAT_MonthlyDecl	v_TableName + v_SourceSQL + metadata
SLV_PIT_Payment	`PIT_Payment`	HUB_Payment	SAT_Payment	v_TableName + v_SourceSQL + metadata

Bridge Package (1 package)

Package	v_TableName	Source Pattern	Key Difference
SLV_BRG_Taxpayer_Owner	`BRG_Taxpayer_Owner`	LNK + HUB + SAT join	Uses BRG_SnapshotDate in v_DeleteSQL

Business Vault Packages (2 packages)

Package	v_TableName	Source Pattern	Key Difference
SLV_BUS_ComplianceScore	`BUS_ComplianceScore`	HUB + 3 OUTER APPLY	Uses SnapshotDate; weighted compliance score
SLV_BUS_MonthlyMetrics	`BUS_MonthlyMetrics`	LNK + SAT + OUTER APPLY	Uses SnapshotDate; GROUP BY aggregation

Common Cloning Steps for All Silver Packages:
1. Copy package in Solution Explorer → Paste → rename to new package name.
2. Variables → change v_TableName to the new entity name. All other expression variables auto-update.
3. OLE_SRC → update v_SourceSQL expression with new Hub/SAT/LNK tables and column aliases.
4. OLE_SRC → re-do Metadata Setup: temporarily switch to 'SQL command', paste runtime SQL, click Columns, switch back to variable mode.
5. OLE_DST → change destination table → re-verify column mappings (especially if column names differ).
6. OnError → SQL_LogErrorOnError → change @PackageName string literal.
7. Verify v_DeleteSQL snapshot column name matches target table DDL (PIT_SnapshotDate, BRG_SnapshotDate, or SnapshotDate).

5.8 Stored Procedures (Alternative Approach)

The DFT-based approach (Sections 5.4–5.7) is the primary implementation. As an alternative, the same logic can be implemented as stored procedures called from Execute SQL Tasks. This is useful for complex Business Vault computations that benefit from a single transaction scope.

© 2026 Mr. Sot So — Royal University of Phnom Penh

When to Consider Stored Procedures Instead of DFT

Business Vault with complex multi-CTE logic: SP keeps all business rules in one place.
Large PIT tables: SP with set-based INSERT can avoid memory pressure from full-cache Lookup.
If using SPs, replace SQL_DeleteSnapshot + DFT_Load with a single SQL_LoadSP Execute SQL Task.
The SP internally performs DELETE + INSERT in a single transaction.

5.9 OnError Handler (Silver Package Level)

Every Silver child package (PIT, Bridge, Business Vault) has an OnError event handler at the package level. The pattern is identical to staging and Bronze child packages.

OnError Event Handler Tasks (3 steps)

Step	Task	Description
1	SCR_CaptureError (Script Task)	Captures System::ErrorDescription into User::v_ErrorMessage
2	SQL_EndStepOnError (Execute SQL Task)	Calls usp_EndStep(@StepID, 'Failed', 0) to close the step as failed.
3	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError(@BatchID, 'Error', @ErrorSource, @ErrorMessage) to record error details.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on all 3 OnError tasks. See Section 4.13 for details.

Configuration is identical to Bronze OnError handler (Section 4.13). Refer to that section for the full step-by-step Script Task code and Execute SQL Task parameter mappings.

Error Propagation Chain for Silver

Silver child OnError (SCR_CaptureError → SQL_EndStepOnError → SQL_LogErrorOnError) → SLV_Load_All.dtsx OnError → Master package OnError.
If a PIT package fails, subsequent Bridge and Business Vault packages will not execute (default behavior).
Bridge tables depend on correct Hub/Satellite data but do NOT depend on PIT tables.
Business Vault tables may depend on current Satellite data.
Consider setting MaximumErrorCount = 0 on EPT tasks for independent packages if partial Silver load is acceptable.

5.10 OnError Handler (Orchestrator Level, SLV_Load_All.dtsx)

© 2026 Mr. Sot So — Royal University of Phnom Penh

The Silver orchestrator (SLV_Load_All.dtsx) has its own OnError handler. Same pattern as all other orchestrators: usp_LogError only, no usp_EndStep, because the orchestrator does not own any steps.

OnError Event Handler Tasks (2 steps)

#	Task	Description
1	SCR_CaptureError (Script Task)	Reads System::ErrorDescription into User::v_ErrorMessage.
2	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError only. No usp_EndStep.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on both OnError tasks.

Step 1: SCR_CaptureError (Script Task)

Identical to all other layers. Reads System::ErrorDescription into User::v_ErrorMessage.

Step 2: SQL_LogErrorOnError (Execute SQL Task)

General Tab

Property	Value
Name	SQL_LogErrorOnError
Connection	CM_ETL_Control_OnError
SQLSourceType	Direct input
ResultSet	None

SQLStatement

```
SET NOCOUNT ON;

EXEC dbo.usp_LogError
    @BatchID       = ?,
    @ErrorSeverity = 'Error',
    @ErrorSource   = ?,
    @ErrorMessage  = ?;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To

CONFIDENTIAL

$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_ErrorSource	Input	NVARCHAR	1	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	2	@ErrorMessage

Silver Orchestrator Dependency Awareness

SLV_Load_All.dtsx runs 6 child packages sequentially: 3 PITs → 1 Bridge → 2 Business Vault.

Dependency analysis for Completion vs Success constraints:
- PIT tables: Independent of each other. PIT_Taxpayer failure should not block PIT_Declaration.
- Bridge: BRG_Taxpayer_Owner depends on HUB/SAT data in Bronze (not on PIT tables).
- Business Vault: BUS_ComplianceScore may read from Satellites and Links directly.

Recommendation: Use Completion constraints between PIT packages (allow partial PIT load), but keep Success constraint before Business Vault packages if they depend on PIT results.

Package Variable Requirements

SLV_Load_All.dtsx needs two variables for OnError:
- User::v_ErrorMessage (String, default empty)
- User::v_ErrorSource (String, Expression: @[System::PackageName])

p_BatchID and p_SnapshotDate are already available as package parameters.

CONFIDENTIAL

6. Gold Packages, Detailed Implementation

The Gold layer transforms the Data Vault model into a star schema optimized for analytics and reporting. It reads from DV_Bronze and DV_Silver, producing 7 Dimension tables and 4 Fact tables in DV_Gold. Two orchestrator packages (GLD_Load_All_Dimensions.dtsx and GLD_Load_All_Facts.dtsx) coordinate the build.

6.1 Package Inventory

Orchestrator Packages (04_Gold folder)

Package	Called By	Purpose
GLD_Load_All_Dimensions.dtsx	Master_Full_Load / Master_Incremental_Load	Loads all 7 Dimension tables
GLD_Load_All_Facts.dtsx	Master_Full_Load / Master_Incremental_Load	Loads all 4 Fact tables

Dimension Child Packages (04_Gold folder)

Package	Target Table	SCD Type	Source(s)
GLD_DIM_Taxpayer.dtsx	DIM_Taxpayer	SCD Type 2	HUB_Taxpayer + SAT_Taxpayer
GLD_DIM_Officer.dtsx	DIM_Officer	SCD Type 2	HUB_Officer + SAT_Officer
GLD_DIM_Category.dtsx	DIM_Category	SCD Type 1	HUB_Category + SAT_Category
GLD_DIM_Structure.dtsx	DIM_Structure	SCD Type 1	HUB_Structure + SAT_Structure
GLD_DIM_Activity.dtsx	DIM_Activity	SCD Type 1	HUB_Activity + SAT_Activity
GLD_DIM_PaymentMethod.dtsx	DIM_PaymentMethod	SCD Type 1	STG_Payment (distinct)
GLD_DIM_Status.dtsx	DIM_Status	SCD Type 1	Reference data

Fact Child Packages (04_Gold folder)

Package	Target Table	Grain	Source(s)
GLD_FACT_MonthlyDeclaration.dtsx	FACT_MonthlyDeclaration	1 row per declaration	HUB/SAT_MonthlyDecl + PIT_Declaration
GLD_FACT_Payment.dtsx	FACT_Payment	1 row per payment	HUB/SAT_Payment + LNK_DeclarationPayment

© 2026 Mr. Sot So — Royal University of Phnom Penh

GLD_FACT_MonthlySnapshot.dtsx	FACT_MonthlySnapshot	1 row per taxpayer/month	BUS_MonthlyMetrics aggregation
GLD_FACT_DeclarationLifecycle.dtsx	FACT_DeclarationLifecycle	1 row per declaration lifecycle event	SAT_MonthlyDecl + SAT_Payment status changes

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

6.2 Gold Table Design

Dimension Table (SCD Type 2): DIM_Taxpayer

```
CREATE TABLE DV_Gold.dbo.DIM_Taxpayer (
    DIM_Taxpayer_SK     INT IDENTITY(1,1) NOT NULL,  -- Surrogate key
    TaxID               VARCHAR(20)     NOT NULL,     -- Natural key
    HUB_Taxpayer_HK     VARBINARY(32)   NOT NULL,     -- DV hash key reference

    -- Descriptive attributes
    LegalBusinessName       VARCHAR(300)    NULL,
    TradingName             VARCHAR(300)    NULL,
    RegistrationDate        DATE            NULL,
    EstimatedAnnualRevenue  DECIMAL(18,2)   NULL,
    IsActive                BIT             NULL,

    -- SCD Type 2 columns
    EffectiveDate       DATETIME        NOT NULL,
    ExpiryDate          DATETIME        NULL,  -- NULL = current
    IsCurrent           BIT             NOT NULL DEFAULT 1,

    -- Audit
    GLD_LoadDate        DATETIME        NOT NULL DEFAULT GETDATE(),
    GLD_BatchID         INT             NOT NULL,

    CONSTRAINT PK_DIM_Taxpayer PRIMARY KEY (DIM_Taxpayer_SK)
);
CREATE INDEX IX_DIM_Taxpayer_TaxID ON DV_Gold.dbo.DIM_Taxpayer (TaxID, IsCurrent);
CREATE INDEX IX_DIM_Taxpayer_HK ON DV_Gold.dbo.DIM_Taxpayer (HUB_Taxpayer_HK, IsCurrent);
```

Dimension Table (SCD Type 1): DIM_Category

```
CREATE TABLE DV_Gold.dbo.DIM_Category (
    DIM_Category_SK     INT IDENTITY(1,1) NOT NULL,  -- Surrogate key
    CategoryID          INT             NOT NULL,     -- Natural key
    HUB_Category_HK     VARBINARY(32)   NOT NULL,

    CategoryName        VARCHAR(100)    NULL,
    CategoryDescription VARCHAR(500)    NULL,
    IsActive            BIT             NULL,

    GLD_LoadDate        DATETIME        NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_DIM_Category PRIMARY KEY (DIM_Category_SK),
    CONSTRAINT UK_DIM_Category_HK UNIQUE (HUB_Category_HK)
);
```

Fact Table: FACT_MonthlyDeclaration

```
CREATE TABLE DV_Gold.dbo.FACT_MonthlyDeclaration (
    FACT_MonthlyDeclaration_SK  INT IDENTITY(1,1) NOT NULL,
```

CONFIDENTIAL

```
    -- Dimension FK surrogate keys
    DIM_Taxpayer_SK     INT             NOT NULL,
    DIM_Category_SK     INT             NULL,

    DIM_Status_SK       INT             NULL,

    -- Degenerate dimensions
    DeclarationID       INT             NOT NULL,
    DeclarationMonth    INT             NOT NULL,
    DeclarationYear     INT             NOT NULL,

    -- Measures
GrossRevenue        DECIMAL(18,2)   NULL,
TaxableRevenue      DECIMAL(18,2)   NULL,
TaxAmount           DECIMAL(18,2)   NULL,
PenaltyAmount       DECIMAL(18,2)   NULL,
InterestAmount      DECIMAL(18,2)   NULL,
TotalAmount         DECIMAL(18,2)   NULL,
    GLD_LoadDate        DATETIME        NOT NULL DEFAULT GETDATE(),
    GLD_BatchID         INT             NOT NULL,
    GLD_SnapshotDate    DATETIME        NOT NULL,

    CONSTRAINT PK_FACT_MonthlyDeclaration PRIMARY KEY (FACT_MonthlyDeclaration_SK),
    CONSTRAINT FK_FACT_MDecl_Taxpayer FOREIGN KEY (DIM_Taxpayer_SK)
        REFERENCES dbo.DIM_Taxpayer (DIM_Taxpayer_SK),
    CONSTRAINT FK_FACT_MDecl_Category FOREIGN KEY (DIM_Category_SK)
        REFERENCES dbo.DIM_Category (DIM_Category_SK)
);
```

SCD Type 1 vs SCD Type 2

SCD Type 1 (DIM_Category, DIM_Structure, DIM_Activity, DIM_PaymentMethod, DIM_Status): Overwrite existing row with new values. Simple MERGE with UPDATE.

SCD Type 2 (DIM_Taxpayer, DIM_Officer): Preserve history. Close existing row (ExpiryDate + IsCurrent=0) and insert new row. Requires surrogate key for fact table FK lookups.

Fact tables always join to DIM on surrogate key. This enables temporal joins for SCD2 dimensions.

6.3 GLD_Load_All_Dimensions.dtsx, Orchestrator

Package Parameters

Parameter	Data Type	Passed From	Purpose
$Package::p_BatchID	Int32	User::v_BatchID (parent)	Current batch identifier

© 2026 Mr. Sot So — Royal University of Phnom Penh

Control Flow, 7 Execute Package Tasks (Sequential)

Step	Task	Parameters Passed to Child
1	EPT_GLD_DIM_Category	p_BatchID
2	EPT_GLD_DIM_Structure	p_BatchID
3	EPT_GLD_DIM_Activity	p_BatchID
4	EPT_GLD_DIM_PaymentMethod	p_BatchID
5	EPT_GLD_DIM_Status	p_BatchID
6	EPT_GLD_DIM_Taxpayer	p_BatchID
7	EPT_GLD_DIM_Officer	p_BatchID

Dimension Load Order

SCD Type 1 dimensions (Category, Structure, Activity, PaymentMethod, Status) load first because they are simpler and faster.

SCD Type 2 dimensions (Taxpayer, Officer) load last because they require more complex close + insert logic.

All dimensions MUST complete before facts begin, facts need dimension surrogate keys for FK lookups.

6.4 GLD_Load_All_Facts.dtsx, Orchestrator

Package Parameters

Parameter	Data Type	Passed From	Purpose
$Package::p_BatchID	Int32	User::v_BatchID (parent)	Current batch identifier

Control Flow, 4 Execute Package Tasks (Sequential)

Step	Task	Parameters Passed to Child
1	EPT_GLD_FACT_MonthlyDeclaration	p_BatchID
2	EPT_GLD_FACT_Payment	p_BatchID
3	EPT_GLD_FACT_MonthlySnapshot	p_BatchID
4	EPT_GLD_FACT_DeclarationLifecycle	p_BatchID

© 2026 Mr. Sot So — Royal University of Phnom Penh

6.5 SCD Type 1 Dimension Child Packages

Five dimension packages use SCD Type 1 (overwrite on change). Three packages (Category, Structure, Activity) use a full 6-component DFT with Lookup + Conditional Split for change detection + OLE DB Command for updates. Two packages (PaymentMethod, Status) use a simplified 4-component insert-only pattern.

❌ **COMMON ERROR, usp_EndStep wrong parameters:** usp_EndStep takes @StepID (not @BatchID) as first parameter. SQL_EndStep SQL: SET NOCOUNT ON; EXEC dbo.usp_EndStep @StepID = ?, @Status = 'Success', @RowCount = ?;, Parameter Mapping: 0 = User::v_StepID (Input, LONG), 1 = User::v_RowCount (Input, LONG). This applies to ALL layers (Bronze, Silver, Gold).

❌ **COMMON ERROR, usp_StartStep "too many arguments":** usp_StartStep takes exactly 3 parameters: @BatchID (Input), @StepName (Input), @StepID (Output). Do NOT add @PackageName or any other parameter. SQL_StartStep SQL: SET NOCOUNT ON; EXEC dbo.usp_StartStep @BatchID = ?, @StepName = ?, @StepID = ? OUTPUT;, Parameter Mapping: 0 = $Package::p_BatchID (Input, LONG), 1 = User::v_StepName (Input, VARCHAR), 2 = User::v_StepID (Output, LONG).

CONFIDENTIAL

6.5.1 GLD_DIM_Category.dtsx, Reference Template

This is the master template for all SCD Type 1 dimension packages. Build this package first, then clone for the others.

Package Parameters

Parameter	Data Type	Passed From	Purpose
`p_BatchID`	Int32	User::v_BatchID (parent)	Current ETL batch identifier

Package Variables

Variable Name	Data Type	Default Value	EvaluateAsExpression?
`v_TableName`	String	`Category`	No
`v_StepName`	String	(expression)	Yes
`v_DestTable`	String	(expression)	Yes
`v_RowCount`	Int32	`0`	No
`v_ErrorMessage`	String	(empty)	No
`v_ErrorSource`	String	(expression)	Yes
v_StepID	Int32	0	No

Expression Definitions (F4 → Properties → EvaluateAsExpression = True)

```
v_StepName:     "Load_DIM_" + @[User::v_TableName]                → Load_DIM_Category
v_DestTable:    "DIM_" + @[User::v_TableName]                     → DIM_Category
v_ErrorSource:  @[System::PackageName]                             → GLD_DIM_Category
```

Connection Managers Used

Name	Type	Database	Used By
`CM_DV_Bronze`	OLE DB	DV_Bronze	OLE_SRC (Data Flow source)
`CM_DV_Gold`	OLE DB	DV_Gold	LKP_DIM, OLE_CMD_Update, OLE_DST
`CM_ETL_Control`	OLE DB	ETL_Control	SQL_StartStep, SQL_EndStep
`CM_ETL_Control_OnError`	OLE DB	ETL_Control	OnError handler tasks

Control Flow Diagram

```
SQL_StartStep
    │  (Success)
    ▼
DFT_Load_DIM    → OLE_SRC → LKP → Match → CSPL → OLE_CMD (updates)
                          └→ No Match → ROW_CNT → OLE_DST (new rows)
    │  (Success)
    ▼
SQL_EndStep
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

Step 1: SQL_StartStep (Execute SQL Task)

Property	Value
Connection	CM_ETL_Control
SQLSourceType	Direct input

```
SET NOCOUNT ON; EXEC dbo.usp_StartStep @BatchID = ?, @StepName = ?, @StepID = ? OUTPUT;
```

Variable Name	Direction	Data Type	Param Name	Size
$Package::p_BatchID	Input	LONG	0	-1
User::v_StepName	Input	VARCHAR	1	200
User::v_StepID	Output	LONG	2	-1

CONFIDENTIAL

Step 2: DFT_Load_DIM (Data Flow Task)

Core of the SCD Type 1 pattern. Reads from Bronze, checks existing DIM records via Lookup, detects attribute changes via Conditional Split, updates changed rows via OLE DB Command, and inserts new rows via OLE DB Destination.

Data Flow Diagram

```
OLE_SRC_BronzeCategory
    │
    ▼
LKP_DIM_Category
    ├─ Match ─→ CSPL_DetectChanges ├─ Changed ─→ OLE_CMD_Update
    │                              └─ Default ─→ (discarded)
    └─ No Match ─→ ROW_CNT ─→ OLE_DST_DIM_Category
```

Creating the Data Flow Task

1. Drag Data Flow Task onto Control Flow between SQL_StartStep and SQL_EndStep.
2. Right-click → Rename to DFT_Load_DIM.
3. Connect green arrows: SQL_StartStep → DFT_Load_DIM → SQL_EndStep.

Adding Components to the Data Flow (6 components)

Double-click DFT_Load_DIM to enter Data Flow designer. Add from SSIS Toolbox:

#	Toolbox Section	Component	Rename To
1	Other Sources	OLE DB Source	`OLE_SRC_BronzeCategory`
2	Common	Lookup	`LKP_DIM_Category`
3	Common	Conditional Split	`CSPL_DetectChanges`
4	Other Destinations	OLE DB Command	`OLE_CMD_Update`
5	Common	Row Count	`ROW_CNT`
6	Other Destinations	OLE DB Destination	`OLE_DST_DIM_Category`

Connecting the Data Flow Paths

#	From	To	Output Type
1	`OLE_SRC`	`LKP_DIM`	Default Output (green)
2	`LKP_DIM`	`CSPL`	Lookup Match Output
3	`LKP_DIM`	`ROW_CNT`	Lookup No Match Output
4	`CSPL`	`OLE_CMD`	Changed output
5	`ROW_CNT`	`OLE_DST`	Default Output

CONFIDENTIAL

Component 1: OLE_SRC_BronzeCategory (OLE DB Source), Full Configuration
Double-click to open OLE DB Source Editor.

Connection Manager Tab

Property	Value
Connection	`CM_DV_Bronze`
Data access mode	SQL command

SQL Command:

```
SELECT hub.HUB_Category_HK, hub.CategoryID,
    sat.CategoryName, sat.CategoryDescription, sat.IsActive
FROM DV_Bronze.dbo.HUB_Category hub
INNER JOIN DV_Bronze.dbo.SAT_Category sat
    ON hub.HUB_Category_HK = sat.HUB_Category_HK AND sat.SAT_EndDate IS NULL
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT hub.HUB_Category_HK, hub.CategoryID, sat.CategoryName, sat.CategoryDescription, sat.IsActive FROM
DV_Bronze.dbo.HUB_Category hub INNER JOIN DV_Bronze.dbo.SAT_Category sat ON hub.HUB_Category_HK =
sat.HUB_Category_HK AND sat.SAT_EndDate IS NULL
```

Columns Tab, click to verify 5 output columns: HUB_Category_HK, CategoryID, CategoryName, CategoryDescription, IsActive.

Error Output Tab, leave default.

CONFIDENTIAL

Component 2: LKP_DIM_Category (Lookup Transform), Full Configuration
Double-click to open Lookup Transformation Editor. 4 tabs: General, Connection, Columns, Advanced.

General Tab

Property	Value
Cache mode	Full cache
No matching entries	Redirect rows to no match output

⚠ **IMPORTANT:** MUST change 'No matching entries' from default 'Fail component' to 'Redirect rows to no match output'. Without this, new business keys cause Lookup failure.

Connection Tab

Property	Value
Connection	`CM_DV_Gold`
Use results of a SQL query	(selected)

Lookup SQL:

```
SELECT HUB_Category_HK,
    CategoryName AS DIM_CategoryName,
    CategoryDescription AS DIM_CategoryDescription,
    IsActive AS DIM_IsActive
FROM DV_Gold.dbo.DIM_Category
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into LKP_DIM Connection tab:

```
SELECT HUB_Category_HK, CategoryName AS DIM_CategoryName, CategoryDescription AS DIM_CategoryDescription,
IsActive AS DIM_IsActive FROM DV_Gold.dbo.DIM_Category
```

⚠ **IMPORTANT:** DIM_ prefix aliases are CRITICAL. Without them, Lookup return columns collide with OLE_SRC columns. The DIM_ prefix creates distinct names. This enables CSPL to compare OLD (DIM_CategoryName) vs NEW (CategoryName).

Click 'Parse Query' to validate metadata.

Columns Tab
1. Drag HUB_Category_HK (left) to HUB_Category_HK (right) to create join condition.
2. Check: DIM_CategoryName, DIM_CategoryDescription, DIM_IsActive, these become new output columns.

Advanced Tab, leave defaults (full cache loads entire DIM into memory).

© 2026 Mr. Sot So — Royal University of Phnom Penh

Component 3: CSPL_DetectChanges (Conditional Split), Full Configuration
Receives Match Output from LKP_DIM. Output Name: Changed. Condition:

```
(ISNULL(CategoryName) ? "" : CategoryName)
  != (ISNULL(DIM_CategoryName) ? "" : DIM_CategoryName)
|| (ISNULL(CategoryDescription) ? "" : CategoryDescription)
  != (ISNULL(DIM_CategoryDescription) ? "" : DIM_CategoryDescription)
|| (ISNULL(IsActive) ? 0 : (DT_I4)IsActive)
  != (ISNULL(DIM_IsActive) ? 0 : (DT_I4)DIM_IsActive)
```

✓ **COPY-PASTE READY:** Copy this CSPL expression (Output Name: Changed):

```
(ISNULL(CategoryName) ? "" : CategoryName) != (ISNULL(DIM_CategoryName) ? "" : DIM_CategoryName) ||
(ISNULL(CategoryDescription) ? "" : CategoryDescription) != (ISNULL(DIM_CategoryDescription) ? "" :
DIM_CategoryDescription) || (ISNULL(IsActive) ? 0 : (DT_I4)IsActive) != (ISNULL(DIM_IsActive) ? 0 :
(DT_I4)DIM_IsActive)
```

⚠ **IMPORTANT:** ISNULL ternary pattern handles NULL comparisons. (DT_I4) cast converts BIT to integer for comparison.

Component 4: OLE_CMD_Update (OLE DB Command), Full Configuration
Right-click → Show Advanced Editor.
1. Connection Managers tab → CM_DV_Gold.
2. Component Properties tab → SqlCommand:

```
UPDATE dbo.DIM_Category SET CategoryName=?, CategoryDescription=?, IsActive=?,
    GLD_LoadDate=GETDATE() WHERE HUB_Category_HK=?
```

✓ **COPY-PASTE READY:** Copy this OLE_CMD SqlCommand:

```
UPDATE dbo.DIM_Category SET CategoryName=?, CategoryDescription=?, IsActive=?, GLD_LoadDate=GETDATE() WHERE
HUB_Category_HK=?
```

3. Column Mappings: Param_0=CategoryName, Param_1=CategoryDescription, Param_2=IsActive, Param_3=HUB_Category_HK.

⚠ **IMPORTANT:** Map source columns (NOT DIM_ prefixed). DIM_ columns are OLD values; we write NEW values.

Component 5: ROW_CNT, VariableName = User::v_RowCount. Counts new rows only.

Component 6: OLE_DST_DIM_Category (OLE DB Destination)

Property	Value
Connection	`CM_DV_Gold`
Data access mode	Table or view — fast load
Table	`dbo.DIM_Category`
Table lock	Checked

Mappings: HUB_Category_HK, CategoryID, CategoryName, CategoryDescription, IsActive. Ignore DIM_Category_SK (IDENTITY) and GLD_LoadDate (DEFAULT).

Debugging: Data Viewer Placements

Location	What to Verify
OLE_SRC → LKP	All Bronze rows have valid HK and SAT values
LKP Match → CSPL	Existing DIM rows with DIM_ prefixed columns populated
CSPL Changed → OLE_CMD	Only rows with actual attribute differences
LKP No Match → ROW_CNT	New business keys not yet in DIM
ROW_CNT → OLE_DST	Final insert count matches v_RowCount

© 2026 Mr. Sot So — Royal University of Phnom Penh

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

Step 3: SQL_EndStep (Execute SQL Task)

```
SET NOCOUNT ON; EXEC dbo.usp_EndStep @StepID = ?, @Status = 'Success', @RowCount = ?;
```

Variable Name	Direction	Data Type	Param Name
User::v_StepID	Input	LONG	0
User::v_RowCount	Input	LONG	1

OnError Event Handler (GLD_DIM_Category.dtsx)

Same 3-step pattern as all layers (Section 4.13 for click-by-click). All OnError tasks: CM_ETL_Control_OnError + TransactionOption = NotSupported.

Step	Task	SQL
1	`SCR_CaptureError`	System::ErrorDescription → User::v_ErrorMessage
2	`SQL_EndStepOnError`	usp_EndStep @BatchID=?, @StepName=?, @RowCount=0, @Status='Failed'
3	`SQL_LogErrorOnError`	usp_LogError @BatchID=?, @StepName=?, @ErrorMessage=?, @PackageName='GLD_DIM_Category'

Complete SCD1 DFT Property Summary

Component	Type	Connection	Key Setting
`OLE_SRC`	OLE DB Source	`CM_DV_Bronze`	SQL command: Hub + SAT join, SAT_EndDate IS NULL
`LKP_DIM`	Lookup	`CM_DV_Gold`	Full cache, redirect no match, DIM_ aliases
`CSPL`	Conditional Split	—	ISNULL ternary for each attribute
`OLE_CMD`	OLE DB Command	`CM_DV_Gold`	Row-by-row UPDATE, map source (not DIM_) columns
`ROW_CNT`	Row Count	—	User::v_RowCount (new rows only)
`OLE_DST`	OLE DB Destination	`CM_DV_Gold`	Fast load, DIM_Category, ignore SK + LoadDate

CONFIDENTIAL

6.5.2 GLD_DIM_Structure.dtsx, Clone from Category

Identical 6-component SCD1 pattern. v_TableName = 'Structure'.

What to Change	New Value
v_TableName	`Structure`
OLE_SRC SQL	HUB_Structure + SAT_Structure (StructureName, StructureDescription, IsActive)
LKP_DIM SQL	DIM_Structure with DIM_ aliases for tracked attributes
CSPL condition	StructureName/StructureDescription/IsActive comparisons (same ISNULL pattern)
OLE_CMD SqlCommand	UPDATE DIM_Structure SET attrs=?, GLD_LoadDate=GETDATE() WHERE HUB_Structure_HK=?
OLE_DST table	dbo.DIM_Structure
OnError @PackageName	'GLD_DIM_Structure'

CONFIDENTIAL

6.5.3 GLD_DIM_Activity.dtsx, Clone from Category

Identical 6-component SCD1 pattern. v_TableName = 'Activity'.

What to Change	New Value
v_TableName	`Activity`
OLE_SRC SQL	HUB_Activity + SAT_Activity (ActivityName, ActivityDescription, IsActive)
LKP_DIM SQL	DIM_Activity with DIM_ aliases for tracked attributes
CSPL condition	ActivityName/ActivityDescription/IsActive comparisons (same ISNULL pattern)
OLE_CMD SqlCommand	UPDATE DIM_Activity SET attrs=?, GLD_LoadDate=GETDATE() WHERE HUB_Activity_HK=?
OLE_DST table	dbo.DIM_Activity
OnError @PackageName	'GLD_DIM_Activity'

CONFIDENTIAL

6.5.4 GLD_DIM_PaymentMethod.dtsx, Simplified Pattern

SIMPLIFIED 4-component DFT (no CSPL, no OLE_CMD). Insert-only, PaymentMethod values never change.

```
OLE_SRC: SELECT DISTINCT sp.PaymentMethod FROM SAT_Payment sp WHERE SAT_EndDate IS NULL AND
PaymentMethod IS NOT NULL
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT DISTINCT sp.PaymentMethod FROM DV_Bronze.dbo.SAT_Payment sp WHERE sp.SAT_EndDate IS NULL AND
sp.PaymentMethod IS NOT NULL
```

```
LKP_DIM: SELECT PaymentMethod FROM DIM_PaymentMethod (match-only, no return columns)
No Match → ROW_CNT → OLE_DST (dbo.DIM_PaymentMethod, map PaymentMethod only)
```

⚠ **IMPORTANT:** When cloning from Category: DELETE CSPL and OLE_CMD. Connect LKP No Match directly to ROW_CNT.

OnError: @PackageName = 'GLD_DIM_PaymentMethod'.

CONFIDENTIAL

6.5.5 GLD_DIM_Status.dtsx, Simplified Pattern

SIMPLIFIED 4-component DFT. v_TableName = 'Status'.

```
OLE_SRC: Multi-SAT UNION:
  SELECT DISTINCT Status AS StatusCode FROM SAT_MonthlyDecl WHERE SAT_EndDate IS NULL
  UNION SELECT DISTINCT Status FROM SAT_Payment WHERE SAT_EndDate IS NULL
  UNION SELECT DISTINCT Status FROM SAT_AnnualDecl WHERE SAT_EndDate IS NULL
```

✓ **COPY-PASTE READY:** Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT DISTINCT Status AS StatusCode FROM DV_Bronze.dbo.SAT_MonthlyDecl WHERE SAT_EndDate IS NULL UNION
SELECT DISTINCT Status FROM DV_Bronze.dbo.SAT_Payment WHERE SAT_EndDate IS NULL UNION SELECT DISTINCT
Status FROM DV_Bronze.dbo.SAT_AnnualDecl WHERE SAT_EndDate IS NULL
```

```
LKP_DIM: SELECT StatusCode FROM DIM_Status (match on StatusCode)
No Match → ROW_CNT → OLE_DST (dbo.DIM_Status, map StatusCode only)
```

OnError: @PackageName = 'GLD_DIM_Status'.

CONFIDENTIAL

6.6 SCD Type 2 Dimension Child Packages

Two packages use SCD Type 2 (insert new version, close old). The DFT uses a 7-component pipeline: OLE_SRC → LKP → (No Match → UNION) + (Match → CSPL → Changed → UNION) → DRV_AddCols → ROW_CNT → OLE_DST. A post-DFT SQL_CloseOldVersions task end-dates superseded rows.

CONFIDENTIAL

6.6.1 GLD_DIM_Taxpayer.dtsx, Reference Template

Package Variables

Variable Name	Data Type	Default / Expression
v_TableName	String	Taxpayer
v_StepName	String	(Expr) "Load_DIM_" + v_TableName
v_LookupSQL	String	(Expr) SELECT with IsCurrent=1 + DIM_ aliases
v_DestTable	String	(Expr) "DIM_" + v_TableName
v_RowCount	Int32	0
v_ErrorMessage	String	(empty)
v_ErrorSource	String	(Expr) @[System::PackageName]

v_LookupSQL Expression:

```
"SELECT HUB_" + @[User::v_TableName] + "_HK,"
+ " LegalBusinessName AS DIM_LegalBusinessName,"
+ " TradingName AS DIM_TradingName,"
+ " EstimatedAnnualRevenue AS DIM_EstimatedAnnualRevenue,"
+ " IsActive AS DIM_IsActive"
+ " FROM DV_Gold.dbo.DIM_" + @[User::v_TableName]
+ " WHERE IsCurrent = 1"
```

Control Flow (4 Steps): SQL_StartStep → DFT_Load_DIM_SCD2 → SQL_CloseOldVersions → SQL_EndStep

DFT Data Flow Diagram

```
OLE_SRC → LKP_DIM ├─ No Match —→ UNION_ALL
                  └─ Match —→ CSPL — Changed —→ UNION_ALL
                                                  ▼
                                DRV_AddCols → ROW_CNT → OLE_DST
```

Creating DFT: 7 components

#	Component	Rename To
1	OLE DB Source	OLE_SRC_BronzeTaxpayer
2	Lookup	LKP_DIM_Taxpayer
3	Conditional Split	CSPL_DetectChanges
4	Union All	UNION_New_Changed
5	Derived Column	DRV_AddCols
6	Row Count	ROW_CNT
7	OLE DB Destination	OLE_DST_DIM_Taxpayer

OLE_SRC SQL:

```
SELECT hub.HUB_Taxpayer_HK, hub.TaxID, sat.LegalBusinessName, sat.TradingName,
    sat.RegistrationDate, sat.EstimatedAnnualRevenue, sat.IsActive
FROM DV_Bronze.dbo.HUB_Taxpayer hub
INNER JOIN DV_Bronze.dbo.SAT_Taxpayer sat
    ON hub.HUB_Taxpayer_HK = sat.HUB_Taxpayer_HK AND sat.SAT_EndDate IS NULL
```

✓ COPY-PASTE READY: Copy this single-line SQL into OLE_SRC SQL command:

```
SELECT hub.HUB_Taxpayer_HK, hub.TaxID, sat.LegalBusinessName, sat.TradingName, sat.RegistrationDate,
sat.EstimatedAnnualRevenue, sat.IsActive FROM DV_Bronze.dbo.HUB_Taxpayer hub INNER JOIN
DV_Bronze.dbo.SAT_Taxpayer sat ON hub.HUB_Taxpayer_HK = sat.HUB_Taxpayer_HK AND sat.SAT_EndDate IS NULL
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

LKP_DIM: Set via expression on parent DFT → F4 → Expressions → [LKP_DIM_Taxpayer].[SqlCommand] = @[User::v_LookupSQL].
General: Full cache, redirect no match. Columns: Match on HUB_Taxpayer_HK, return DIM_ columns.

CSPL condition:

```
(ISNULL(LegalBusinessName) ? "" : LegalBusinessName) != (ISNULL(DIM_LegalBusinessName) ? "" :
DIM_LegalBusinessName)
|| (ISNULL(TradingName) ? "" : TradingName) != (ISNULL(DIM_TradingName) ? "" : DIM_TradingName)
|| (ISNULL(EstimatedAnnualRevenue) ? 0 : EstimatedAnnualRevenue) !=
(ISNULL(DIM_EstimatedAnnualRevenue) ? 0 : DIM_EstimatedAnnualRevenue)
|| (ISNULL(IsActive) ? 0 : (DT_I4)IsActive) != (ISNULL(DIM_IsActive) ? 0 : (DT_I4)DIM_IsActive)
```

UNION_ALL: Merges No Match + Changed outputs.

DRV_AddCols (Derived Column): Adds 3 columns:

Derived Column Name	Expression	Data Type
EffectiveDate	GETDATE()	DT_DBTIMESTAMP
IsCurrent	(DT_I4)1	DT_I4
GLD_BatchID	@[$Package::p_BatchID]	DT_I4

OLE_DST: DIM_Taxpayer, fast load, table lock. Mappings:

Input	Destination
HUB_Taxpayer_HK	HUB_Taxpayer_HK
TaxID	TaxID
LegalBusinessName	LegalBusinessName
TradingName	TradingName
RegistrationDate	RegistrationDate
EstimatedAnnualRevenue	EstimatedAnnualRevenue
IsActive	IsActive
EffectiveDate	EffectiveDate
IsCurrent	IsCurrent
GLD_BatchID	GLD_BatchID
—	DIM_Taxpayer_SK (IDENTITY, ignore)
—	ExpiryDate (NULL for new rows, ignore)
—	GLD_LoadDate (DEFAULT, ignore)

© 2026 Mr. Sot So — Royal University of Phnom Penh

Step 3: SQL_CloseOldVersions (Execute SQL Task)

Property	Value
Connection	`CM_DV_Gold`
ResultSet	None

SQL Statement (no parameter mapping needed):

```
UPDATE dim SET dim.ExpiryDate = GETDATE(), dim.IsCurrent = 0
FROM DV_Gold.dbo.DIM_Taxpayer dim
WHERE dim.IsCurrent = 1
AND EXISTS (
    SELECT 1 FROM DV_Gold.dbo.DIM_Taxpayer newer
    WHERE newer.HUB_Taxpayer_HK = dim.HUB_Taxpayer_HK
    AND newer.IsCurrent = 1
    AND newer.DIM_Taxpayer_SK > dim.DIM_Taxpayer_SK)
```

✓ **TIP:** Logic: Finds current rows where a newer current row exists (higher SK for same HK) and closes the older one by setting ExpiryDate and IsCurrent=0.

OnError: @PackageName = 'GLD_DIM_Taxpayer'.

CONFIDENTIAL

6.6.2 GLD_DIM_Officer.dtsx, Clone from Taxpayer

Same 7-component SCD2 pattern. v_TableName = 'Officer'. Changes:

What to Change	New Value
v_TableName	`Officer`
v_LookupSQL	FirstName AS DIM_FirstName, LastName AS DIM_LastName, Department AS DIM_Department, IsActive AS DIM_IsActive
OLE_SRC SQL	HUB_Officer + SAT_Officer (OfficerCode, FirstName, LastName, Department, IsActive)
CSPL condition	FirstName/LastName/Department/IsActive comparisons
OLE_DST	DIM_Officer: OfficerCode, HUB_Officer_HK, FirstName, LastName, Department, IsActive, EffectiveDate, IsCurrent, GLD_BatchID
SQL_CloseOldVersions	UPDATE DIM_Officer (same pattern, different table)
OnError	@PackageName = 'GLD_DIM_Officer'

CONFIDENTIAL

6.7 Fact Child Packages

Four Fact packages use a DELETE + INSERT pattern: SQL_DeleteSnapshot clears existing snapshot data, then DFT_Load_Fact inserts fresh data with dimension surrogate key lookups. Control flow: SQL_StartStep → SQL_DeleteSnapshot → DFT_Load_Fact → SQL_EndStep. All Fact packages use expression-driven v_SourceSQL and v_DeleteSQL variables, embedding p_BatchID and p_SnapshotDate via DECLARE statements so no OLE DB Source parameter mapping is needed.

CONFIDENTIAL

✓ **COPY-PASTE READY:** Run this script once after creating the Gold DDL (Section 2.4). Copy all ALTER statements and execute in SSMS against DV_Gold database.

```
ALTER TABLE DV_Gold.dbo.FACT_DeclarationLifecycle ALTER COLUMN DIM_Status_SK INT NULL;
ALTER TABLE DV_Gold.dbo.FACT_DeclarationLifecycle ALTER COLUMN DIM_Taxpayer_SK INT NULL;
-- FACT_DeclarationLifecycle

ALTER TABLE DV_Gold.dbo.FACT_MonthlySnapshot ALTER COLUMN DIM_Taxpayer_SK INT NULL;
-- FACT_MonthlySnapshot

ALTER TABLE DV_Gold.dbo.FACT_Payment ALTER COLUMN DIM_Status_SK INT NULL;
ALTER TABLE DV_Gold.dbo.FACT_Payment ALTER COLUMN DIM_PaymentMethod_SK INT NULL;
ALTER TABLE DV_Gold.dbo.FACT_Payment ALTER COLUMN DIM_Taxpayer_SK INT NULL;
-- FACT_Payment

ALTER TABLE DV_Gold.dbo.FACT_MonthlyDeclaration ALTER COLUMN DIM_Status_SK INT NULL;
ALTER TABLE DV_Gold.dbo.FACT_MonthlyDeclaration ALTER COLUMN DIM_Category_SK INT NULL;
ALTER TABLE DV_Gold.dbo.FACT_MonthlyDeclaration ALTER COLUMN DIM_Taxpayer_SK INT NULL;
-- FACT_MonthlyDeclaration
```

ALTER TABLE — Allow NULL Dimension Surrogate Keys in Fact Tables

⚠ **PREREQUISITE:** All Fact Lookup transforms use 'Ignore failure' for no-match handling, which produces NULL surrogate keys when a dimension record is missing. The Fact table DIM_*_SK columns MUST allow NULLs. Run the following ALTER TABLE script BEFORE building the Fact packages:

6.7.1 GLD_FACT_MonthlyDeclaration.dtsx, Reference Template

Package Parameters

Parameter	Data Type	Description
`p_BatchID`	Int32	Current ETL batch ID from orchestrator
`p_SnapshotDate`	DateTime	Gold layer snapshot date from orchestrator

Package Variables

Variable Name	Data Type	Default / Expression	EvaluateAsExpression?
`v_TableName`	String	`FACT_MonthlyDeclaration`	No
`v_StepName`	String	(Expr) "Load_" + v_TableName	Yes
`v_DeleteSQL`	String	(Expr) see below	Yes
`v_SourceSQL`	String	(Expr) see below	Yes
`v_DestTable`	String	(Expr) v_TableName	Yes
`v_RowCount`	Int32	`0`	No
`v_StepID`	Int32	`0`	No
`v_ErrorMessage`	String	(empty)	No
`v_ErrorSource`	String	(Expr) @[System::PackageName]	Yes

Expression Definitions (F4 → Properties → EvaluateAsExpression = True)

```
v_StepName:     "Load_" + @[User::v_TableName]          → Load_FACT_MonthlyDeclaration
v_DestTable:    @[User::v_TableName]                     → FACT_MonthlyDeclaration
v_ErrorSource:  @[System::PackageName]                   → GLD_FACT_MonthlyDeclaration
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

v_DeleteSQL Expression (EvaluateAsExpression = True)

```
"SET NOCOUNT ON;"
+ " DELETE FROM DV_Gold.dbo." + @[User::v_TableName]
+ " WHERE GLD_SnapshotDate = '"
+ (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate]
```

✓ **COPY-PASTE READY:** Copy this single-line expression into v_DeleteSQL Expression property:

```
"SET NOCOUNT ON; DELETE FROM DV_Gold.dbo." + @[User::v_TableName] + " WHERE GLD_SnapshotDate = '" +
(DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate] + "'"
```

```
+ "'"
```

At runtime (v_TableName = 'FACT_MonthlyDeclaration', p_SnapshotDate = 2026-02-22):

```
SET NOCOUNT ON; DELETE FROM DV_Gold.dbo.FACT_MonthlyDeclaration WHERE GLD_SnapshotDate = '2026-
02-22'
```

✓ **TIP:** (DT_WSTR,30)(DT_DBDATE) is a double cast: DateTime → Date-only → String. This embeds the snapshot date directly. No parameter mapping needed on SQL_DeleteSnapshot.

CONFIDENTIAL

v_SourceSQL Expression (EvaluateAsExpression = True)

⚠ **IMPORTANT:** This is the full SSIS expression string. Copy it exactly into the v_SourceSQL variable's Expression property (Variables window → select v_SourceSQL → F4 → Expression). SSIS evaluates this at runtime, producing a complete SQL query with embedded parameter values.

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + ";"
+ " DECLARE @GLD_SnapshotDate DATETIME = '"
+ (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate]
+ "';"
+ " SELECT"
+ "   lnk.HUB_Taxpayer_HK,"
+ "   sat_tp.CategoryID,"
+ "   sat.Status,"
+ "   hd.DeclarationID,"
+ "   sat.DeclarationMonth,"
+ "   sat.DeclarationYear,"
+ "   sat.GrossRevenue,"
+ "   sat.TaxableRevenue,"
+ "   sat.TaxAmount,"
+ "   sat.PenaltyAmount,"
+ "   sat.InterestAmount,"
+ "   sat.TotalAmount,"
+ "   @GLD_BatchID AS GLD_BatchID,"
+ "   @GLD_SnapshotDate AS GLD_SnapshotDate"
+ " FROM DV_Bronze.dbo.SAT_MonthlyDecl sat"
+ " INNER JOIN DV_Bronze.dbo.HUB_Declaration hd"
+ "   ON sat.HUB_Declaration_HK = hd.HUB_Declaration_HK"
+ " INNER JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk"
+ "   ON hd.HUB_Declaration_HK = lnk.HUB_Declaration_HK"
+ " LEFT JOIN DV_Bronze.dbo.SAT_Taxpayer sat_tp"
+ "   ON lnk.HUB_Taxpayer_HK = sat_tp.HUB_Taxpayer_HK"
+ "   AND sat_tp.SAT_EndDate IS NULL"
+ " WHERE sat.SAT_EndDate IS NULL"
```

✓ **COPY-PASTE READY:** Copy this single-line expression into v_SourceSQL Expression property:

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + "; DECLARE @GLD_SnapshotDate DATETIME
= '" + (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate] + "'; SELECT lnk.HUB_Taxpayer_HK,
sat_tp.CategoryID, sat.Status, hd.DeclarationID, sat.DeclarationMonth, sat.DeclarationYear,
sat.GrossRevenue, sat.TaxableRevenue, sat.TaxAmount, sat.PenaltyAmount, sat.InterestAmount,
sat.TotalAmount, @GLD_BatchID AS GLD_BatchID, @GLD_SnapshotDate AS GLD_SnapshotDate FROM
DV_Bronze.dbo.SAT_MonthlyDecl sat INNER JOIN DV_Bronze.dbo.HUB_Declaration hd ON sat.HUB_Declaration_HK =
hd.HUB_Declaration_HK INNER JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk ON hd.HUB_Declaration_HK =
lnk.HUB_Declaration_HK LEFT JOIN DV_Bronze.dbo.SAT_Taxpayer sat_tp ON lnk.HUB_Taxpayer_HK =
sat_tp.HUB_Taxpayer_HK AND sat_tp.SAT_EndDate IS NULL WHERE sat.SAT_EndDate IS NULL"
```

At runtime (p_BatchID = 1, p_SnapshotDate = 2026-02-22):

```
DECLARE @GLD_BatchID INT = 1;
DECLARE @GLD_SnapshotDate DATETIME = '2026-02-22';
SELECT
    lnk.HUB_Taxpayer_HK,
    sat_tp.CategoryID,
    sat.Status,
    hd.DeclarationID,
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
    sat.DeclarationMonth, sat.DeclarationYear,
    sat.GrossRevenue, sat.TaxableRevenue, sat.TaxAmount,
    sat.PenaltyAmount, sat.InterestAmount, sat.TotalAmount,
    @GLD_BatchID AS GLD_BatchID,
    @GLD_SnapshotDate AS GLD_SnapshotDate
FROM DV_Bronze.dbo.SAT_MonthlyDecl sat
INNER JOIN DV_Bronze.dbo.HUB_Declaration hd
    ON sat.HUB_Declaration_HK = hd.HUB_Declaration_HK
INNER JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk
    ON hd.HUB_Declaration_HK = lnk.HUB_Declaration_HK
LEFT JOIN DV_Bronze.dbo.SAT_Taxpayer sat_tp
    ON lnk.HUB_Taxpayer_HK = sat_tp.HUB_Taxpayer_HK
    AND sat_tp.SAT_EndDate IS NULL
WHERE sat.SAT_EndDate IS NULL
```

✓ **TIP:** No OLE DB Source parameter mapping needed. (DT_WSTR,10) casts INT to string. (DT_WSTR,30)(DT_DBDATE) double-casts DateTime to date string. The DECLARE approach embeds all parameters directly in the SQL string.

CONFIDENTIAL

Connection Managers Used

Connection Manager	Database	Used By
`CM_DV_Bronze`	DV_Bronze	OLE_SRC (reads SAT + HUB + LNK)
`CM_DV_Gold`	DV_Gold	SQL_DeleteSnapshot, 3 Lookup transforms, OLE_DST
`CM_ETL_Control`	ETL_Control	SQL_StartStep, SQL_EndStep
`CM_ETL_Control_OnError`	ETL_Control	OnError handler tasks

Control Flow (4 Steps)

```
SQL_StartStep
    │  (Success)
    ▼
SQL_DeleteSnapshot     ← v_DeleteSQL expression
    │  (Success)
    ▼
DFT_Load_Fact          ← OLE_SRC → LKP_Taxpayer → LKP_Category → LKP_Status → ROW_CNT → OLE_DST
    │  (Success)
    ▼
SQL_EndStep
```

CONFIDENTIAL

Step 1: SQL_StartStep (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control`
SQLSourceType	Direct input

SQL Statement:

```
SET NOCOUNT ON;
EXEC dbo.usp_StartStep @BatchID = ?, @StepName = ?, @StepID = ? OUTPUT;
```

Parameter Mapping:

Variable Name	Direction	Data Type	Param Name	Size
`$Package::p_BatchID`	Input	LONG	`0`	-1
`User::v_StepName`	Input	VARCHAR	`1`	200
`User::v_StepID`	Output	LONG	`2`	-1

CONFIDENTIAL

Step 2: SQL_DeleteSnapshot (Execute SQL Task)

Property	Value
Connection	`CM_DV_Gold`
SQLSourceType	Direct input (expression override)

Setting SQL via Expression:
1. Click SQL_DeleteSnapshot in Control Flow tab.
2. F4 (Properties window) → scroll to Expressions.
3. Click the '...' button next to Expressions.
4. Property = SqlStatementSource, Expression = @[User::v_DeleteSQL].
5. Click OK. The SqlStatementSource now shows the icon indicating expression override.

No parameter mapping needed, the snapshot date is embedded in the v_DeleteSQL expression string via (DT_WSTR,30)(DT_DBDATE) cast.

CONFIDENTIAL

Step 3: DFT_Load_Fact (Data Flow Task)

The Fact DFT chains 3 Lookup transforms to resolve dimension surrogate keys (DIM_Taxpayer_SK, DIM_Category_SK, DIM_Status_SK) before inserting into the Fact table. Each Lookup uses 'Ignore failure' for no-match handling, which passes rows through with NULL surrogate keys when a dimension record is missing.

Data Flow Diagram

```
OLE_SRC (v_SourceSQL, CM_DV_Bronze)
    │  (14 columns: 3 lookup keys + 9 measures + 2 audit)
    ▼
LKP_DIM_Taxpayer       ← HUB_Taxpayer_HK → DIM_Taxpayer_SK
    │  (Lookup Match Output, single output, NULL SK if no match)
    ▼
LKP_DIM_Category       ← CategoryID → DIM_Category_SK
    │  (Lookup Match Output)
    ▼
LKP_DIM_Status         ← Status → DIM_Status_SK
    │  (Lookup Match Output)
    ▼
ROW_CNT —→ OLE_DST_FACT_MonthlyDeclaration
```

Creating the Data Flow Task

1. In the Control Flow tab, drag a Data Flow Task from the SSIS Toolbox between SQL_DeleteSnapshot and SQL_EndStep.
2. Right-click → Rename to DFT_Load_Fact.
3. Connect green Success precedence constraints: SQL_DeleteSnapshot → DFT_Load_Fact → SQL_EndStep.

6 Components to Add (in order from SSIS Toolbox)

#	Component Name	Toolbox Category	Type
1	`OLE_SRC_FACT_MonthlyDecl`	Other Sources	OLE DB Source
2	`LKP_DIM_Taxpayer`	Common	Lookup
3	`LKP_DIM_Category`	Common	Lookup
4	`LKP_DIM_Status`	Common	Lookup
5	`ROW_CNT`	Common	Row Count
6	`OLE_DST_FACT_MonthlyDecl`	Other Destinations	OLE DB Destination

Data Flow Path Connections (green arrows)

From	To	Output Type	How to Connect
`OLE_SRC`	`LKP_DIM_Taxpayer`	Default Output	Drag green arrow from OLE_SRC to LKP
`LKP_DIM_Taxpayer`	`LKP_DIM_Category`	Lookup Match Output	Drag green arrow (only output when Ignore failure)
`LKP_DIM_Category`	`LKP_DIM_Status`	Lookup Match Output	Drag green arrow
`LKP_DIM_Status`	`ROW_CNT`	Lookup Match Output	Drag green arrow
`ROW_CNT`	`OLE_DST`	Default Output	Drag green arrow

⚠ **IMPORTANT:** Lookup No Match Handling ('Ignore failure' Pattern: All 3 Lookups MUST set General Tab → 'Specify how to handle rows with no matching entries' = 'Ignore failure'. With this setting, rows without a matching dimension record pass through with NULL for the output column (e.g., DIM_Taxpayer_SK = NULL). There is only ONE output path per Lookup (Lookup Match Output)) no separate No Match output exists. This is the simplest pattern for Fact dimension key resolution.

© 2026 Mr. Sot So — Royal University of Phnom Penh

✓ **TIP:** Why 'Ignore failure' instead of 'Redirect rows to no match output'? Redirect creates TWO separate output paths per Lookup, requiring Union All transforms to merge them back. With 3 chained Lookups this would require 3 extra Union All transforms. 'Ignore failure' avoids this complexity entirely while still allowing NULL surrogate keys for missing dimension records.

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

Component 1: OLE_SRC_FACT_MonthlyDecl (OLE DB Source), Full Configuration
Double-click OLE_SRC to open the OLE DB Source Editor.

Connection Manager Tab:

Property	Value
OLE DB Connection Manager	`CM_DV_Bronze`
Data access mode	SQL command from variable
Variable name	`User::v_SourceSQL`

Columns Tab, Metadata Setup (one-time):
When using 'SQL command from variable', SSIS cannot resolve metadata at design time. Follow this procedure:
1. Temporarily set Data access mode to 'SQL command'.
2. Paste the runtime SQL (the DECLARE + SELECT query shown above) into the SQL command text box.
3. Click Columns tab, SSIS reads column metadata from the query. Verify 14 output columns appear.
4. Switch back to 'SQL command from variable'.
5. Set Variable name to User::v_SourceSQL.
6. Click OK.

Output Columns (14 columns):

Column	Data Type	Source	Purpose
`HUB_Taxpayer_HK`	DT_BYTES (32)	lnk.HUB_Taxpayer_HK	LKP_DIM_Taxpayer join key
`CategoryID`	DT_I4	sat_tp.CategoryID	LKP_DIM_Category join key
`Status`	DT_STR (50)	sat.Status	LKP_DIM_Status join key
`DeclarationID`	DT_I4	hd.DeclarationID	Fact degenerate dimension
`DeclarationMonth`	DT_I4	sat.DeclarationMonth	Fact measure
`DeclarationYear`	DT_I4	sat.DeclarationYear	Fact measure
`GrossRevenue`	DT_NUMERIC (18,2)	sat.GrossRevenue	Fact measure
`TaxableRevenue`	DT_NUMERIC (18,2)	sat.TaxableRevenue	Fact measure
`TaxAmount`	DT_NUMERIC (18,2)	sat.TaxAmount	Fact measure
`PenaltyAmount`	DT_NUMERIC (18,2)	sat.PenaltyAmount	Fact measure
`InterestAmount`	DT_NUMERIC (18,2)	sat.InterestAmount	Fact measure
`TotalAmount`	DT_NUMERIC (18,2)	sat.TotalAmount	Fact measure
`GLD_BatchID`	DT_I4	@GLD_BatchID	Audit (from DECLARE)
`GLD_SnapshotDate`	DT_DBTIMESTAMP	@GLD_SnapshotDate	Audit (from DECLARE)

Error Output Tab: Leave default. If the source SQL fails, the DFT fails and triggers OnError.

© 2026 Mr. Sot So — Royal University of Phnom Penh

Component 2: LKP_DIM_Taxpayer (Lookup Transform), Full Configuration
Double-click LKP_DIM_Taxpayer to open the Lookup Transformation Editor. 4 tabs: General, Connection, Columns, Advanced.

General Tab:

Property	Value
Cache mode	Full cache
Connection type	OLE DB Connection Manager
Specify how to handle rows with no matching entries	Ignore failure

⚠ **IMPORTANT:** 'Ignore failure' is critical. With this setting, when a row has no matching DIM_Taxpayer record (e.g., new Taxpayer not yet in dimension), the row passes through with DIM_Taxpayer_SK = NULL. The row is NOT dropped, NOT redirected, and NOT failed. This produces a single output path.

Connection Tab:

Property	Value
OLE DB Connection Manager	`CM_DV_Gold`
Use results of an SQL query	Selected
SQL Query	`SELECT HUB_Taxpayer_HK, DIM_Taxpayer_SK FROM DV_Gold.dbo.DIM_Taxpayer WHERE IsCurrent = 1`

Click 'Preview' to verify the query returns data. Click 'Parse Query' to validate syntax.

✓ **TIP:** WHERE IsCurrent = 1 filters to only current SCD Type 2 dimension records. Without this filter, a Taxpayer with multiple historical versions would load multiple rows into the Lookup cache, causing duplicate match errors.

Columns Tab, Step-by-Step Setup:
1. In Available Input Columns (left panel), you see all 14 columns from OLE_SRC. In Available Lookup Columns (right panel), you see HUB_Taxpayer_HK and DIM_Taxpayer_SK from the Lookup SQL.
2. Drag HUB_Taxpayer_HK from the left panel to HUB_Taxpayer_HK in the right panel. This creates the join condition (shown as a line connecting the two columns).
3. In the bottom grid, CHECK the checkbox for DIM_Taxpayer_SK under the 'Lookup Column' header. This adds it as an output column.
4. Set Output Alias = DIM_Taxpayer_SK (default is fine).
5. Verify the bottom grid shows: Input Column = HUB_Taxpayer_HK, Lookup Column = HUB_Taxpayer_HK (join). Second row: Lookup Column = DIM_Taxpayer_SK, Output Alias = DIM_Taxpayer_SK (output).

Advanced Tab:

Property	Value
Enable memory restriction	Unchecked (use Full cache in memory)
Treat duplicate keys as errors	Checked

© 2026 Mr. Sot So — Royal University of Phnom Penh

Component 3: LKP_DIM_Category (Lookup Transform), Full Configuration
Double-click LKP_DIM_Category to open the Lookup Transformation Editor.

General Tab:

Property	Value
Cache mode	Full cache
Connection type	OLE DB Connection Manager
No matching entries	Ignore failure

Connection Tab:

Property	Value
OLE DB Connection Manager	`CM_DV_Gold`
SQL Query	`SELECT CategoryID, DIM_Category_SK FROM DV_Gold.dbo.DIM_Category`

Columns Tab:
1. Drag CategoryID (input, from OLE_SRC pipeline) to CategoryID (lookup). This creates the join.
2. CHECK DIM_Category_SK checkbox in bottom grid. Output Alias = DIM_Category_SK.
3. Verify: Join on CategoryID → CategoryID. Output: DIM_Category_SK.

Note: CategoryID comes from sat_tp.CategoryID in the OLE_SRC query (from SAT_Taxpayer via LEFT JOIN). If a Taxpayer has no SAT_Taxpayer record, CategoryID = NULL, and the Lookup will produce DIM_Category_SK = NULL as well.

CONFIDENTIAL

Component 4: LKP_DIM_Status (Lookup Transform), Full Configuration
Double-click LKP_DIM_Status to open the Lookup Transformation Editor.

General Tab:

Property	Value
Cache mode	Full cache
Connection type	OLE DB Connection Manager
No matching entries	Ignore failure

Connection Tab:

Property	Value
OLE DB Connection Manager	`CM_DV_Gold`
SQL Query	`SELECT StatusCode, DIM_Status_SK FROM DV_Gold.dbo.DIM_Status`

Columns Tab:
1. Drag Status (input, from OLE_SRC pipeline) to StatusCode (lookup). Note the different column names, SSIS allows mapping columns with different names.
2. CHECK DIM_Status_SK checkbox in bottom grid. Output Alias = DIM_Status_SK.
3. Verify: Join on Status → StatusCode. Output: DIM_Status_SK.

> ⚠ **IMPORTANT:** Status (input) maps to StatusCode (lookup), the column names are different. In the Columns tab, you must manually drag from the left 'Status' column to the right 'StatusCode' column. SSIS will NOT auto-match columns with different names.

CONFIDENTIAL

Component 5: ROW_CNT (Row Count Transform)

Property	Value
VariableName	`User::v_RowCount`

Select ROW_CNT → F4 (Properties) → set VariableName to User::v_RowCount. Counts all rows flowing into the Fact table (including rows with NULL surrogate keys).

Component 6: OLE_DST_FACT_MonthlyDecl (OLE DB Destination), Full Configuration
Double-click OLE_DST to open the OLE DB Destination Editor.

Connection Manager Tab:

Property	Value
OLE DB Connection Manager	`CM_DV_Gold`
Data access mode	Table or view — fast load
Name of the table or the view	`[dbo].[FACT_MonthlyDeclaration]`
Table lock	Checked
Check constraints	Checked
Keep identity	Unchecked (IDENTITY auto-generates SK)

Mappings Tab, Column Mappings:
Click the Mappings tab. The left panel shows Available Input Columns (17 columns: 14 from OLE_SRC + 3 from Lookups). The right panel shows Available Destination Columns (16 columns from FACT_MonthlyDeclaration DDL).

Input Column	Destination Column	Source
(not mapped, IDENTITY)	`FACT_MonthlyDeclaration_SK`	Auto-generated by SQL Server
`DIM_Taxpayer_SK`	`DIM_Taxpayer_SK`	LKP_DIM_Taxpayer output
`DIM_Category_SK`	`DIM_Category_SK`	LKP_DIM_Category output
`DIM_Status_SK`	`DIM_Status_SK`	LKP_DIM_Status output
`DeclarationID`	`DeclarationID`	OLE_SRC (hd.DeclarationID)
`DeclarationMonth`	`DeclarationMonth`	OLE_SRC (sat.DeclarationMonth)
`DeclarationYear`	`DeclarationYear`	OLE_SRC (sat.DeclarationYear)
`GrossRevenue`	`GrossRevenue`	OLE_SRC (sat.GrossRevenue)
`TaxableRevenue`	`TaxableRevenue`	OLE_SRC (sat.TaxableRevenue)
`TaxAmount`	`TaxAmount`	OLE_SRC (sat.TaxAmount)
`PenaltyAmount`	`PenaltyAmount`	OLE_SRC (sat.PenaltyAmount)
`InterestAmount`	`InterestAmount`	OLE_SRC (sat.InterestAmount)
`TotalAmount`	`TotalAmount`	OLE_SRC (sat.TotalAmount)
(not mapped, DEFAULT)	`GLD_LoadDate`	GETDATE() column default
`GLD_BatchID`	`GLD_BatchID`	OLE_SRC (DECLARE @GLD_BatchID)
`GLD_SnapshotDate`	`GLD_SnapshotDate`	OLE_SRC (DECLARE @GLD_SnapshotDate)

⚠ **IMPORTANT:** Do NOT map FACT_MonthlyDeclaration_SK (it is an IDENTITY column (auto-incrementing). Do NOT map GLD_LoadDate) it has a DEFAULT constraint of GETDATE(). If you map either of these, the insert will fail. In the Mappings tab, leave them as '<ignore>' in the Input Column dropdown.

Error Output Tab: Leave default. Primary key violations cause the DFT to fail, triggering OnError.

© 2026 Mr. Sot So — Royal University of Phnom Penh

Step 4: SQL_EndStep (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control`
SQLSourceType	Direct input

SQL Statement:

```
SET NOCOUNT ON;
EXEC dbo.usp_EndStep @StepID = ?, @Status = 'Success', @RowCount = ?;
```

Parameter Mapping:

Variable Name	Direction	Data Type	Param Name	Size
`User::v_StepID`	Input	LONG	`0`	-1
`User::v_RowCount`	Input	LONG	`1`	-1

CONFIDENTIAL

OnError Event Handler (GLD_FACT_MonthlyDeclaration.dtsx)

The OnError handler uses the same 3-step pattern as all other layers. All OnError tasks MUST use CM_ETL_Control_OnError and set TransactionOption = NotSupported.

OnError Step 1: SCR_CaptureError (Script Task)

Property	Value
ReadOnlyVariables	`System::ErrorDescription`
ReadWriteVariables	`User::v_ErrorMessage`

```
public void Main()
{
    string errorDesc = Dts.Variables["System::ErrorDescription"].Value.ToString();
    Dts.Variables["User::v_ErrorMessage"].Value = errorDesc;
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

OnError Step 2: SQL_EndStepOnError (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control_OnError`
TransactionOption	NotSupported

```
EXEC dbo.usp_EndStep @StepID = ?, @Status = 'Failed', @RowCount = 0, @ErrorMessage = ?;
```

Parameter Mapping: Param 0 = User::v_StepID (LONG), Param 1 = User::v_ErrorMessage (VARCHAR, 4000)

OnError Step 3: SQL_LogErrorOnError (Execute SQL Task)

Property	Value
Connection	`CM_ETL_Control_OnError`
TransactionOption	NotSupported

```
EXEC dbo.usp_LogError @BatchID = ?, @StepLogID = ?, @ErrorSeverity = 'Error', @ErrorSource = ?,
@ErrorMessage = ?;
```

Parameter Mapping: Param 0 = $Package::p_BatchID (LONG), Param 1 = User::v_StepID (LONG), Param 2 = User::v_ErrorSource (VARCHAR, 200), Param 3 = User::v_ErrorMessage (VARCHAR, 4000)

CONFIDENTIAL

Debugging: Data Viewer Placements

Location	What to Verify
OLE_SRC → LKP_DIM_Taxpayer	All 14 columns present with correct HASHBYTES, CategoryID, Status values
LKP_DIM_Taxpayer → LKP_DIM_Category	DIM_Taxpayer_SK populated (or NULL for missing Taxpayers)
LKP_DIM_Category → LKP_DIM_Status	DIM_Category_SK populated (or NULL for missing Categories)
LKP_DIM_Status → ROW_CNT	All 3 SK columns + original measures. DIM_Status_SK populated (or NULL)
ROW_CNT → OLE_DST	Final 17-column pipeline: 3 SKs + 9 measures + 2 audit + 3 join keys (pass-through)

Complete Fact DFT Property Summary

Component	Type	Connection	Key Setting
`OLE_SRC`	OLE DB Source	`CM_DV_Bronze`	SQL command from variable (v_SourceSQL)
`LKP_DIM_Taxpayer`	Lookup	`CM_DV_Gold`	Full cache, Ignore failure, IsCurrent=1 filter
`LKP_DIM_Category`	Lookup	`CM_DV_Gold`	Full cache, Ignore failure
`LKP_DIM_Status`	Lookup	`CM_DV_Gold`	Full cache, Ignore failure, Status→StatusCode
`ROW_CNT`	Row Count	-	`User::v_RowCount`
`OLE_DST`	OLE DB Destination	`CM_DV_Gold`	Fast load, table lock, IDENTITY+DEFAULT excluded

6.7.2 GLD_FACT_Payment.dtsx, Clone from MonthlyDeclaration

Loads payment facts with dimension key lookups for Taxpayer, PaymentMethod, and Status. Same 4-step control flow. Clone from MonthlyDeclaration template. Key difference: resolves HUB_Taxpayer_HK via a 2-hop Link chain (Payment → LNK_DeclarationPayment → LNK_TaxpayerDeclaration → HUB_Taxpayer_HK).

Variable Changes:

Variable	New Value
`v_TableName`	`FACT_Payment`

All expressions auto-update: v_StepName → Load_FACT_Payment, v_DestTable → FACT_Payment, v_DeleteSQL auto-updates table name.

CONFIDENTIAL

v_SourceSQL Expression (EvaluateAsExpression = True):

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + ";"
+ " DECLARE @GLD_SnapshotDate DATETIME = '"
+ (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate]
+ "';"
+ " SELECT"
+ "   lnk_td.HUB_Taxpayer_HK,"
+ "   sp.PaymentMethod,"
+ "   sp.Status,"
+ "   hp.PaymentID,"
+ "   sp.DeclarationID,"
+ "   sp.AnnualDeclarationID,"
+ "   sp.PaymentAmount,"
+ "   sp.PaymentDate,"
+ "   @GLD_BatchID AS GLD_BatchID,"
+ "   @GLD_SnapshotDate AS GLD_SnapshotDate"
+ " FROM DV_Bronze.dbo.SAT_Payment sp"
+ " INNER JOIN DV_Bronze.dbo.HUB_Payment hp"
+ "   ON sp.HUB_Payment_HK = hp.HUB_Payment_HK"
+ " LEFT JOIN DV_Bronze.dbo.LNK_DeclarationPayment lnk_dp"
+ "   ON hp.HUB_Payment_HK = lnk_dp.HUB_Payment_HK"
+ " LEFT JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk_td"
+ "   ON lnk_dp.HUB_Declaration_HK = lnk_td.HUB_Declaration_HK"
+ " WHERE sp.SAT_EndDate IS NULL"
```

✓ **COPY-PASTE READY:** Copy this single-line expression into v_SourceSQL Expression property:

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + "; DECLARE @GLD_SnapshotDate DATETIME
= '" + (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate] + "'; SELECT lnk_td.HUB_Taxpayer_HK,
sp.PaymentMethod, sp.Status, hp.PaymentID, sp.DeclarationID, sp.AnnualDeclarationID, sp.PaymentAmount,
sp.PaymentDate, @GLD_BatchID AS GLD_BatchID, @GLD_SnapshotDate AS GLD_SnapshotDate FROM
DV_Bronze.dbo.SAT_Payment sp INNER JOIN DV_Bronze.dbo.HUB_Payment hp ON sp.HUB_Payment_HK =
hp.HUB_Payment_HK LEFT JOIN DV_Bronze.dbo.LNK_DeclarationPayment lnk_dp ON hp.HUB_Payment_HK =
lnk_dp.HUB_Payment_HK LEFT JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk_td ON lnk_dp.HUB_Declaration_HK =
lnk_td.HUB_Declaration_HK WHERE sp.SAT_EndDate IS NULL"
```

At runtime (p_BatchID = 1, p_SnapshotDate = 2026-02-22):

```
DECLARE @GLD_BatchID INT = 1;
DECLARE @GLD_SnapshotDate DATETIME = '2026-02-22';
SELECT
    lnk_td.HUB_Taxpayer_HK,
    sp.PaymentMethod,
    sp.Status,
    hp.PaymentID,
    sp.DeclarationID, sp.AnnualDeclarationID,
    sp.PaymentAmount, sp.PaymentDate,
    @GLD_BatchID AS GLD_BatchID,
    @GLD_SnapshotDate AS GLD_SnapshotDate
FROM DV_Bronze.dbo.SAT_Payment sp
INNER JOIN DV_Bronze.dbo.HUB_Payment hp
    ON sp.HUB_Payment_HK = hp.HUB_Payment_HK
LEFT JOIN DV_Bronze.dbo.LNK_DeclarationPayment lnk_dp
    ON hp.HUB_Payment_HK = lnk_dp.HUB_Payment_HK
LEFT JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk_td
    ON lnk_dp.HUB_Declaration_HK = lnk_td.HUB_Declaration_HK
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
WHERE sp.SAT_EndDate IS NULL
```

⚠ **IMPORTANT:** 2-hop Link resolution: Payment has no direct Link to Taxpayer. The path is HUB_Payment → LNK_DeclarationPayment → LNK_TaxpayerDeclaration → HUB_Taxpayer_HK. Both Link joins use LEFT JOIN because a Payment may not yet be linked to a Declaration (AnnualDeclarationID-only payments).

CONFIDENTIAL

OLE_SRC Output Columns (10 columns):

Column	Data Type	Purpose
HUB_Taxpayer_HK	DT_BYTES (32)	LKP_DIM_Taxpayer join key
PaymentMethod	DT_STR (50)	LKP_DIM_PaymentMethod join key
Status	DT_STR (50)	LKP_DIM_Status join key
PaymentID	DT_I4	Fact degenerate dimension
DeclarationID	DT_I4	Fact measure (nullable)
AnnualDeclarationID	DT_I4	Fact measure (nullable)
PaymentAmount	DT_NUMERIC (18,2)	Fact measure
PaymentDate	DT_DBDATE	Fact measure
GLD_BatchID	DT_I4	Audit (DECLARE)
GLD_SnapshotDate	DT_DBTIMESTAMP	Audit (DECLARE)

DFT_Load_Fact Data Flow (5 components):

```
OLE_SRC (CM_DV_Bronze, v_SourceSQL)
    ▼
LKP_DIM_Taxpayer       ← HUB_Taxpayer_HK → DIM_Taxpayer_SK
    ▼
LKP_DIM_PaymentMethod  ← PaymentMethod → DIM_PaymentMethod_SK
    ▼
LKP_DIM_Status         ← Status → DIM_Status_SK
    ▼
ROW_CNT —→ OLE_DST_FACT_Payment
```

Lookup Configurations (all 3: Full cache, Ignore failure):

Lookup	Connection	SQL Query	Join	Output
LKP_DIM_Taxpayer	CM_DV_Gold	SELECT HUB_Taxpayer_HK, DIM_Taxpayer_SK FROM DIM_Taxpayer WHERE IsCurrent = 1	HUB_Taxpayer_HK	DIM_Taxpayer_SK
LKP_DIM_PaymentMethod	CM_DV_Gold	SELECT PaymentMethod, DIM_PaymentMethod_SK FROM DIM_PaymentMethod	PaymentMethod	DIM_PaymentMethod_SK
LKP_DIM_Status	CM_DV_Gold	SELECT StatusCode, DIM_Status_SK FROM DIM_Status	Status → StatusCode	DIM_Status_SK

OLE_DST: CM_DV_Gold, dbo.FACT_Payment, fast load, table lock.

Column Mappings:

Input Column	Destination Column	Source
(not mapped, IDENTITY)	FACT_Payment_SK	Auto-generated
DIM_Taxpayer_SK	DIM_Taxpayer_SK	LKP_DIM_Taxpayer
DIM_PaymentMethod_SK	DIM_PaymentMethod_SK	LKP_DIM_PaymentMethod
DIM_Status_SK	DIM_Status_SK	LKP_DIM_Status
PaymentID	PaymentID	OLE_SRC
DeclarationID	DeclarationID	OLE_SRC
AnnualDeclarationID	AnnualDeclarationID	OLE_SRC
PaymentAmount	PaymentAmount	OLE_SRC
PaymentDate	PaymentDate	OLE_SRC
(not mapped, DEFAULT)	GLD_LoadDate	GETDATE()

© 2026 Mr. Sot So — Royal University of Phnom Penh

GLD_BatchID	GLD_BatchID	OLE_SRC (DECLARE)
GLD_SnapshotDate	GLD_SnapshotDate	OLE_SRC (DECLARE)

OnError: 3-step pattern (SCR_CaptureError → SQL_EndStepOnError → SQL_LogErrorOnError). @PackageName = 'GLD_FACT_Payment'.

CONFIDENTIAL

6.7.3 GLD_FACT_MonthlySnapshot.dtsx, Clone from MonthlyDeclaration

⚠ **IMPORTANT:** This is the ONLY Fact package that reads from DV_Silver (not DV_Bronze). Use CM_DV_Silver for OLE_SRC connection manager. The source is BUS_MonthlyMetrics from the Silver layer.

Variable Changes:

Variable	New Value
`v_TableName`	`FACT_MonthlySnapshot`

Expressions auto-update: v_StepName → Load_FACT_MonthlySnapshot, v_DestTable → FACT_MonthlySnapshot, v_DeleteSQL auto-updates table name.

CONFIDENTIAL

v_SourceSQL Expression (EvaluateAsExpression = True):

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + ";"
+ " DECLARE @GLD_SnapshotDate DATETIME = '"
+ (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate]
+ "';"
+ " SELECT"
+ "   bm.HUB_Taxpayer_HK,"
+ "   bm.MetricMonth,"
+ "   bm.MetricYear,"
+ "   bm.TotalDeclarations,"
+ "   bm.TotalGrossRevenue,"
+ "   bm.TotalTaxAmount,"
+ "   bm.TotalPayments,"
+ "   bm.TotalPaymentAmount,"
+ "   bm.PaymentCoverageRate,"
+ "   @GLD_BatchID AS GLD_BatchID,"
+ "   @GLD_SnapshotDate AS GLD_SnapshotDate"
+ " FROM DV_Silver.dbo.BUS_MonthlyMetrics bm"
+ " WHERE bm.SnapshotDate = @GLD_SnapshotDate"
```

✓ **COPY-PASTE READY:** Copy this single-line expression into v_SourceSQL Expression property:

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + "; DECLARE @GLD_SnapshotDate DATETIME
= '" + (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate] + "'; SELECT bm.HUB_Taxpayer_HK, bm.MetricMonth,
bm.MetricYear, bm.TotalDeclarations, bm.TotalGrossRevenue, bm.TotalTaxAmount, bm.TotalPayments,
bm.TotalPaymentAmount, bm.PaymentCoverageRate, @GLD_BatchID AS GLD_BatchID, @GLD_SnapshotDate AS
GLD_SnapshotDate FROM DV_Silver.dbo.BUS_MonthlyMetrics bm WHERE bm.SnapshotDate = @GLD_SnapshotDate"
```

At runtime (p_BatchID = 1, p_SnapshotDate = 2026-02-22):

```
DECLARE @GLD_BatchID INT = 1;
DECLARE @GLD_SnapshotDate DATETIME = '2026-02-22';
SELECT
    bm.HUB_Taxpayer_HK,
    bm.MetricMonth, bm.MetricYear,
    bm.TotalDeclarations, bm.TotalGrossRevenue, bm.TotalTaxAmount,
    bm.TotalPayments, bm.TotalPaymentAmount, bm.PaymentCoverageRate,
    @GLD_BatchID AS GLD_BatchID,
    @GLD_SnapshotDate AS GLD_SnapshotDate
FROM DV_Silver.dbo.BUS_MonthlyMetrics bm
WHERE bm.SnapshotDate = @GLD_SnapshotDate
```

✓ **TIP:** The WHERE clause filters BUS_MonthlyMetrics by snapshot date so only the current snapshot's pre-aggregated metrics are loaded into the Fact table.

© 2026 Mr. Sot So — Royal University of Phnom Penh

OLE_SRC Configuration:

Property	Value
OLE DB Connection Manager	CM_DV_Silver (NOT CM_DV_Bronze)
Data access mode	SQL command from variable
Variable name	`User::v_SourceSQL`

⚠ **IMPORTANT:** This is the ONLY place in the entire Gold layer that uses CM_DV_Silver. All other Gold OLE_SRC components use CM_DV_Bronze. Make sure to change the connection manager when cloning from MonthlyDeclaration.

OLE_SRC Output Columns (11 columns):

Column	Data Type	Purpose
`HUB_Taxpayer_HK`	DT_BYTES (32)	LKP_DIM_Taxpayer join key
`MetricMonth`	DT_I4	Fact measure
`MetricYear`	DT_I4	Fact measure
`TotalDeclarations`	DT_I4	Fact measure
`TotalGrossRevenue`	DT_NUMERIC (18,2)	Fact measure
`TotalTaxAmount`	DT_NUMERIC (18,2)	Fact measure
`TotalPayments`	DT_I4	Fact measure
`TotalPaymentAmount`	DT_NUMERIC (18,2)	Fact measure
`PaymentCoverageRate`	DT_NUMERIC (5,2)	Fact measure
`GLD_BatchID`	DT_I4	Audit (DECLARE)
`GLD_SnapshotDate`	DT_DBTIMESTAMP	Audit (DECLARE)

DFT_Load_Fact Data Flow (4 components, only 1 Lookup):

```
OLE_SRC (CM_DV_Silver, v_SourceSQL)
    ▼
LKP_DIM_Taxpayer       ← HUB_Taxpayer_HK → DIM_Taxpayer_SK
    ▼
ROW_CNT —→ OLE_DST_FACT_MonthlySnapshot
```

✓ **TIP:** Only 1 Lookup needed. BUS_MonthlyMetrics already contains pre-aggregated metrics per Taxpayer, no Category or Status dimension resolution required.

LKP_DIM_Taxpayer: CM_DV_Gold, Full cache, Ignore failure. SQL: SELECT HUB_Taxpayer_HK, DIM_Taxpayer_SK FROM DIM_Taxpayer WHERE IsCurrent = 1. Join: HUB_Taxpayer_HK. Output: DIM_Taxpayer_SK.

OLE_DST: CM_DV_Gold, dbo.FACT_MonthlySnapshot, fast load, table lock.

Column Mappings:

Input Column	Destination Column	Source
(not mapped, IDENTITY)	`FACT_MonthlySnapshot_SK`	Auto-generated
`DIM_Taxpayer_SK`	`DIM_Taxpayer_SK`	LKP_DIM_Taxpayer
`MetricMonth`	`MetricMonth`	OLE_SRC
`MetricYear`	`MetricYear`	OLE_SRC
`TotalDeclarations`	`TotalDeclarations`	OLE_SRC
`TotalGrossRevenue`	`TotalGrossRevenue`	OLE_SRC
`TotalTaxAmount`	`TotalTaxAmount`	OLE_SRC
`TotalPayments`	`TotalPayments`	OLE_SRC
`TotalPaymentAmount`	`TotalPaymentAmount`	OLE_SRC
`PaymentCoverageRate`	`PaymentCoverageRate`	OLE_SRC
(not mapped, DEFAULT)	`GLD_LoadDate`	GETDATE()
`GLD_BatchID`	`GLD_BatchID`	OLE_SRC (DECLARE)

© 2026 Mr. Sot So — Royal University of Phnom Penh

GLD_SnapshotDate	GLD_SnapshotDate	OLE_SRC (DECLARE)

OnError: 3-step pattern. @PackageName = 'GLD_FACT_MonthlySnapshot'.

CONFIDENTIAL

6.7.4 GLD_FACT_DeclarationLifecycle.dtsx, Clone from MonthlyDeclaration

⚠ **IMPORTANT:** This Fact package does NOT filter by SAT_EndDate IS NULL. It reads ALL SAT_MonthlyDecl historical versions to capture the full lifecycle of status changes per declaration. Each SAT version is a lifecycle event (e.g., Filed → Reviewed → Approved).

Variable Changes:

Variable	New Value
`v_TableName`	`FACT_DeclarationLifecycle`

Expressions auto-update: v_StepName → Load_FACT_DeclarationLifecycle, v_DestTable → FACT_DeclarationLifecycle, v_DeleteSQL auto-updates table name.

CONFIDENTIAL

v_SourceSQL Expression (EvaluateAsExpression = True):

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + ";"
+ " DECLARE @GLD_SnapshotDate DATETIME = '"
+ (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate]
+ "';"
+ " SELECT"
+ "   lnk.HUB_Taxpayer_HK,"
+ "   sd.Status,"
+ "   hd.DeclarationID,"
+ "   sd.Status AS LifecycleEvent,"
+ "   sd.SAT_LoadDate AS EventDate,"
+ "   @GLD_BatchID AS GLD_BatchID,"
+ "   @GLD_SnapshotDate AS GLD_SnapshotDate"
+ " FROM DV_Bronze.dbo.SAT_MonthlyDecl sd"
+ " INNER JOIN DV_Bronze.dbo.HUB_Declaration hd"
+ "   ON sd.HUB_Declaration_HK = hd.HUB_Declaration_HK"
+ " INNER JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk"
+ "   ON hd.HUB_Declaration_HK = lnk.HUB_Declaration_HK"
```

✓ **COPY-PASTE READY:** Copy this single-line expression into v_SourceSQL Expression property:

```
"DECLARE @GLD_BatchID INT = " + (DT_WSTR,10)@[$Package::p_BatchID] + "; DECLARE @GLD_SnapshotDate DATETIME
= '" + (DT_WSTR,30)(DT_DBDATE)@[$Package::p_SnapshotDate] + "'; SELECT lnk.HUB_Taxpayer_HK, sd.Status,
hd.DeclarationID, sd.Status AS LifecycleEvent, sd.SAT_LoadDate AS EventDate, @GLD_BatchID AS GLD_BatchID,
@GLD_SnapshotDate AS GLD_SnapshotDate FROM DV_Bronze.dbo.SAT_MonthlyDecl sd INNER JOIN
DV_Bronze.dbo.HUB_Declaration hd ON sd.HUB_Declaration_HK = hd.HUB_Declaration_HK INNER JOIN
DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk ON hd.HUB_Declaration_HK = lnk.HUB_Declaration_HK"
```

At runtime (p_BatchID = 1, p_SnapshotDate = 2026-02-22):

```
DECLARE @GLD_BatchID INT = 1;
DECLARE @GLD_SnapshotDate DATETIME = '2026-02-22';
SELECT
    lnk.HUB_Taxpayer_HK,
    sd.Status,
    hd.DeclarationID,
    sd.Status AS LifecycleEvent,
    sd.SAT_LoadDate AS EventDate,
    @GLD_BatchID AS GLD_BatchID,
    @GLD_SnapshotDate AS GLD_SnapshotDate
FROM DV_Bronze.dbo.SAT_MonthlyDecl sd
INNER JOIN DV_Bronze.dbo.HUB_Declaration hd
    ON sd.HUB_Declaration_HK = hd.HUB_Declaration_HK
INNER JOIN DV_Bronze.dbo.LNK_TaxpayerDeclaration lnk
    ON hd.HUB_Declaration_HK = lnk.HUB_Declaration_HK
```

⚠ **IMPORTANT:** No WHERE sat.SAT_EndDate IS NULL filter. This is intentional, every SAT version is a status change event. A Declaration with 3 SAT versions (Filed, Reviewed, Approved) produces 3 rows in FACT_DeclarationLifecycle. The SAT_LoadDate becomes the EventDate for each lifecycle event.

© 2026 Mr. Sot So — Royal University of Phnom Penh

OLE_SRC Output Columns (7 columns):

Column	Data Type	Purpose
HUB_Taxpayer_HK	DT_BYTES (32)	LKP_DIM_Taxpayer join key
Status	DT_STR (50)	LKP_DIM_Status join key
DeclarationID	DT_I4	Fact degenerate dimension
LifecycleEvent	DT_STR (50)	Fact measure (= Status alias)
EventDate	DT_DBTIMESTAMP	Fact measure (= SAT_LoadDate)
GLD_BatchID	DT_I4	Audit (DECLARE)
GLD_SnapshotDate	DT_DBTIMESTAMP	Audit (DECLARE)

DFT_Load_Fact Data Flow (5 components, 2 Lookups):

```
OLE_SRC (CM_DV_Bronze, v_SourceSQL)
    ▼
LKP_DIM_Taxpayer       ← HUB_Taxpayer_HK → DIM_Taxpayer_SK
    ▼
LKP_DIM_Status         ← Status → DIM_Status_SK
    ▼
ROW_CNT —→ OLE_DST_FACT_DeclarationLifecycle
```

Lookup Configurations (both: Full cache, Ignore failure):

Lookup	Connection	SQL Query	Join	Output
LKP_DIM_Taxpayer	CM_DV_Gold	SELECT HUB_Taxpayer_HK, DIM_Taxpayer_SK FROM DIM_Taxpayer WHERE IsCurrent = 1	HUB_Taxpayer_HK	DIM_Taxpayer_SK
LKP_DIM_Status	CM_DV_Gold	SELECT StatusCode, DIM_Status_SK FROM DIM_Status	Status → StatusCode	DIM_Status_SK

✓ **TIP:** Only 2 Lookups needed. No DIM_Category lookup because DeclarationLifecycle tracks status change events, not declaration attributes.

OLE_DST: CM_DV_Gold, dbo.FACT_DeclarationLifecycle, fast load, table lock.

Column Mappings:

Input Column	Destination Column	Source
(not mapped, IDENTITY)	FACT_DeclarationLifecycle_SK	Auto-generated
DIM_Taxpayer_SK	DIM_Taxpayer_SK	LKP_DIM_Taxpayer
DIM_Status_SK	DIM_Status_SK	LKP_DIM_Status
DeclarationID	DeclarationID	OLE_SRC
LifecycleEvent	LifecycleEvent	OLE_SRC (sd.Status alias)
EventDate	EventDate	OLE_SRC (sd.SAT_LoadDate alias)
(not mapped, DEFAULT)	GLD_LoadDate	GETDATE()
GLD_BatchID	GLD_BatchID	OLE_SRC (DECLARE)
GLD_SnapshotDate	GLD_SnapshotDate	OLE_SRC (DECLARE)

OnError: 3-step pattern. @PackageName = 'GLD_FACT_DeclarationLifecycle'.

© 2026 Mr. Sot So — Royal University of Phnom Penh

Fact Package Comparison Summary

Package	Source Layer	OLE_SRC Connection	Lookups	Key Difference
FACT_MonthlyDeclaration	Bronze	`CM_DV_Bronze`	3 (Taxpayer, Category, Status)	TEMPLATE, SAT_EndDate IS NULL
FACT_Payment	Bronze	`CM_DV_Bronze`	3 (Taxpayer, PaymentMethod, Status)	2-hop Link chain for Taxpayer resolution
FACT_MonthlySnapshot	Silver	`CM_DV_Silver`	1 (Taxpayer only)	ONLY Fact reading from DV_Silver
FACT_DeclarationLifecycle	Bronze	`CM_DV_Bronze`	2 (Taxpayer, Status)	ALL SAT versions (no EndDate filter)

6.8 Gold Package Cloning, Quick Reference

SCD Type 1 Cloning (5 packages)

Package	v_TableName	Pattern	Key Notes
DIM_Category	`Category`	Full 6-component	TEMPLATE, build first
DIM_Structure	`Structure`	Full 6-component	Same as Category
DIM_Activity	`Activity`	Full 6-component	Same as Category
DIM_PaymentMethod	`PaymentMethod`	Simplified 4-component	No Hub, no CSPL/OLE_CMD
DIM_Status	`Status`	Simplified 4-component	Multi-SAT UNION source

SCD Type 2 Cloning (2 packages)

Package	v_TableName	Tracked Attributes
DIM_Taxpayer	`Taxpayer`	LegalBusinessName, TradingName, EstimatedAnnualRevenue, IsActive
DIM_Officer	`Officer`	FirstName, LastName, Department, IsActive

Fact Package Cloning (4 packages)

Package	v_TableName	Source	Lookups
FACT_MonthlyDeclaration	`FACT_MonthlyDeclaration`	Bronze	Taxpayer, Category, Status
FACT_Payment	`FACT_Payment`	Bronze	Taxpayer, PaymentMethod, Status
FACT_MonthlySnapshot	`FACT_MonthlySnapshot`	Silver	Taxpayer only
FACT_DeclarationLifecycle	`FACT_DeclarationLifecycle`	Bronze	Taxpayer, Status

6.9 Stored Procedures (Alternative Approach)

The DFT-based approach (Sections 6.5–8) is the primary implementation. As an alternative, the same logic can be implemented as stored procedures, which is particularly useful for SCD Type 1 (MERGE statement) and complex Fact join chains.

When to Consider Stored Procedures

SCD1 dimensions: MERGE statement handles insert + update in one atomic statement.
SCD2 dimensions: Multi-step UPDATE + INSERT benefits from transaction isolation in a SP.
Complex Fact joins: When the join chain spans 7+ tables, a SP keeps logic in one place.

© 2026 Mr. Sot So — Royal University of Phnom Penh

If using SPs, replace DFT_Load + SQL_UpdateChanged/SQL_CloseOldVersions with a single SQL_LoadSP task.

6.10 OnError Handler (Gold Package Level)

Every Gold child package (Dimension and Fact) has an OnError event handler. The pattern is identical to all other layers.

OnError Event Handler Tasks (3 steps)

Step	Task	Description
1	SCR_CaptureError (Script Task)	Captures System::ErrorDescription into User::v_ErrorMessage
2	SQL_EndStepOnError (Execute SQL Task)	Calls usp_EndStep(@StepID, 'Failed', 0) to close the step as failed.
3	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError(@BatchID, 'Error', @ErrorSource, @ErrorMessage) to record error details.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on all 3 OnError tasks. See Section 4.13 for details.

Configuration is identical to Bronze OnError handler (Section 4.13). Refer to that section for the full step-by-step Script Task code and Execute SQL Task parameter mappings.

Error Propagation Chain for Gold

Gold child OnError (SCR_CaptureError → SQL_EndStepOnError → SQL_LogErrorOnError) → GLD_Load_All_Dimensions/Facts OnError → Master package OnError.

If a dimension package fails, subsequent dimension packages will not execute (default).

CRITICAL: If any dimension fails, the fact orchestrator should NOT execute.

This is enforced at the Master level: EPT_GLD_Load_All_Dimensions must succeed before EPT_GLD_Load_All_Facts.

Fact tables depend on dimension surrogate keys — loading facts with missing dimensions causes data integrity issues.

6.11 OnError Handler (Orchestrator Level, Gold Layer)

The Gold layer has two orchestrators (GLD_Load_All_Dimensions.dtsx and GLD_Load_All_Facts.dtsx), each with its own OnError handler. Same pattern as all other orchestrators: usp_LogError only, no usp_EndStep.

© 2026 Mr. Sot So — Royal University of Phnom Penh

OnError Event Handler Tasks (2 steps, both orchestrators)

#	Task	Description
1	SCR_CaptureError (Script Task)	Reads System::ErrorDescription into User::v_ErrorMessage.
2	SQL_LogErrorOnError (Execute SQL Task)	Calls usp_LogError only. No usp_EndStep.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on both OnError tasks.

Step 1: SCR_CaptureError (Script Task)

Identical to all other layers. Reads System::ErrorDescription into User::v_ErrorMessage.

Step 2: SQL_LogErrorOnError (Execute SQL Task)

General Tab

Property	Value
Name	SQL_LogErrorOnError
Connection	CM_ETL_Control_OnError
SQLSourceType	Direct input
ResultSet	None

SQLStatement, GLD_Load_All_Dimensions.dtsx

```
SET NOCOUNT ON;

EXEC dbo.usp_LogError
    @BatchID       = ?,
    @ErrorSeverity = 'Error',
    @ErrorSource   = ?,
    @ErrorMessage  = ?;
```

v_ErrorSource, Automatic via Expression

v_ErrorSource uses EvaluateAsExpression = True with expression: @[System::PackageName].
At runtime, SSIS resolves this to the current package name (e.g., GLD_Load_All_Dimensions).
No manual change needed when cloning orchestrator packages.

CONFIDENTIAL

Parameter Mapping Tab (same for both)

Variable Name	Direction	Data Type	Param #	Maps To
$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_ErrorSource	Input	NVARCHAR	1	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	2	@ErrorMessage

Gold Orchestrator, Critical Dimension-Before-Facts Dependency

GLD_Load_All_Dimensions.dtsx runs 7 dimension packages (5 SCD1 + 2 SCD2).
GLD_Load_All_Facts.dtsx runs 4 fact packages.

If a dimension orchestrator fails, the fact orchestrator MUST NOT run.
This is enforced at the Master_Full_Load.dtsx level with a Success constraint:
EPT_GLD_Load_All_Dimensions ──[Success]──→ EPT_GLD_Load_All_Facts

Within GLD_Load_All_Dimensions.dtsx:
SCD1 dimensions are independent of each other — Completion constraints are acceptable.
SCD2 dimensions (DIM_Taxpayer, DIM_Officer) are independent of SCD1 dimensions.
However, if ANY dimension fails, consider the entire dimension load as failed
to prevent fact tables from loading with missing surrogate keys (FK violations or -1 Unknown rows).

Package Variable Requirements

Each Gold orchestrator needs two variables for OnError:
User::v_ErrorMessage (String, default empty)
User::v_ErrorSource (String, Expression: @[System::PackageName])

GLD_Load_All_Dimensions.dtsx receives: p_BatchID only.
GLD_Load_All_Facts.dtsx receives: p_BatchID + p_SnapshotDate.

© 2026 Mr. Sot So — Royal University of Phnom Penh

7. Package Variables & Parameters

7.1 Master_Complete_Pipeline.dtsx Variables

Variable Name	Data Type	Default Value	Purpose
User::v_BatchID	Int32	0	Output from usp_StartBatch, passed to children
User::v_LoadType	String	(expression)	Expression: @[$Project::pp_LoadType]. Mirrors project parameter.
User::v_ProcessName	String	Master_ETL	Process name for batch registration
User::v_BatchStatus	String	Running	Tracks batch status: Running → Success / Failed
User::v_ErrorCount	Int32	0	Accumulated error counter
User::v_SnapshotDate	DateTime	(today)	Snapshot date for Silver/Gold layers
User::v_ErrorMessage	String	(empty)	Captured by OnError event handler

7.2 Master_Full_Load.dtsx Variables

Variable Name	Data Type	Default Value	Purpose
User::v_BatchID	Int32	0	Received from parent package
User::v_LoadType	String	FULL	Received from parent, always 'FULL'
User::v_SnapshotDate	DateTime	(today)	Received from parent
User::v_ErrorCount	Int32	0	Layer-level error counter
User::v_ErrorMessage	String	(empty)	Captured by OnError event handler

7.3 Master_Incremental_Load.dtsx Variables

Variable Name	Data Type	Default Value	Purpose
User::v_BatchID	Int32	0	Received from parent package
User::v_LoadType	String	INCREMENTAL	Received from parent, always 'INCREMENTAL'
User::v_SnapshotDate	DateTime	(today)	Received from parent
User::v_ErrorCount	Int32	0	Layer-level error counter
User::v_ErrorMessage	String	(empty)	Captured by OnError event handler

© 2026 Mr. Sot So — Royal University of Phnom Penh

8. Master_Full_Load.dtsx, Detailed Implementation

This package orchestrates a complete full load across all four data warehouse layers. It receives BatchID and LoadType from Master_Complete_Pipeline.dtsx and never creates its own batch.

8.1 Package Parameters (received from parent)

Parameter Name	Data Type	Required	Description
p_BatchID	Int32	Yes	Batch ID from Master_Complete_Pipeline
p_LoadType	String	Yes	Always 'FULL' for this package
p_SnapshotDate	DateTime	Yes	Date for Silver PITs and Gold Facts

8.2 Control Flow Design

The control flow follows the strict layer sequence: Staging, then Bronze, then Silver, then Gold.

Step	Task / Container	Description
1	EPT_STG_Full_Load_All (Execute Package Task)	Full load all 9 staging tables. Passes p_BatchID and p_LoadType='FULL'.
2	EPT_BRZ_Load_All_Hubs (Execute Package Task)	Load 9 Hubs. Passes p_BatchID.
3	EPT_BRZ_Load_All_Satellites (Execute Package Task)	Load 9 Satellites. Passes p_BatchID.
4	EPT_BRZ_Load_All_Links (Execute Package Task)	Load 5 Links. Passes p_BatchID.
5	EPT_SLV_Load_All (Execute Package Task)	Load 3 PITs, 1 Bridge, 2 Business tables. Passes p_BatchID and p_SnapshotDate.
6	EPT_GLD_Load_All_Dimensions (Execute Package Task)	Load 7 Dimensions. Passes p_BatchID.
7	EPT_GLD_Load_All_Facts (Execute Package Task)	Load 4 Facts. Passes p_BatchID and p_SnapshotDate.

8.3 Step 1: EPT_STG_Full_Load_All

This is the critical step where both BatchID and LoadType are passed to the staging orchestrator.

Parameter Bindings Tab

Child Parameter	Bind to Variable/Parameter	Direction	Data Type

© 2026 Mr. Sot So — Royal University of Phnom Penh

$Package::p_BatchID	$Package::p_BatchID	Input	Int32
$Package::p_LoadType	$Package::p_LoadType	Input	String

Parameter Propagation for Staging

Master_Full_Load.dtsx passes LoadType='FULL' to STG_Full_Load_All.dtsx.

STG_Full_Load_All.dtsx then calls each individual STG_*.dtsx package, passing both BatchID and LoadType.

Each individual staging procedure (e.g., usp_Load_STG_Category) checks LoadType to decide: TRUNCATE + full reload vs. watermark-based delta.

8.4 Steps 2-4: Bronze Layer Packages

All three Bronze packages (Hubs, Satellites, Links) receive only BatchID. They do not need LoadType because Data Vault loads are always insert-only (no truncation).

Parameter Bindings (same for all three)

Child Parameter	Bind to Variable/Parameter	Direction	Data Type
$Package::p_BatchID	$Package::p_BatchID	Input	Int32

8.5 Steps 5-7: Silver and Gold Layer Packages

Silver and Gold packages require both BatchID and SnapshotDate because PIT tables and Fact tables are snapshot-based.

Parameter Bindings (SLV_Load_All, GLD_Load_All_Facts)

Child Parameter	Bind to Variable/Parameter	Direction	Data Type
$Package::p_BatchID	$Package::p_BatchID	Input	Int32
$Package::p_SnapshotDate	$Package::p_SnapshotDate	Input	DateTime

Parameter Bindings (GLD_Load_All_Dimensions)

Child Parameter	Bind to Variable/Parameter	Direction	Data Type
$Package::p_BatchID	$Package::p_BatchID	Input	Int32

CONFIDENTIAL

8.6 OnError Event Handler

Master_Full_Load.dtsx has its own OnError event handler that captures errors at this level. Because it does NOT own the batch, it does NOT call usp_EndBatch. Instead, it logs errors via usp_LogError and lets the error propagate up to Master_Complete_Pipeline.dtsx.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on all OnError tasks.

SQL_LogErrorOnError (Execute SQL Task inside OnError)

Connection: CM_ETL_Control_OnError (separate connection for OnError tasks).

```
SET NOCOUNT ON;

EXEC ETL_Control.dbo.usp_LogError
    @BatchID = ?,
    @ErrorSeverity = 'High',
    @ErrorSource = ?,
    @ErrorMessage = ?;
```

Parameter Mapping Tab

Variable Name	Direction	Data Type	Param #	Maps To
$Package::p_BatchID	Input	LONG	0	@BatchID
User::v_ErrorSource	Input	NVARCHAR	1	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	2	@ErrorMessage

v_ErrorSource uses expression @[System::PackageName] which auto-resolves at runtime. No manual change needed for Master_Incremental_Load.dtsx.

CONFIDENTIAL

9. Master_Incremental_Load.dtsx, Detailed Implementation

This package is structurally identical to Master_Full_Load.dtsx with two key differences: (1) it passes LoadType='INCREMENTAL' to STG_Incremental_Load_All.dtsx instead of STG_Full_Load_All.dtsx, and (2) the staging package uses watermark-based delta extraction instead of truncate-and-reload.

9.1 Control Flow Design

Step	Task / Container	Description
1	EPT_STG_Incremental_Load_All (Execute Package Task)	Incremental load all 9 staging tables via watermarks. Passes p_BatchID and p_LoadType='INCREMENTAL'.
2	EPT_BRZ_Load_All_Hubs (Execute Package Task)	Load 9 Hubs (same as full load). Passes p_BatchID.
3	EPT_BRZ_Load_All_Satellites (Execute Package Task)	Load 9 Satellites. Passes p_BatchID.
4	EPT_BRZ_Load_All_Links (Execute Package Task)	Load 5 Links. Passes p_BatchID.
5	EPT_SLV_Load_All (Execute Package Task)	Load Silver layer. Passes p_BatchID and p_SnapshotDate.
6	EPT_GLD_Load_All_Dimensions (Execute Package Task)	Load 7 Dimensions. Passes p_BatchID.
7	EPT_GLD_Load_All_Facts (Execute Package Task)	Load 4 Facts. Passes p_BatchID and p_SnapshotDate.

9.2 Incremental Load Behavior

When LoadType='INCREMENTAL', the staging control flow uses the ETL_Watermark table to extract only records modified since the last successful load:

```
-- SQL_GetWatermark (Execute SQL Task) reads the last watermark:

SELECT LastLoadDate AS LastWatermark
FROM ETL_Control.dbo.ETL_Watermark
WHERE TableName = 'Category' AND ColumnName = 'UpdatedDate' AND IsActive = 1;

-- DFT_Incremental_Load (OLE DB Source) uses parameterized query with ? placeholder:
SELECT ... FROM TaxSystemDB.dbo.Category WHERE UpdatedDate > ?;  -- ? mapped to
User::v_LastWatermark

-- SQL_UpdateWatermark (Execute SQL Task) advances the watermark:
UPDATE ETL_Control.dbo.ETL_Watermark
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

```
SET LastValue = CONVERT(VARCHAR(500), GETDATE(), 120), LastLoadDate = GETDATE(), UpdatedDate =
GETDATE()
WHERE TableName = 'Category' AND ColumnName = 'UpdatedDate' AND IsActive = 1;
```

9.3 Steps 2-7: Bronze, Silver, Gold

Steps 2 through 7 are identical to Master_Full_Load.dtsx. Bronze, Silver, and Gold layers process whatever data exists in their source layers. The Data Vault insert-only pattern means these layers do not distinguish between full and incremental loads.

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh

10. Master_Complete_Pipeline.dtsx, Detailed Implementation

This is the top-level entry point. A SQL Server Agent job triggers this package. It owns the full batch lifecycle and decides whether to run a full load or incremental load.

10.1 Connection Managers

Name	Type	Connection String	Used By
CM_ETL_Control	OLE DB	Expression-based (pp_ServerName + pp_ETLControlDB)	SQL_StartMasterBatch, SQL_EndMasterBatch
CM_ETL_Control_OnError	OLE DB	Expression-based (pp_ServerName + pp_ETLControlDB)	OnError handler: SQL_EndBatchOnError, SQL_LogErrorOnError
CM_TaxSystemDB	OLE DB	Expression-based (pp_ServerName + pp_TaxSystemDB)	Source system data extraction
CM_DV_Staging	OLE DB	Expression-based (pp_ServerName + DV_Staging)	Staging layer load (STG_*.dtsx)
CM_DV_Bronze	OLE DB	Expression-based (pp_ServerName + DV_Bronze)	Bronze layer load (BRZ_*.dtsx)
CM_DV_Silver	OLE DB	Expression-based (pp_ServerName + DV_Silver)	Silver layer load (SLV_*.dtsx)
CM_DV_Gold	OLE DB	Expression-based (pp_ServerName + DV_Gold)	Gold layer load (GLD_*.dtsx)

10.2 Control Flow Design

The control flow is a linear sequence of tasks with a conditional branch in the middle:

Step	Task / Container	Description
1	SQL_StartMasterBatch (Execute SQL Task)	Calls usp_StartBatch with ProcessName='Master_ETL'. Maps OUTPUT parameter to User::v_BatchID.
2	SCR_ValidateAndInitialize (Script Task)	Validates v_BatchID > 0 and pp_LoadType is FULL or INCREMENTAL. Sets User::v_SnapshotDate = DateTime.Now.
3	EXT_EvaluateLoadType (Expression Task)	Expression: @[$Project::pp_LoadType] == @[$Project::pp_LoadType]. Acts as branch decision point.

© 2026 Mr. Sot So — Royal University of Phnom Penh

4a	EPT_Master_Full_Load (Execute Package Task)	Expression: @[$Project::pp_LoadType] == "FULL". Passes p_BatchID, p_LoadType, p_SnapshotDate.
4b	EPT_Master_Incremental_Load (Execute Package Task)	Expression: @[$Project::pp_LoadType] == "INCREMENTAL". Passes p_BatchID, p_LoadType, p_SnapshotDate.
5	SCR_SetBatchSuccess (Script Task)	Sets v_BatchStatus = "Success". Runs on success from 4a/4b (Logical OR).
6	SQL_EndMasterBatch (Execute SQL Task)	Calls usp_EndBatch with v_BatchID and v_BatchStatus. Connected on success from Step 5.

10.3 Step 1: SQL_StartMasterBatch (Execute SQL Task)

General Tab

- Connection: CM_ETL_Control (expression-based)
- SQLStatement (Direct Input):

```
DECLARE @BatchID INT;
EXEC usp_StartBatch
    @ProcessName = 'Master_ETL',
    @BatchID = @BatchID OUTPUT;
SELECT @BatchID AS BatchID;
```

Result Set Tab

- ResultSet: Single row
- Result Name: BatchID → Variable Name: User::v_BatchID

Important: ResultSet Mapping

Use 'Single row' result set, NOT 'Full result set'.
The Result Name must match the column alias in the SELECT statement.
Map to User::v_BatchID (Int32). This is the only place BatchID is ever created.

10.4 Step 2: SCR_ValidateAndInitialize (Script Task)

This Script Task performs critical input validation and initialization before the pipeline branches. It validates that BatchID was created successfully, that pp_LoadType contains a valid value, and sets the SnapshotDate.

Script Task Configuration

- Name: SCR_ValidateAndInitialize
- ScriptLanguage: Microsoft Visual C# 2022

© 2026 Mr. Sot So — Royal University of Phnom Penh

- ReadOnlyVariables: $Project::pp_LoadType, User::v_BatchID
- ReadWriteVariables: User::v_SnapshotDate
- Script (C#):

```
public void Main()
{
    // 1. Validate BatchID was created by usp_StartBatch
    int batchId = (int)Dts.Variables["User::v_BatchID"].Value;
    if (batchId <= 0)
    {
        Dts.Events.FireError(0, "SCR_ValidateAndInitialize",
            "usp_StartBatch returned invalid BatchID: " + batchId,
            "", 0);
        Dts.TaskResult = (int)ScriptResults.Failure;
        return;
    }

    // 2. Validate pp_LoadType is FULL or INCREMENTAL
    string loadType = (string)Dts.Variables["$Project::pp_LoadType"].Value;
    if (loadType != "FULL" && loadType != "INCREMENTAL")
    {
        Dts.Events.FireError(0, "SCR_ValidateAndInitialize",
            "Invalid pp_LoadType: '" + loadType + "'. Must be 'FULL' or 'INCREMENTAL'.",
            "", 0);
        Dts.TaskResult = (int)ScriptResults.Failure;
        return;
    }

    // 3. Set SnapshotDate
    Dts.Variables["User::v_SnapshotDate"].Value = DateTime.Now;

    Dts.TaskResult = (int)ScriptResults.Success;
}
```

What Happens on Validation Failure

FireError() triggers the package-level OnError event handler.
SCR_CaptureError catches the error and sets v_BatchStatus = 'Failed'.
SQL_EndBatchOnError closes the batch with Status = 'Failed'. SQL_LogErrorOnError logs the error message.
The pipeline stops immediately, no EPT tasks execute.

10.5 Step 3: EXT_EvaluateLoadType (Expression Task)

An Expression Task acts as the branch decision point. It evaluates a simple expression and is the origin for the two expression-based precedence constraints that route to the correct child master package.

© 2026 Mr. Sot So — Royal University of Phnom Penh

Expression Task Configuration

- Name: EXT_EvaluateLoadType
- Expression: @[$Project::pp_LoadType] == @[$Project::pp_LoadType]

Why Use an Expression Task?

The Expression Task provides a clean single branch point in the control flow.
Both precedence constraints (FULL and INCREMENTAL) originate from this one task.
This keeps the control flow visually clean and logically organized in the SSIS designer.
The expression itself always evaluates to TRUE, the real routing logic is in the precedence constraints.

10.6 Step 4: Branch, Precedence Constraints

Two expression-based precedence constraints originate from the Expression Task (Step 3), routing to the correct Execute Package Task:

Target Package	Evaluation Operation	Expression
Master_Full_Load.dtsx	Expression	@[$Project::pp_LoadType] == "FULL"
Master_Incremental_Load.dtsx	Expression	@[$Project::pp_LoadType] == "INCREMENTAL"

10.7 Step 4a/4b: EPT_Master_Full_Load / EPT_Master_Incremental_Load

Both Execute Package Tasks share the same configuration pattern. The only difference is which .dtsx file they reference.

Package Tab (Execute Package Task Editor)

- ReferenceType: Project Reference
- PackageNameFromProjectReference: Master_Full_Load.dtsx (or Master_Incremental_Load.dtsx)
- ExecuteOutOfProcess: False

Parameter Bindings Tab

Child Parameter	Bind to Variable	Direction	Data Type
$Package::p_BatchID	User::v_BatchID	Input	Int32
$Package::p_LoadType	User::v_LoadType	Input	String
$Package::p_SnapshotDate	User::v_SnapshotDate	Input	DateTime

© 2026 Mr. Sot So — Royal University of Phnom Penh

10.8 Step 5: SCR_SetBatchSuccess (Script Task)

This Script Task transitions v_BatchStatus from 'Running' to 'Success' before the batch is closed. It only executes on the success path.

Script Task Configuration

- Name: SCR_SetBatchSuccess
- ScriptLanguage: Microsoft Visual C# 2019
- ReadWriteVariables: User::v_BatchStatus
- Script (C#):

```
public void Main()
{
    Dts.Variables["User::v_BatchStatus"].Value = "Success";
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

Precedence Constraints (Incoming)

This task has TWO incoming constraints from EPT_Master_Full_Load and EPT_Master_Incremental_Load:

From Task	Evaluation	Value
EPT_Master_Full_Load	Completion + Success	Success
EPT_Master_Incremental_Load	Completion + Success	Success

Multiple Constraints: Use Logical OR

Since only ONE of the two EPT tasks executes per run, SCR_SetBatchSuccess must use Logical OR.
Right-click SCR_SetBatchSuccess → Properties → Multiple constraints: Logical OR.
If you use Logical AND (default), this task will NEVER execute because one EPT always skips.

10.9 Step 6: SQL_EndMasterBatch (Execute SQL Task)

General Tab

- Connection: CM_ETL_Control (expression-based)
- SQLStatement:

```
SET NOCOUNT ON;

EXEC usp_EndBatch
    @BatchID = ?,     -- Parameter 0 → User::v_BatchID
    @Status = ?;      -- Parameter 1 → User::v_BatchStatus
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

Parameter Mapping Tab

Variable Name	Direction	Data Type	Parameter Name	Maps To
User::v_BatchID	Input	LONG	0	@BatchID
User::v_BatchStatus	Input	VARCHAR	1	@Status (default = 'Success')

How v_BatchStatus Transitions

v_BatchStatus = 'Running', package starts (default value).
v_BatchStatus = 'Success', Step 5: SCR_SetBatchSuccess sets it on the success path.
v_BatchStatus = 'Failed', OnError: SCR_CaptureError overrides it when any error occurs.
SQL_EndMasterBatch (Step 6) and SQL_EndBatchOnError both read v_BatchStatus via parameter mapping.

10.10 OnError Event Handler

The OnError event handler captures failures and ensures the batch is marked as Failed even if an unhandled exception occurs.

Important: Use CM_ETL_Control_OnError connection + TransactionOption = NotSupported on all 3 OnError tasks (SCR_CaptureError, SQL_EndBatchOnError, SQL_LogErrorOnError).

Event Handler Configuration

- Executable: Master_Complete_Pipeline (package level)
- Event handler: OnError

SCR_CaptureError (Script Task inside OnError)

- ReadOnlyVariables: System::ErrorDescription
- ReadWriteVariables: User::v_ErrorMessage, User::v_BatchStatus
- Script (C#):

```
public void Main()
{
    string errMsg = Dts.Variables["System::ErrorDescription"].Value.ToString();
    Dts.Variables["User::v_ErrorMessage"].Value = errMsg;
    Dts.Variables["User::v_BatchStatus"].Value = "Failed";
    Dts.TaskResult = (int)ScriptResults.Success;
}
```

© 2026 Mr. Sot So — Royal University of Phnom Penh

SQL_EndBatchOnError (Execute SQL Task inside OnError)

- Connection: CM_ETL_Control_OnError (separate connection for OnError tasks).
- Runs after SCR_CaptureError. v_BatchStatus is already set to 'Failed' by the Script Task.
- SQLStatement:

```
SET NOCOUNT ON;

EXEC usp_EndBatch @BatchID = ?, @Status = ?;
```

Parameter Mapping Tab (SQL_EndBatchOnError)

Variable Name	Direction	Data Type	Parameter Name	Maps To
User::v_BatchID	Input	LONG	0	@BatchID
User::v_BatchStatus	Input	VARCHAR	1	@Status

SQL_LogErrorOnError (Execute SQL Task inside OnError)

- Connection: CM_ETL_Control_OnError (separate connection for OnError tasks).
- Runs after SQL_EndBatchOnError. Logs the error details to ETL_ErrorLog.
- SQLStatement:

```
SET NOCOUNT ON;

EXEC usp_LogError
    @BatchID = ?,
    @ErrorSeverity = 'Critical',
    @ErrorSource = ?,
    @ErrorMessage = ?;
```

Parameter Mapping Tab (SQL_LogErrorOnError)

Variable Name	Direction	Data Type	Parameter Name	Maps To
User::v_BatchID	Input	LONG	0	@BatchID
User::v_ErrorSource	Input	NVARCHAR	1	@ErrorSource
User::v_ErrorMessage	Input	NVARCHAR	2	@ErrorMessage

CONFIDENTIAL

11. Complete Parameter Flow Reference

This section shows exactly which parameters flow through the entire hierarchy, from Master_Complete_Pipeline down to the individual staging procedures.

11.1 Full Load Path

From Package	To Package	Parameters Passed
Master_Complete_Pipeline	Master_Full_Load.dtsx	BatchID, LoadType='FULL', SnapshotDate
Master_Full_Load	STG_Full_Load_All.dtsx	BatchID, LoadType='FULL'
STG_Full_Load_All	STG_Category.dtsx (etc.)	BatchID, LoadType='FULL'
STG_Category.dtsx	usp_Load_STG_Category	@BatchID, @LoadType='FULL'
Master_Full_Load	BRZ_Load_All_Hubs.dtsx	p_BatchID
Master_Full_Load	BRZ_Load_All_Satellites.dtsx	p_BatchID
Master_Full_Load	BRZ_Load_All_Links.dtsx	p_BatchID
Master_Full_Load	SLV_Load_All.dtsx	BatchID, SnapshotDate
Master_Full_Load	GLD_Load_All_Dimensions.dtsx	p_BatchID
Master_Full_Load	GLD_Load_All_Facts.dtsx	BatchID, SnapshotDate

11.2 Incremental Load Path

From Package	To Package	Parameters Passed
Master_Complete_Pipeline	Master_Incremental_Load.dtsx	BatchID, LoadType='INCREMENTAL', SnapshotDate
Master_Incremental_Load	STG_Incremental_Load_All.dtsx	BatchID, LoadType='INCREMENTAL'
STG_Incremental_Load_All	STG_Category.dtsx (etc.)	BatchID, LoadType='INCREMENTAL'
STG_Category.dtsx	usp_Load_STG_Category	@BatchID, @LoadType='INCREMENTAL'
Master_Incremental_Load	BRZ_Load_All_Hubs.dtsx	p_BatchID
Master_Incremental_Load	BRZ_Load_All_Satellites.dtsx	p_BatchID
Master_Incremental_Load	BRZ_Load_All_Links.dtsx	p_BatchID
Master_Incremental_Load	SLV_Load_All.dtsx	BatchID, SnapshotDate
Master_Incremental_Load	GLD_Load_All_Dimensions.dtsx	p_BatchID
Master_Incremental_Load	GLD_Load_All_Facts.dtsx	BatchID, SnapshotDate

© 2026 Mr. Sot So — Royal University of Phnom Penh

12. Error Handling Strategy

Error handling follows a layered approach: each package captures and logs errors at its own level, then propagates the failure upward. Only the top-level Master_Complete_Pipeline.dtsx is responsible for calling usp_EndBatch.

12.1 Error Handling Per Package Level

Package	OnError Behavior	Batch Control
Master_Complete_Pipeline	SCR_CaptureError → SQL_EndBatchOnError (usp_EndBatch) → SQL_LogErrorOnError (usp_LogError).	OWNS batch lifecycle
Master_Full_Load	SCR_CaptureError → SQL_LogErrorOnError (usp_LogError only). Propagates failure up.	Does NOT call usp_EndBatch
Master_Incremental_Load	SCR_CaptureError → SQL_LogErrorOnError (usp_LogError only). Propagates failure up.	Does NOT call usp_EndBatch
STG_Full_Load_All	SCR_CaptureError → SQL_LogErrorOnError (usp_LogError only). Propagates up.	Does NOT call usp_EndBatch
Individual STG_*.dtsx	TRY/CATCH in stored procedure calls usp_EndStep with Status='Failed'.	Step-level logging only

12.2 OnError Event Handler Template

Every master and orchestrator package uses the same OnError pattern (your preferred Option 2):

6. **Script Task**, Reads System::ErrorDescription into User::v_ErrorMessage
7. **Execute SQL Task (usp_EndStep or usp_EndBatch)**, Child packages call usp_EndStep to mark step as Failed. Master calls usp_EndBatch.
8. **Execute SQL Task (usp_LogError)**, Separate task to log error details. Never combine with usp_EndStep/usp_EndBatch in one Execute SQL Task.

Never Call usp_EndBatch in Child Packages

Only Master_Complete_Pipeline.dtsx should call usp_EndBatch (via SQL_EndBatchOnError).
If a child package calls usp_EndBatch on failure, the parent package will attempt to

© 2026 Mr. Sot So — Royal University of Phnom Penh

call it again, resulting in incorrect batch status overwrites.
Child packages use SQL_EndStepOnError + SQL_LogErrorOnError (never usp_EndBatch).

CONFIDENTIAL

CONFIDENTIAL

© 2026 Mr. Sot So — Royal University of Phnom Penh